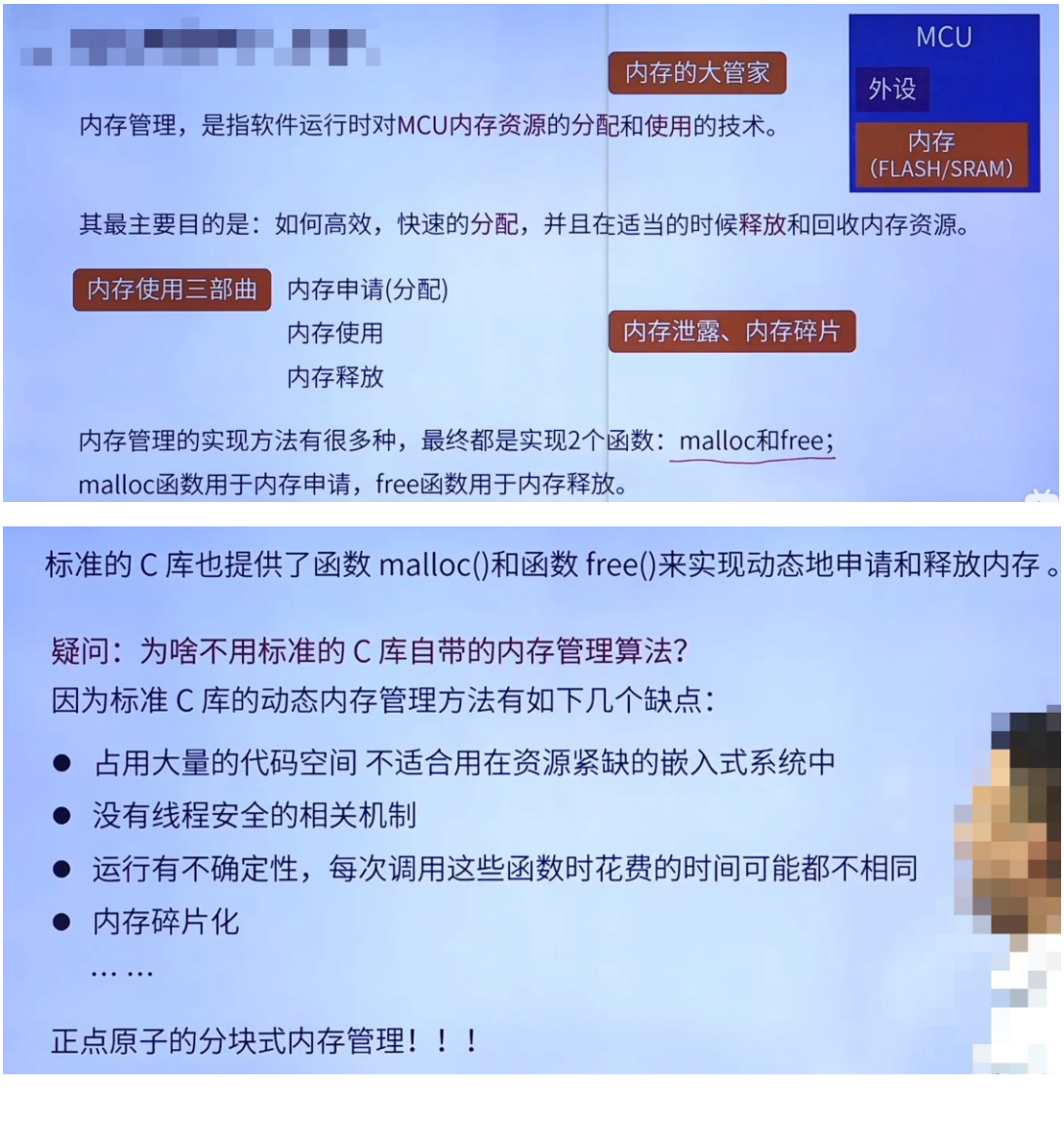
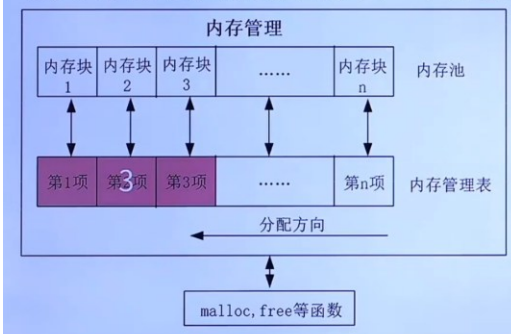
1 内存管理

1.1 内存管理简介



1.2 分块式内存管理介绍

分块式内存管理由内存池和内存管理表两部分组成。内存池被等分为n块，对应的内存管理表，大小也为n，内存管理表的每一个项对应内存池的一块内存。



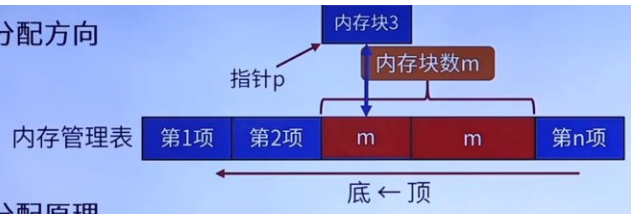
内存管理表的项值代表的意义：

当该项值为0时，代表对应的内存块未被占用；

当该项值非零时，代表该项对应的内存块已经被占用，其数值则代表被连续占用的内存块数。

当内存管理刚初始化的时候，内存管理表全部清零，表示没有任何内存块被占用。

分配方向



x项的值=0?

首先从最末端开始找空内存。

分配原理

当指针p调用malloc申请内存时，

① 先判断p要分配的内存块数(m)

② 从第n项开始，向下查找，直到找到m块连续的空内存块(即对应内存管理表项为0)

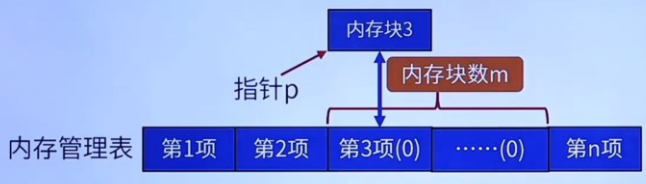
③ 将这m个内存管理表项的值都设置为m（标记被占用）

④ 把最后的这个空内存块的地址返回指针p，完成一次分配

注意：如果当内存不够时(找到最后也没有找到连续m块空闲内存)，则返回NULL给p，表示分配失败。

释放原理

当指针p申请的内存用完，需要释放的时候，调用free函数实现。



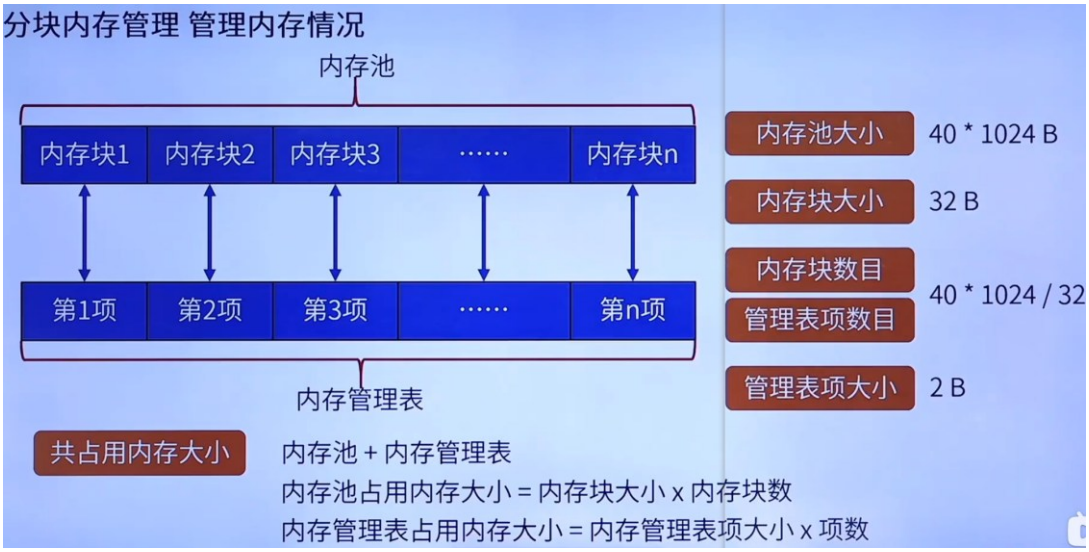
free函数实现：

① 先判断p指向的内存地址所对应的内存块

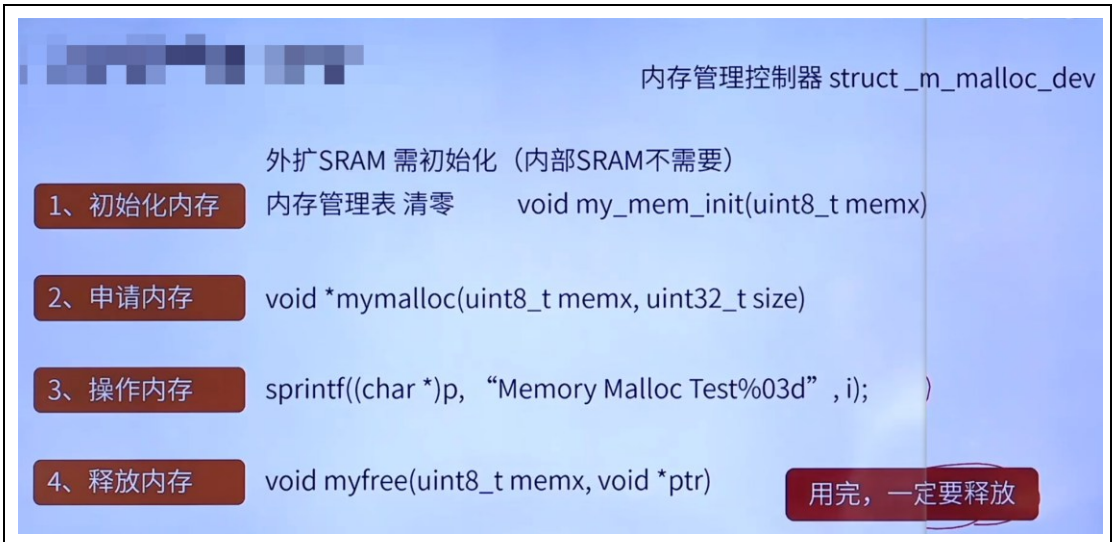
内存管理表项目的值:所分配内存块的数目

② 找到对应的内存管理表项目，得到p所占用的内存块数目m

③ 将这m个内存管理表项目的值都清零，标记释放，完成一次内存释放


分块内存管理 管理内存情况
内存池
内存块1
内存块2
内存块3
……
内存块n
第1项
第2项
第3项
……
第n项
内存管理表
内存池大小
40 * 1024 B
内存块大小
32 B
内存块数目
管理表项数目
40 * 1024 / 32
管理表项大小
2 B
共占用内存大小
内存池 + 内存管理表
内存池占用内存大小 = 内存块大小 x 内存块数
内存管理表占用内存大小 = 内存管理表项大小 x 项数

1.3 内存管理使用


内存管理控制器 struct _m_malloc_dev
外扩SRAM 需初始化（内部SRAM不需要）
1、初始化内存
内存管理表 清零
void my_mem_init(uint8_t memx)
2、申请内存
void *mymalloc(uint8_t memx, uint32_t size)
3、操作内存
sprintf((char *)p, “Memory Malloc Test%03d”, i);
4、释放内存
void myfree(uint8_t memx, void *ptr)
用完，一定要释放

战舰

```
/* 内存管理控制器 */
struct _m_mallco_dev
{
    void    (*init)(uint8_t);
    uint8_t (*perused)(uint8_t);
    uint8_t *membase[SRAMBANK];
    uint16_t *memmap[SRAMBANK];
    uint8_t memrdy[SRAMBANK];
};

struct _m_mallco_dev mallco_dev
{
    my_mem_init,
    my_mem_perused,
    mem1base, mem2base,
    mem1mapbase, mem2mapbase,
    0, 0,
};
```

不同板子，存在不同

```
#define SRAMBANK 2  /* 定义管理的内存片数*/

/* 函数指针，指向内存初始化函数，用于初始化内存管理 */
/* 函数指针，指向内存使用率函数，用于获取内存使用率 */
/* 内存池指针，指向内存池 */
/* 内存管理表指针，指向内存管理表 */
/* 内存管理表就绪标志，用于表示内存管理表是否已初始化 */

/* 内存池 */
static __align(64) uint8_t mem1base[MEM1_MAX_SIZE];
static __align(64) uint8_t mem2base[MEM2_MAX_SIZE];
```

战舰

```
/* 内存管理控制器 */
struct _m_mallco_dev
{
    void    (*init)(uint8_t);
    uint8_t (*perused)(uint8_t);
    uint8_t *membase[SRAMBANK];
    uint16_t *memmap[SRAMBANK];
    uint8_t memrdy[SRAMBANK];
};

struct _m_mallco_dev mallco_dev
{
    my_mem_init,
    my_mem_perused,
    mem1base, mem2base,
    mem1mapbase, mem2mapbase,
    0, 0,
};
```

不同板子，存在不同

```
#define SRAMBANK 2  /* 定义管理的内存片数*/

/* 函数指针，指向内存初始化函数，用于初始化内存管理 */
/* 函数指针，指向内存使用率函数，用于获取内存使用率 */
/* 内存池指针，指向内存池 */
/* 内存管理表指针，指向内存管理表 */
/* 内存管理表就绪标志，用于表示内存管理表是否已初始化 */

        MEMx_MAX_SIZE           内存池中内存块大小
        MEMx_ALLOC_TABLE_SIZE   内存管理表中项数
/* 内存池 */
static __align(64) uint8_t mem1base[MEM1_MAX_SIZE];
static __align(64) uint8_t mem2base[MEM2_MAX_SIZE];
/* 内存管理表 */
static uint16_t mem1mapbase[MEM1_ALLOC_TABLE_SIZE];
static uint16_t mem2mapbase[MEM2_ALLOC_TABLE_SIZE];
```

内存池

uint8_t mem1base[MEM1_MAX_SIZE]

MEM1_ALLOC_TABLE_SIZE个内存块

```
/* mem1内存参数设定.mem1是F103内部的SRAM. */
#define MEM1_BLOCK_SIZE         32
#define MEM1_MAX_SIZE           40 * 1024
#define MEM1_ALLOC_TABLE_SIZE   MEM1_MAX_SIZE/MEM1_BLOCK_SIZE

/* mem2内存参数设定.mem2是F103外扩SRAM */
#define MEM2_BLOCK_SIZE         32
#define MEM2_MAX_SIZE           963 *1024
#define MEM2_ALLOC_TABLE_SIZE   MEM2_MAX_SIZE/MEM2_BLOCK_SIZE
```

内存管理表

uint16_t mem1mapbase[MEM1_ALLOC_TABLE_SIZE]

MEM1_ALLOC_TABLE_SIZE个项

项的大小为2字节

合理分配内存

size = MEM_MAX_SIZE + (MEM_MAX_SIZE / MEM_MAX_BLOCK) * sizeof(MT_TYPE)

MEM_MAX_SIZE = (MEM1_BLOCK_SIZE * size) / (MEM1_BLOCK_SIZE + sizeof(MT_TYPE))

内存管理的内存片数

正点原子板子	内部SRAM		外扩SRAM	外扩SDRAM	内存管理片数
MINI	SRAM		无	无	1
精英	SRAM		无	无	1
战舰	SRAM		有	无	2
探索者	SRAM	CCM	有	无	3
阿波罗F429	SRAM	CCM	无	有	3
阿波罗F767	SRAM	DTCM	无	有	3
阿波罗H743	AXI_SRAM	SRAM1~4	无	有	6
	ITCM	DTCM			
北极星F750	SRAM	ITCM	无	有	4
		DTCM			
北极星H750	AXI_SRAM	SRAM1~4	无	有	6
	ITCM	DTCM			
Mini Pro H750	AXI_SRAM	SRAM1~4	无	无	5
	ITCM	DTCM			

2 SD 卡

2.1 SD 卡介绍

2.1.1 SD 卡简介

SD卡存储容量等级分为四个

SDSC（Secure Digital Standard Capacity）

SDHC（Secure Digital High Capacity）

SDXC（Secure Digital eXtended Capacity）

SDUC（Secure Digital Ultra Capacity）

文件系统：存储设备上组织文件的方法

注意：STM32最大支持32GB SD卡

SD卡类型	协议规范	文件系统	容量等级
SDSC	SD1.0	FAT12,16	上限至2GB
SDHC	SD2.0	FAT32	2GB至32GB
SDXC	SD3.0	exFAT	32GB至2TB
SDUC	SD7.0	exFAT	2TB至128TB

SD规范 https://www.bilibili.com/read/cv18024118/

SD卡速度等级分为：Speed Class、UHS Speed Class和Video Speed Class

	标志	串列数据最低写入速度	SD总线模式	用途
UHS速度等级	U3	30MB/秒	UHS-II UHS-I	4K/2K视频录制
	U1	10MB/秒		全高清视频录制 连续拍摄高清静态影像
Speed Class	Class 10	10MB/秒	高速（HS）	
	Class 6	6MB/秒	普通速度（NS）	高清或全高清视频录制
	Class 4	4MB/秒		
	Class 2	2MB/秒		一般视频录制

Video Speed Class 分为V6、V10、V30、V60、V90对应不同视频格式

UHS Ⅰ　UHS Ⅱ Ⅲ

SD卡驱动方式

微处理器对SD卡进行操作，可通过SPI接口、SDIO接口。不同接口， SD卡引脚功能不一样。

SD卡引脚定义

SDIO接口通信线：
CLK/CMD/DAT0~3

SPI接口通信线：
CS/CLK/MOSI/MISO

引脚编号	引脚名称	功能（SDIO模式）	功能（SPI模式）
Pin 1	DAT3/CS	数据线3	片选信号
Pin 2	CMD/MOSI	命令线	主机输出，从机输入
Pin 3	VSS1	电源地	电源地
Pin 4	VDD	电源	电源
Pin 5	CLK	时钟	时钟
Pin 6	VSS2	电源地	电源地
Pin 7	DAT0/MISO	数据线0	主机输入，从机输出
Pin 8	DAT1	数据线1	保留
Pin 9	DAT2	数据线2	保留

SDIO接口通信线：CLK/CMD/DAT0~3

CLK：时钟线，由SDIO主机产生，由STM32微控制器SDIO外设输出

CMD：命令线，SDIO主机通过该线发送命令控制SD卡，若命令要求SD卡响应，SD卡也是通过该线传输响应信息。

类似IIC的应答信号

DAT0~3：数据线，用于接收或发送数据；SD卡可将DAT0拉低表示处于忙状态

注意：SPI接口的MISO也有该特性

SPI接口通信线：CS/CLK/MOSI/MISO。关于通信线的说明，参考SPI实验

TF卡只比SD卡少了一个电源引脚VSS2，其他引脚功能类似。

TF卡引脚定义

引脚编号	引脚名称	功能（SDIO模式）	功能（SPI模式）
Pin 1	DAT2	数据线2	保留
Pin 2	DAT3/CS	数据线3	片选信号
Pin 3	CMD/MOSI	命令线	主机输出，从机输入
Pin 4	VDD	电源	电源
Pin 5	CLK	时钟	时钟
Pin 6	VSS	电源地	电源地
Pin 7	DAT0/MISO	数据线0	主机输入，从机输出
Pin 8	DAT1	数据线1	保留

SD卡和TF卡只有引脚和形状大小不同，内部结构类似，操作时序完全相同，可用完全相同的代码驱动

2.1.2 SD 卡寄存器简介

SD卡寄存器

SD卡有8个寄存器，但不能直接进行读写操作，需要通过命令来控制。SD卡协议定义了一些命令用于实现某一特定功能，SD卡根据收到的命令要求对内部寄存器进行修改。

名称	宽度(bit)	描述
CID	128	卡标识寄存器，提供制造商ID、OEM/应用ID、产品名称、版本、序列号、制造日期等信息（每个卡都是唯一的）
RCA	16	相对卡地址(Relative card address)寄存器，提供本地系统中卡的地址，动态由卡建议，在主机初始化的时候确定 注意：仅SDIO模式下有，SPI模式下无RCA
CSD	128	卡特定数据寄存器，提供SD卡操作条件相关信息和数据
SCR	64	SD配置寄存器，提供SD卡一些特定的数据
OCR	32	操作条件寄存器，主要是SD卡的操作电压等信息

《Physical Layer Simplified Specification V2.0》P85 第5章内容

OCR寄存器

OCR bit position	OCR Fields Definition
0-3	reserved
4	reserved
5	reserved
6	reserved
7	Reserved for Low Voltage Range
8	reserved
9	reserved
10	reserved
11	reserved
12	reserved
13	reserved
14	reserved
15	2.7-2.8
16	2.8-2.9
17	2.9-3.0
18	3.0-3.1
19	3.1-3.2
20	3.2-3.3
21	3.3-3.4
22	3.4-3.5
23	3.5-3.6
24-29	reserved
30	Card Capacity Status (CCS)[1]
31	Card power up status bit (busy)[2]

VDD Voltage Window

SD卡支持范围中，相应位置1

CCS位为1表示V2.0 SDHC卡

CCS位为0表示V2.0 SDSC卡

CSD寄存器

CSD_STRUCTURE	CSD structure version	Valid for SD Memory Card Physical Specification Version/Card Capacity
0 区分卡类型	CSD Version 1.0	Version 1.01-1.10 Version 2.00/Standard Capacity
1	CSD Version 2.0	Version 2.00/High Capacity
2-3	reserved	

SCR寄存器

Description	Field	Width	Cell Type	SCR Slice
SCR Structure	SCR_STRUCTURE	4	R	[63:60]
SD Memory Card - Spec. Version	SD_SPEC	4	R	[59:56]
data_status_after erases	DATA_STAT_AFTER_ERASE	1	R	[55:55]
SD Security Support	SD_SECURITY	3	R	[54:52]
DAT Bus widths supported	SD_BUS_WIDTHS	4	R	[51:48]
		16	R	[47:32]
		32	R	[31:0]

SD_BUS_WIDTHS	Supported Bus Widths
Bit 0	1 bit (DAT0)
Bit 1	reserved
Bit 2	4 bit (DAT0-3)
Bit 3	reserved

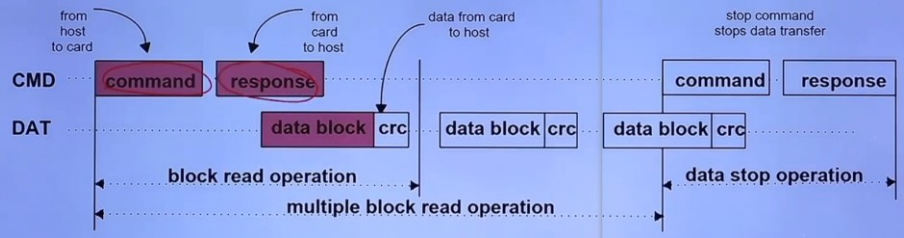
“原子哥”在线教 http://www.o

2.1.3 SDIO 模式

2.1.3.1 SD 卡常用命令介绍

SDIO模式

SD总线上的通信基于命令和数据位流传输。



命令：应用相关命令(ACMD)和通用命令(CMD)，通过命令线CMD传输，固定长度48位

响应：SD卡接收到命令，会有一个响应，用来反应SD卡状态。有2种响应类型：短响应(48位，格式与命令一样)和长响应(136位)。

数据：主机发送的数据 / SD发送的数据。SD数据是以块(Block)形式传输，SDHC卡数据块长度一般为512字节。数据块需要CRC保证数据传输成功。

SD卡命令格式

SD卡的命令格式由6个字节组成，发送数据时高位在前，SD卡的写入命令格式如下：

字节1(cmd)			字节2~5(参数内容)	字节6(校验)	
47	46	45 : 40	39 : 8	7 : 1	0
0	1	command	命令参数	CRC7	1

红色表示固定值

起始位 传输方向 1命令 0响应　　停止位

Byte1：命令字的第一个字节为命令号（如CMD0、CMD1等），格式为“0 1 x x x x x x”

Byte2~Byte5：命令参数，有些命令参数是保留位，没有定义参数的内容，保留位应设置为0

Byte6：用于校验命令传输内容正确性，前7位为CRC(循环冗余校验)校验位，最后一位为停止位0

注意：使用SDIO接口驱动，CRC7校验值必须正确；而SPI接口驱动，CRC7校验默认关闭，即为CRC

SDIO模式

SD卡命令 SDIO模式和SPI模式，可使用的命令和特定类支持的命令有所不同。

SD卡系统的命令被分为多个类：

- 基本命令 Class 0（CMD0/CMD8/CMD9/CMD10/CMD12/CMD2/CMD3）、
- 面向块读取命令 Class 2（CMD16/CMD17/CMD18）、
- 面向块写入命令 Class 4（CMD16/CMD24/CMD25）、
- 擦除命令 Class 5、
- 加锁命令 Class 7、
- 特定于应用命令 Class 8（CMD55）、
- 面向块写保护命令 Class 6、
- I/O模式命令 Class 9、
- SD卡特定应用命令（ACMD41 / ACMD6 / ACMD51）、
- Switch功能命令

强制性

所有SD卡都应支持

SD卡常用命令

SDIO模式

	命令	参数	响应	正点原子官方
SD卡初始化过程	CMD0(0x0)	NONE	无	复位SD卡
	CMD8(0X08)	VHS+Check pattern	R7	主机发送接口状态命令
	ACMD41(0X29)	HCS+VDD电压	R3	主机发送容量支持信息HCS和OCR寄存器内容
	CMD2(0X2)	NONE	R2	读取SD卡的CID寄存器值
	CMD3(0X3)	NONE	R6	要求SD卡发布新的相对地址
	CMD9(0X9)	RCA	R2	获得选定卡的CSD寄存器内容
	CMD7(0X7)	RCA	R1b	选中SD卡
读取	CMD16(0x10)	block length	R1	设置SD卡的块大小(字节)
	CMD17(0X11)	地址	R1	读取一个块的数据
	CMD18(0X12)	地址	R1	连续读取多个块的数据
	CMD12(0X0C)	NONE	R1b	多块读取强制卡停止传输
写入	CMD13(0x0D)	RCA	R1	被选中的卡返回其状态
	ACMD23(0x97)	Number of block	R1	设置需要预擦除的数据块数，提高SD卡多数据块写入性能（速度）
	CMD24(0X18)	地址	R1	写入一个块的数据
	CMD25(0X19)	地址	R1	连续写入多个块的数据
	CMD55(0X37)	NONE	R1	通知SD卡，接下来发送是应用命令

SD卡响应

SDIO模式

SD卡和单片机的通信采用发送应答机制。

每发送一个命令，SD卡都会给出一个应答，以告知主机该命令的执行情况，或者返回主机需要获取的数据。使用SDIO接口时，响应通过CMD线传输。

SD卡响应因使用接口不同，格式也不同。响应具体有R1、R1b、R2、R3、R7。

响应内容大小可以分为短响应48bit和长响应136bit。

响应	长度(SDIO)	描述
R1	48bit	正常响应命令
R1b	48bit	格式与R1相同，可选增加忙信号(数据线0上传输)
R2	136bit	SDIO:根据命令可返回CID/CSD寄存器值
R3	48bit	SDIO:ACMD41命令的响应，返回OCR寄存器值
R6	48bit	已发布的RCA响应
R7	48bit	CMD8命令的响应(卡支持电压信息)

《Physical Layer Simplified Specification V2.0》P69 4.9 SDIO响应内容

SDIO模式

R1响应：如果有传输到卡的数据，那么在数据线0有busy信号(R1b)

Bit position	47	46	[45:40]	[39:8]	[7:1]	0
Width (bits)	1	1	6	32	7	1
Value	'0'	'0'	x	x	x	'1'
Description	start bit	transmission bit	command index	card status	CRC7	end bit

通过响应内容中的command index获知响应哪个命令

R2响应：CID寄存器内容作为CMD2和CMD10响应，CSD寄存器内容作为CMD9响应

Bit position	135	134	[133:128]	[127:1]	0
Width (bits)	1	1	6	127	1
Value	'0'	'0'	'111111'	x	'1'
Description	start bit	transmission bit	reserved	CID or CSD register incl. internal CRC7	end bit

R3响应：OCR寄存器的值作为ACMD41的响应

Bit position	47	46	[45:40]	[39:8]	[7:1]	0
Width (bits)	1	1	6	32	7	1
Value	'0'	'0'	'111111'	x	'1111111'	'1'
Description	start bit	transmission bit	reserved	OCR register	reserved	end bit

R7响应：专用于命令CMD8的响应，返回卡支持电压范围和检测模式

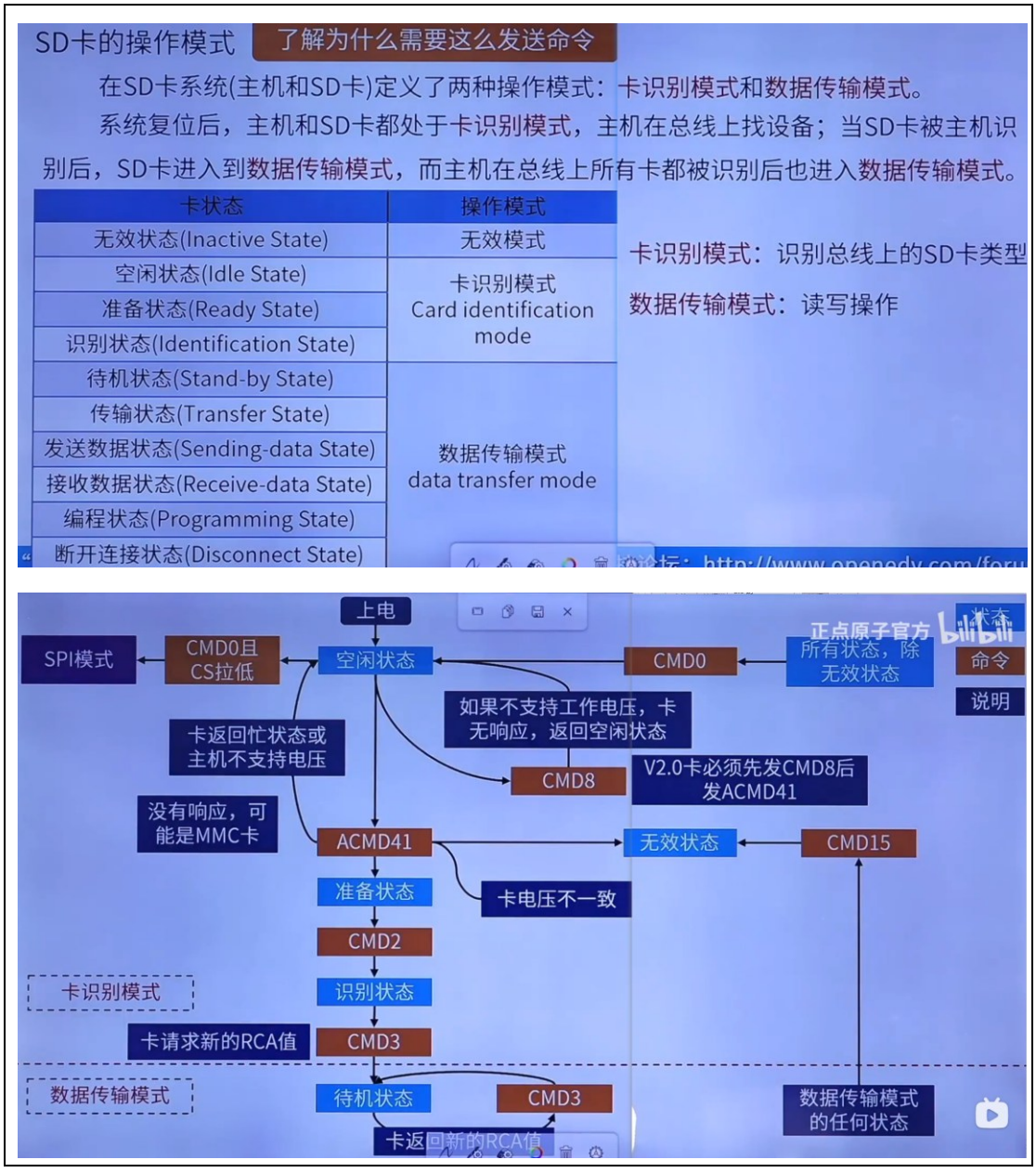
Bit position	47	46	[45:40]	[39:20]	[19:16]	[15:8]	[7:1]	0
Width (bits)	1	1	6	20	4	8	7	1
Value	'0'	'0'	'001000'	'00000h'	x	x	x	'1'
Description	start bit	transmission bit	command index	reserved bits	voltage accepted	echo-back of check pattern	CRC7	end bit

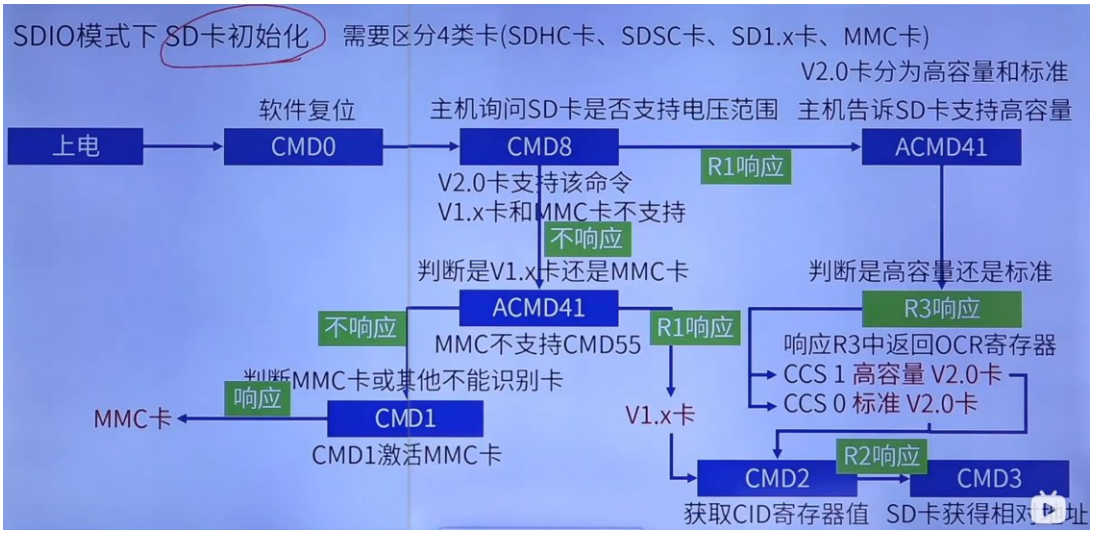
R6响应：专用于命令CMD3的响应(RCA响应)

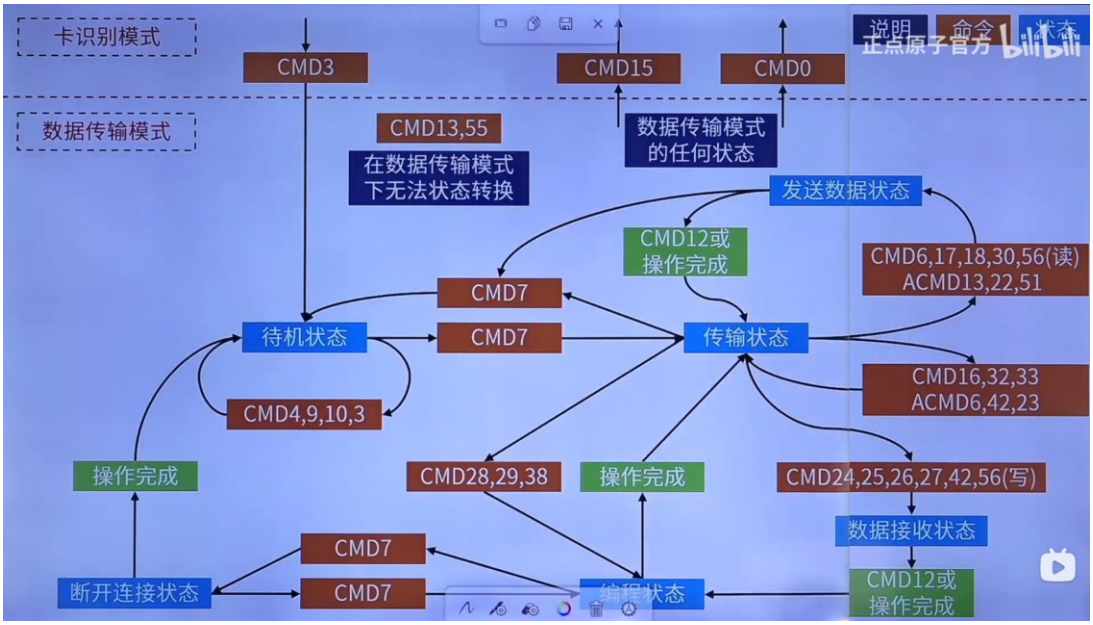
Bit position	47	46	[45:40]	[39:8] Argument field		[7:1]	0
Width (bits)	1	1	6	16	16	7	1
Value	'0'	'0'	x	x	x	x	'1'
Description	start bit	transmission bit	command index ('000011')	New published RCA [31:16] of the card	[15:0] card status bits: 23,22,19,12:0 (see Table 4-35)	CRC7	end bit

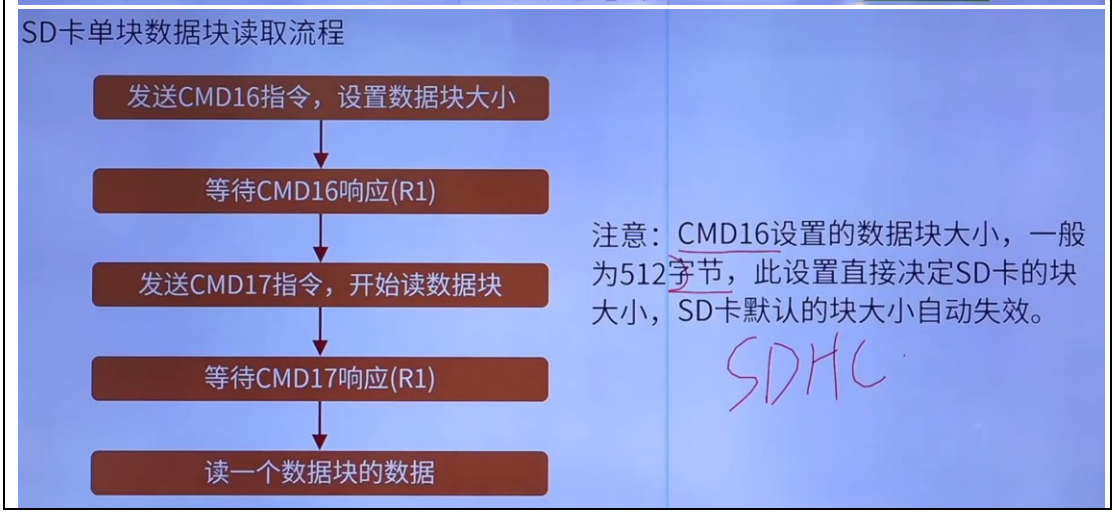
SPI模式没有RCA寄存器，也不支持CMD3命令，没有R6响应

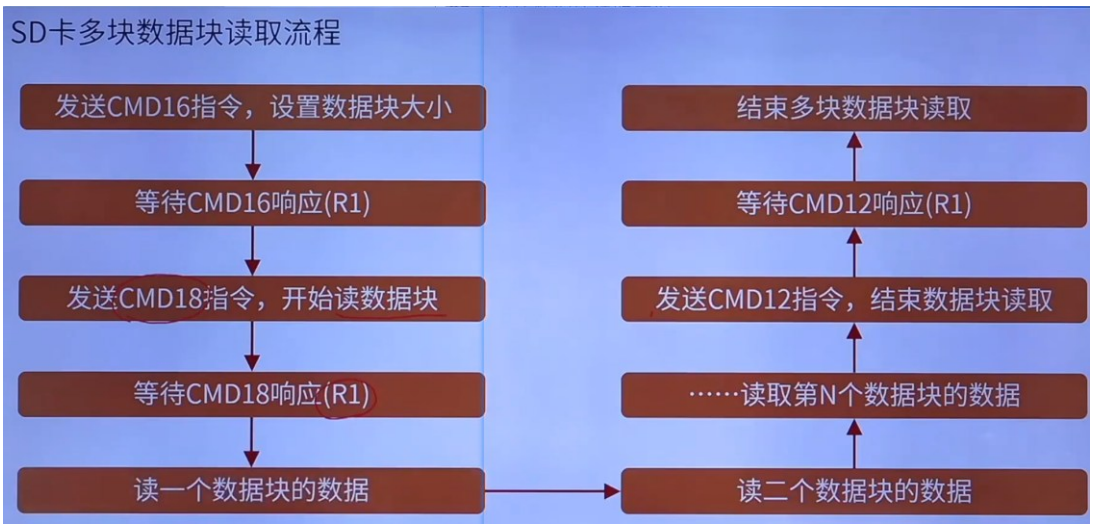
2.1.3.2 SD 卡操作步骤介绍



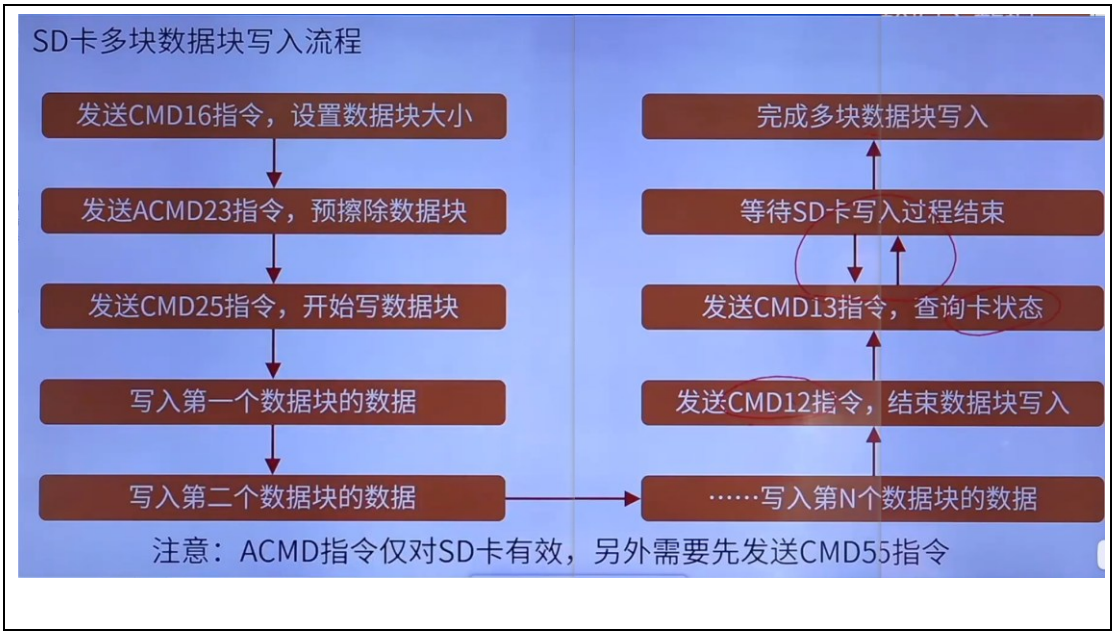

SDIO模式下SD卡初始化
需要区分4类卡(SDHC卡、SDSC卡、SD1.x卡、MMC卡)
V2.0卡分为高容量和标准
软件复位
主机询问SD卡是否支持电压范围
主机告诉SD卡支持高容量
上电
CMD0
CMD8
ACMD41
R1响应
V2.0卡支持该命令
V1.x卡和MMC卡不支持
不响应
判断是V1.x卡还是MMC卡
判断是高容量还是标准
ACMD41
R3响应
不响应
R1响应
MMC不支持CMD55
响应R3中返回OCR寄存器
CCS 1 高容量 V2.0卡
CCS 0 标准 V2.0卡
判断MMC卡或其他不能识别卡
响应
MMC卡
CMD1
CMD1激活MMC卡
V1.x卡
R2响应
CMD2
CMD3
获取CID寄存器值
SD卡获得相对地址


卡识别模式
数据传输模式
CMD3
CMD15
CMD0
CMD13,55
在数据传输模式下无法状态转换
数据传输模式的任何状态
发送数据状态
CMD12或操作完成
CMD6,17,18,30,56(读)
ACMD13,22,51
CMD7
待机状态
CMD7
传输状态
CMD16,32,33
ACMD6,42,23
CMD4,9,10,3
操作完成
CMD28,29,38
操作完成
CMD24,25,26,27,42,56(写)
数据接收状态
CMD7
断开连接状态
CMD7
编程状态
CMD12或操作完成


SD卡单块数据块读取流程
发送CMD16指令，设置数据块大小
等待CMD16响应(R1)
发送CMD17指令，开始读数据块
等待CMD17响应(R1)
读一个数据块的数据
注意：CMD16设置的数据块大小，一般为512字节，此设置直接决定SD卡的块大小，SD卡默认的块大小自动失效。
SDHC

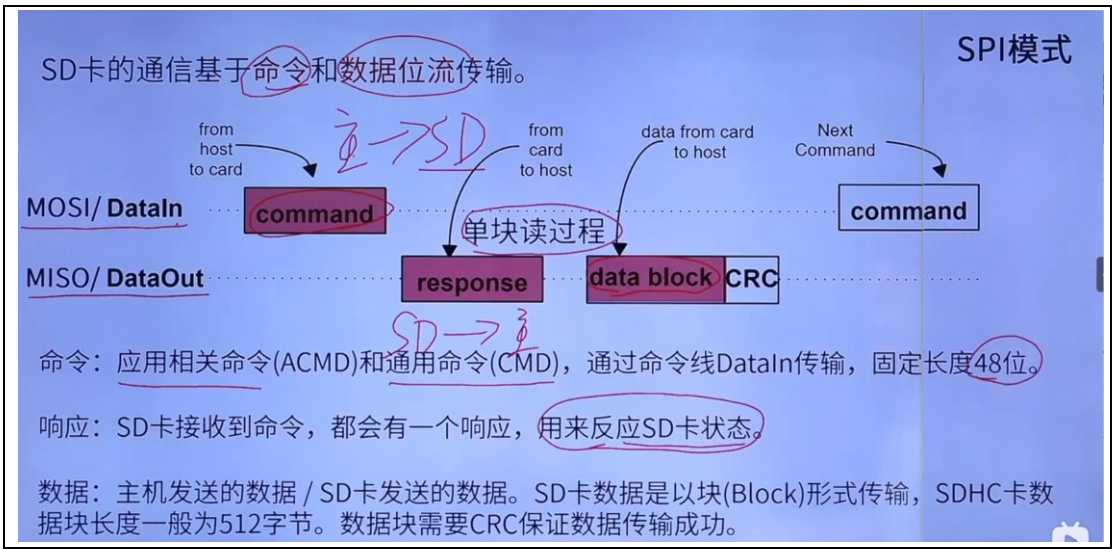

SD卡多块数据块读取流程
发送CMD16指令，设置数据块大小
等待CMD16响应(R1)
发送CMD18指令，开始读数据块
等待CMD18响应(R1)
读一个数据块的数据
读二个数据块的数据
……读取第N个数据块的数据
发送CMD12指令，结束数据块读取
等待CMD12响应(R1)
结束多块数据块读取

SD卡单块数据块写入流程
发送CMD16指令，设置数据块大小
此处省略了等待指令响应的步骤
发送CMD13指令，查询卡状态
等待READY_FOR_DATA位 = 1
发送CMD24指令，开始写入数据
写一个数据块的数据



2.1.4 SPI 模式

2.1.4.1 SD 卡常用命令介绍



SD卡命令格式

SD卡的命令格式由6个字节组成，发送数据时高位在前，SD卡的写入命令格式如下：

字节1(cmd)			字节2~5(参数内容)	字节6(校验)	
47	46	45 : 40	39 : 8	7 : 1	0
0	1	command	命令参数	CRC7	1
起始位					停止位

红色表示固定值

某些命令的CRC是固定的
CMD0 CRC:0x95
CMD8 CRC:0x87

Byte1：命令字的第一个字节为命令号（如CMD0、CMD1等），格式为“0 1 x x x x x x”

Byte2~Byte5：命令参数，有些命令参数是保留位，没有定义参数的内容，保留位应设置为0

Byte6：用于校验命令传输内容正确性，前7位为CRC(循环冗余校验)校验位，最后一位为停止位0

注意：在SPI模式下，CRC必须发，但SD卡会读到CRC时会自动忽略它，校验位全设为1即可

SD命令

SDIO模式和SPI模式，可使用的命令和特定类支持的命令有所不同。 **SPI模式**

SD卡系统的命令被分为多个类：

- 基本命令 Class 0（CMD0/CMD8/CMD9/CMD10/CMD12/CMD58）、
- 面向块读取命令 Class 2（CMD16/CMD17/CMD18）、
- 面向块写入命令 Class 4（CMD16/CMD24/CMD25）、
- 擦除命令 Class 5、
- 加锁命令 Class 7、
- 特定于应用命令 Class 8（CMD55）、
- 面向块写保护命令 Class 6、
- I/O模式命令 Class 9、
- SD卡特定应用命令（ACMD41/ ACMD13）、
- Switch功能命令 class10

强制性
所有SD卡都应支持

注意：发送ACMD命令之前，必须先发送CMD55，接下来要发送的是应用命令(APP CMD)，而非标准命令

SD卡常用命令 SPI模式

	命令	参数	响应	描述
SD卡初始化过程	CMD0(0x0)	NONE	R1	复位SD卡
	CMD8(0X08)	VHS+Check pattern	R7	主机发送接口状态命令
	ACMD41(0X29)	HCS	R3	主机发送容量支持信息 激活卡的初始化过程
	CMD58(0X3A)	NONE	R3	读取SD卡的OCR寄存器值
读取	CMD17(0X11)	地址	R1	读取一个块的数据
	CMD18(0X12)	地址	R1	连续读取多个块的数据
	CMD12(0X0C)	NONE	R1b	多块读取强制卡停止传输
写入	ACMD23(0x97)	Number of block	R1	设置需要预擦除的数据块数，提高SD卡多数据块写入性能（速度）
	CMD24(0X18)	地址	R1	写入一个块的数据
	CMD25(0X19)	地址	R1	连续写入多个块的数据
	CMD55(0X37)	NONE	R1	通知SD卡，接下来发送是应用命令
	CMD9(0X09)	NONE	R1	读取卡特定数据寄存器(CSD)
	CMD10(0X0A)	NONE	R1	读取卡标志数据寄存器(CID)

SD响应

SPI模式

SD卡和单片机的通信采用发送应答机制。

每发送一个命令，SD卡都会给出一个应答，以告知主机该命令的执行情况，或者返回主机需要获取的数据。使用SPI接口时，通过MISO传输。

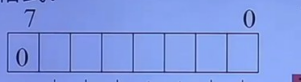
SD卡响应因使用接口不同，格式也不同。响应具体有R1、R1b、R2、R3、R7。

响应	长度(SPI)	描述
R1	8bit	正常响应命令
R1b	8bit	格式与R1相同，可选增加忙信号(MISO上传输)
R2	16bit	SPI:CMD13命令的响应，返回SD_STATUS
R3	40bit	SPI:CMD58命令的响应，返回OCR寄存器值
R7	40bit	CMD8命令的响应(卡支持电压信息)

《Physical Layer Simplified Specification V2.0》P120 7.3.2 SPI响应内容

SPI模式

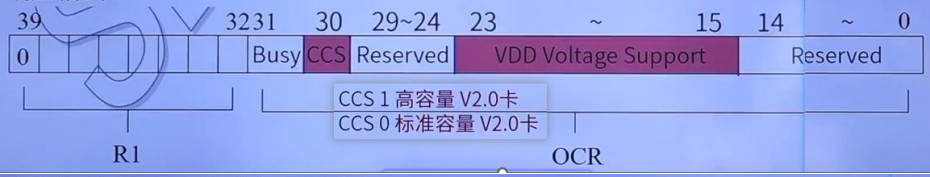
R1响应格式：



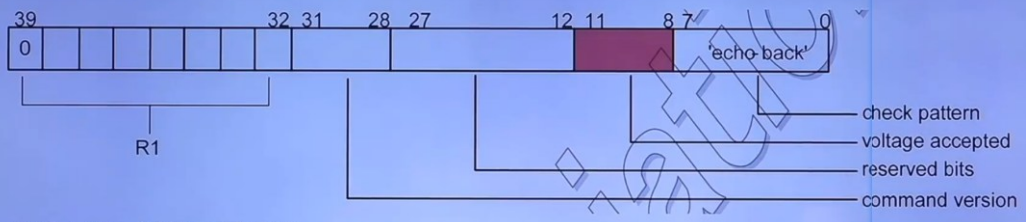
in idle state 1表示SD卡处于空闲状态
erase reset 接收到无需擦除操作的命令，擦除操作被复位
illegal command 接收到一个无效的命令代码
com crc error 接收到上一个命令的CRC校验错误
erase sequence error 擦除命令的控制顺序错误
address error 读写的数据地址不对齐(数据地址需按块大小对齐)
parameter error 命令的参数错误

注意：有错误，对应位置1

R3响应格式：



R7响应格式（CMD8命令专属响应）



Bit[11:8]：操作电压反馈

0：未定义

1：2.7V~3.6V

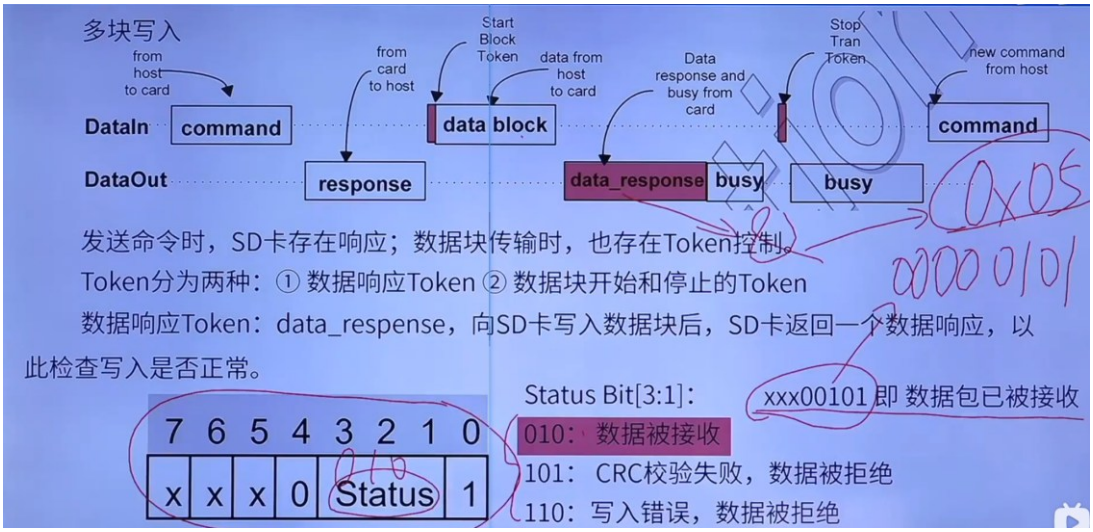
2：低电压

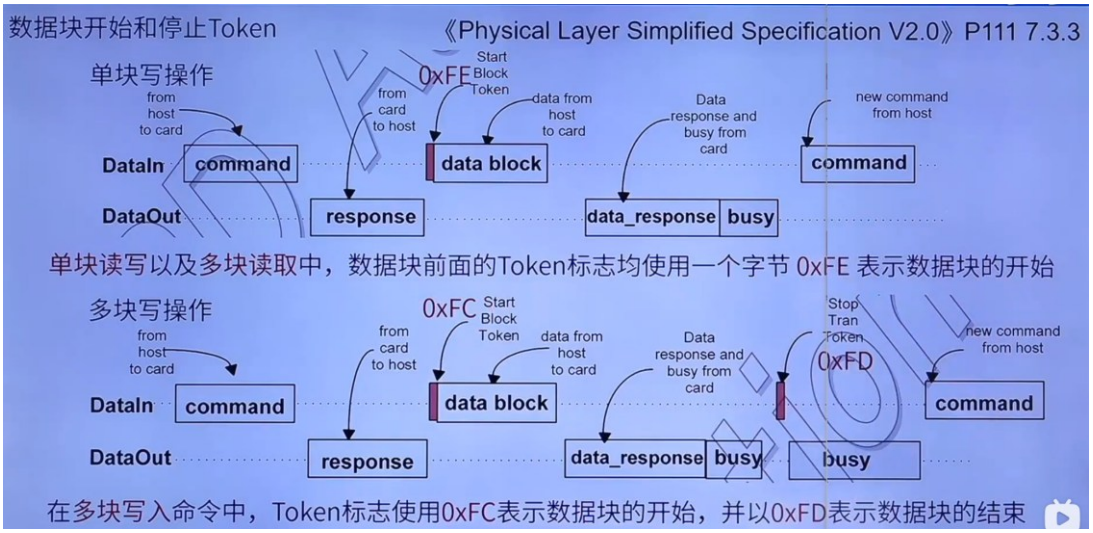
4：保留位

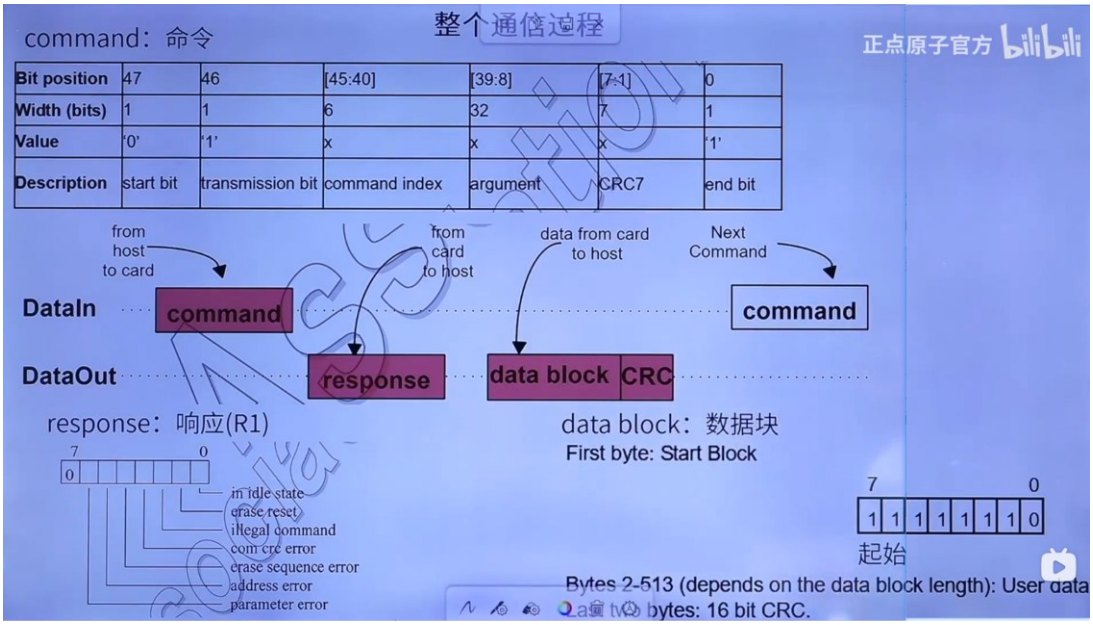
8：保留位

其它：未定义

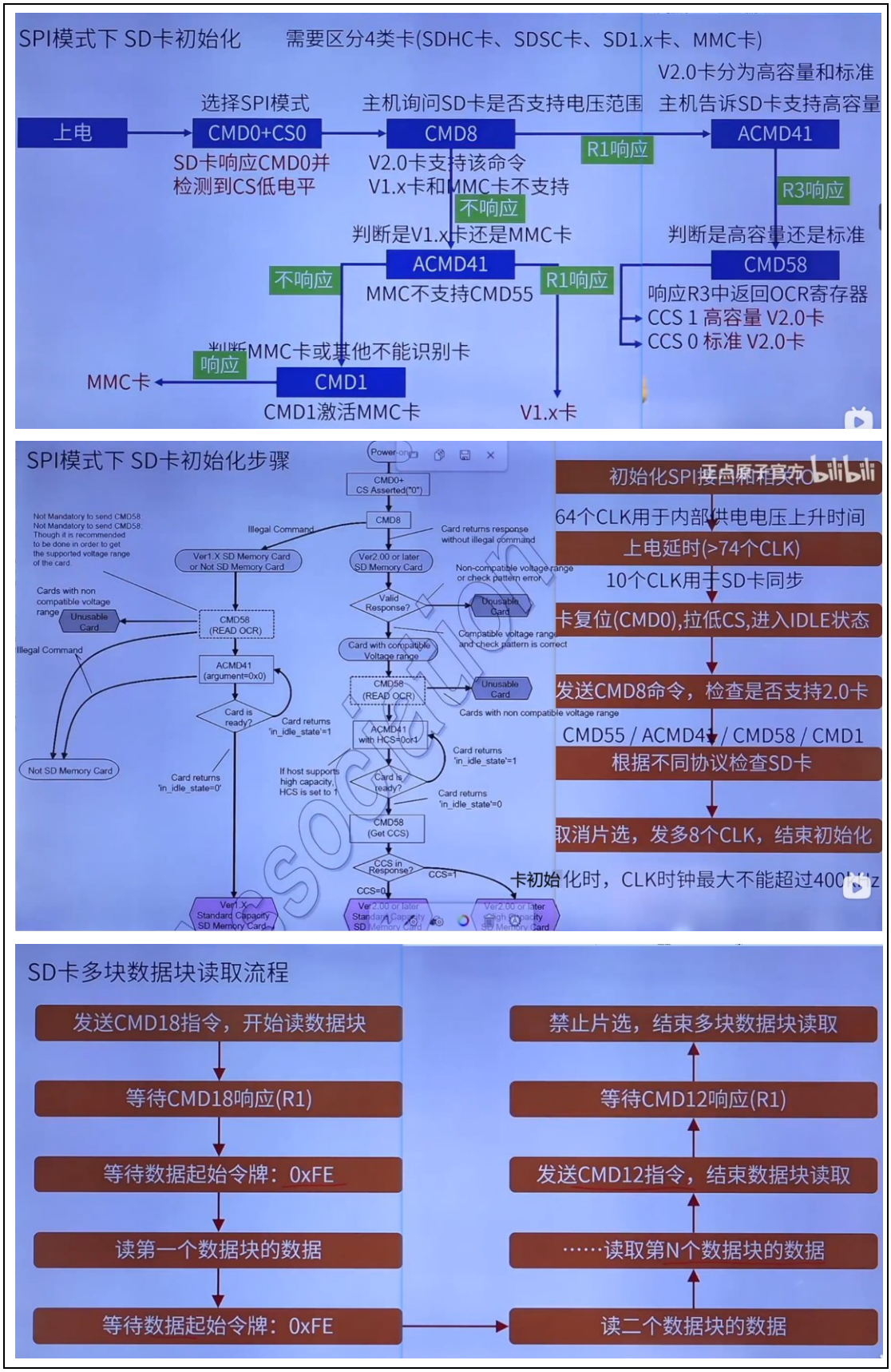
发送完CMD8命令(主机支持电压范围)后，SD卡R7响应，进而查看SD卡支持操作电压


多块写入
from host to card
from card to host
Start Block Token
data from host to card
Data response and busy from card
Stop Tran Token
new command from host
DataIn
command
data block
command
DataOut
response
data_response
busy
busy
0x05
0000 0101
发送命令时，SD卡存在响应；数据块传输时，也存在Token控制。
Token分为两种：① 数据响应Token ② 数据块开始和停止的Token
数据响应Token：data_respense，向SD卡写入数据块后，SD卡返回一个数据响应，以此检查写入是否正常。
7 6 5 4 3 2 1 0
x x x 0 Status 1
Status Bit[3:1]：
010：数据被接收
101：CRC校验失败，数据被拒绝
110：写入错误，数据被拒绝
xxx00101 即 数据包已被接收


数据块开始和停止Token
《Physical Layer Simplified Specification V2.0》P111 7.3.3
单块写操作
0xFE
from host to card
from card to host
Start Block Token
data from host to card
Data response and busy from card
new command from host
DataIn
command
data block
command
DataOut
response
data_response
busy
单块读写以及多块读取中，数据块前面的Token标志均使用一个字节 0xFE 表示数据块的开始
多块写操作
0xFC
Stop Tran Token
0xFD
DataIn
command
data block
command
DataOut
response
data_response
busy
busy
在多块写入命令中，Token标志使用0xFC表示数据块的开始，并以0xFD表示数据块的结束


command：命令
整个通信过程
正点原子官方 bilibili
Bit position | 47 | 46 | [45:40] | [39:8] | [7:1] | 0
Width (bits) | 1 | 1 | 6 | 32 | 7 | 1
Value | '0' | '1' | x | x | x | '1'
Description | start bit | transmission bit | command index | argument | CRC7 | end bit
from host to card
from card to host
data from card to host
Next Command
DataIn
command
command
DataOut
response
data block
CRC
response：响应(R1)
7 0
0
in idle state
erase reset
illegal command
com crc error
erase sequence error
address error
parameter error
data block：数据块
First byte: Start Block
7 0
1 1 1 1 1 1 1 0
起始
Bytes 2-513 (depends on the data block length): User data
Last two bytes: 16 bit CRC.

2.1.4.2 SD 卡操作步骤介绍

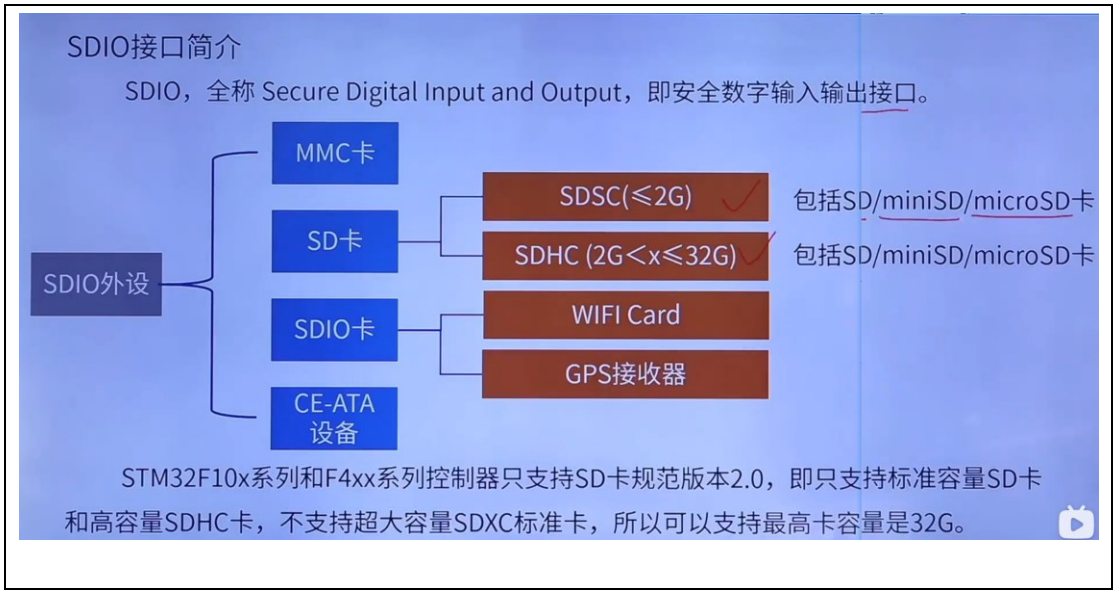


2.2 SPI 驱动 SD 卡步骤

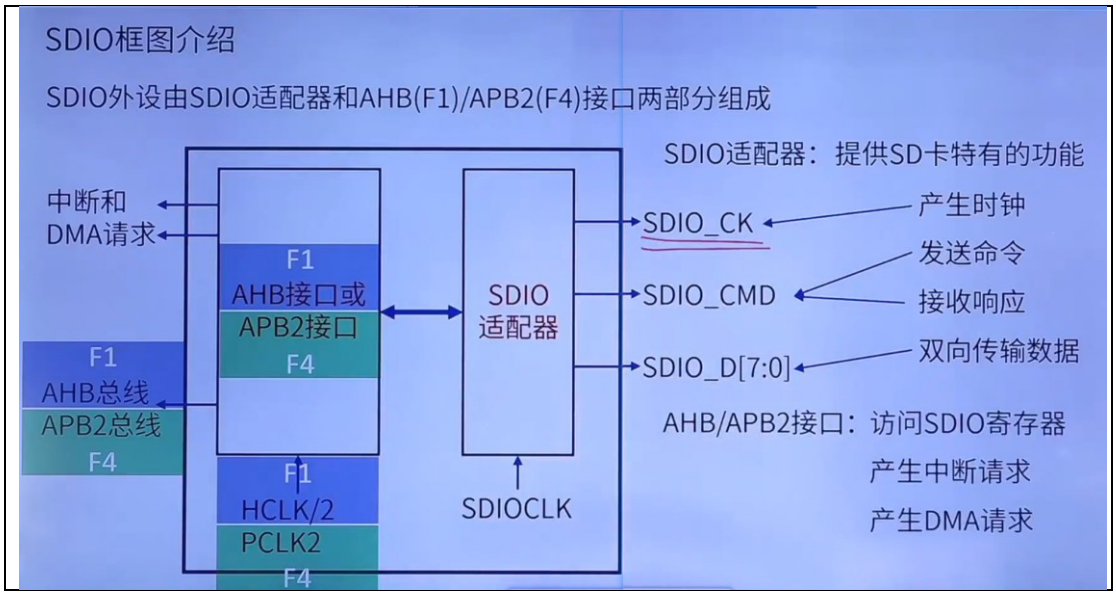
1、初始化SD卡SPI模式相关GPIO口	详看原理图，GPIO模式要设置为复用推挽模式
2、SD卡初始化	参考前面的SPI模式下SD卡初始化步骤
3、数据块写入	参考前面写入流程
4、数据块读取	参考前面读取流程

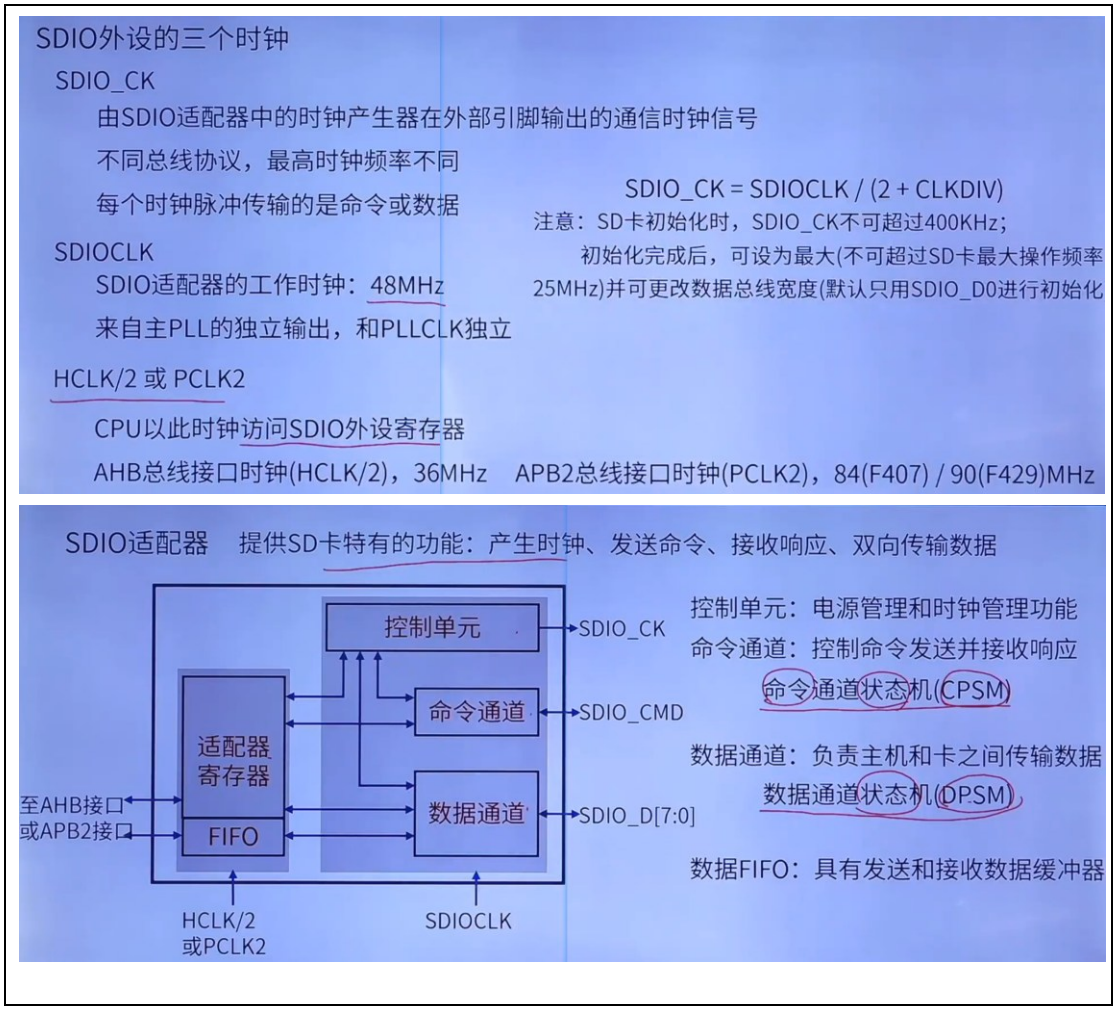
2.3 SDIO 外设介绍

2.3.1 SDIO 接口简介

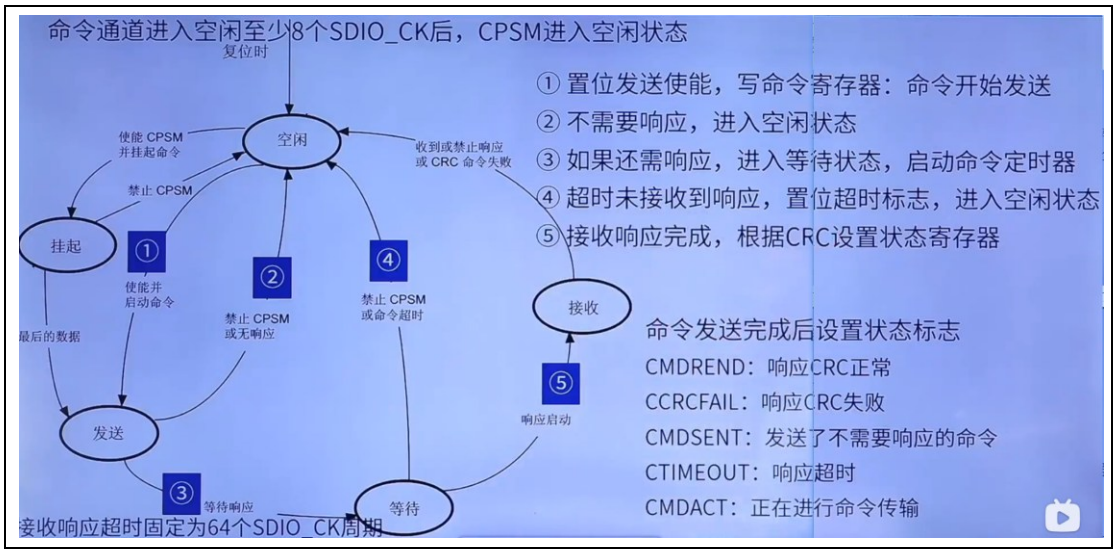


2.3.2 SDIO 框图介绍

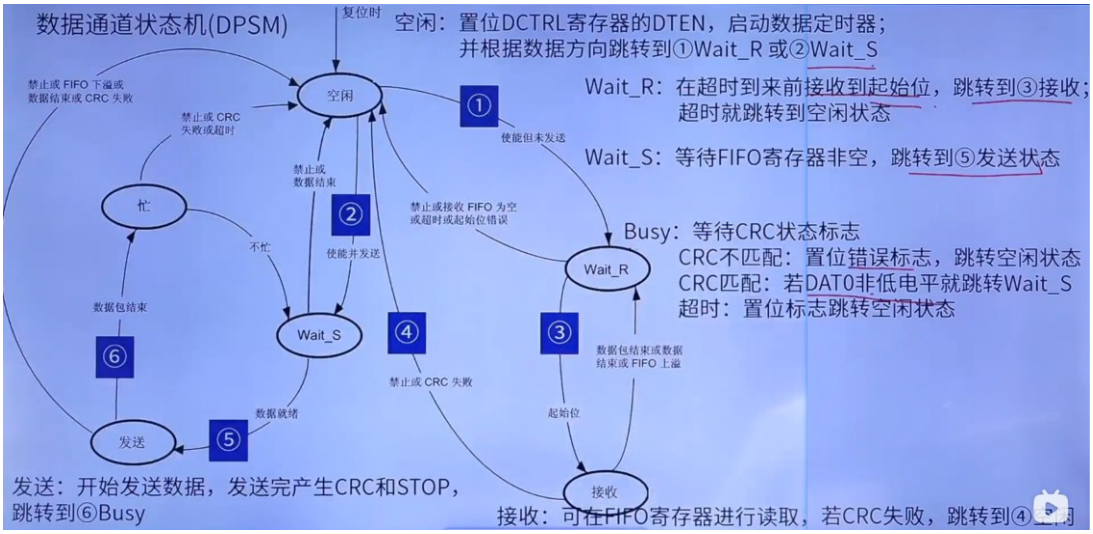




2.3.3 SDIO 命令与响应



数据通道状态机(DPSM)



空闲：置位DCTRL寄存器的DTEN，启动数据定时器；
并根据数据方向跳转到①Wait_R 或②Wait_S

Wait_R：在超时到来前接收到起始位，跳转到③接收；
超时就跳转到空闲状态

Wait_S：等待FIFO寄存器非空，跳转到⑤发送状态

Busy：等待CRC状态标志
CRC不匹配：置位错误标志，跳转空闲状态
CRC匹配：若DAT0非低电平就跳转Wait_S
超时：置位标志跳转空闲状态

发送：开始发送数据，发送完产生CRC和STOP，
跳转到⑥Busy

接收：可在FIFO寄存器进行读取，若CRC失败，跳转到④[illegible]

数据FIFO

包括32个字的数据缓冲，和发送与接收电路

TXACT和RXACT标志分别表明当前处于发送还是接收状态，由数据通道子单元对这两个标志互斥的置位

发送FIFO

SDIO发送数据时，由数据通道单元置位TXACT，表示数据传输正在进行

当TXACT位为1，通过APB2接口(F4) / AHB接口(F1)将数据写入到发送FIFO

发送FIFO相关状态：TXFIFOF,TXFIFOE,TXFIFOHE,TXDAVL,TXUNDERRUN

下溢：读空FIFO后继续读则导致下溢

SDIO_STA 状态寄存器

可避免发送模式下溢

硬件流控

可避免接收模式上溢

接收FIFO

SDIO接收数据时，由数据通道单元置位RXACT

当RXACT位为1，通过接收FIFO存放从数据通道接收到的数据

接收FIFO的相关状态： RXFIFOF,RXFIFOE,RXFIFOHF,RXDAVL,RXOVERR

上溢：写满FIFO后继续写则导致上溢

命令格式

命令是用于开始一项操作。命令在CMD线上串行发送，属于半双工模式，命令和响应可分别发送和接收。

命令固定长度为48位。

1位 start bit + 1位 transmission bit + 6位 command index + 32位argument + 7位CRC + 1位end bit

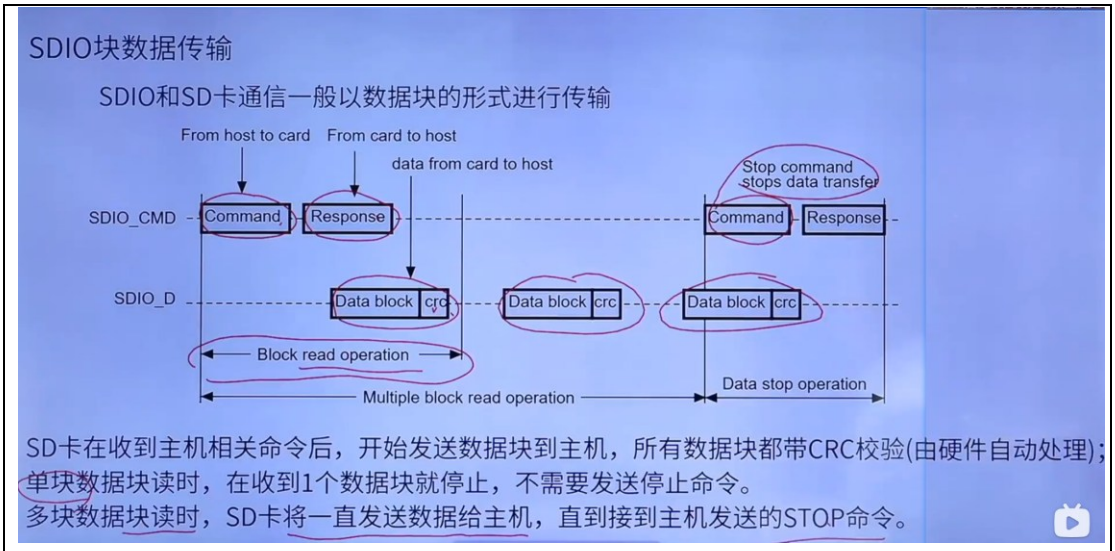
位	宽度	数值	说明
47	1	0	开始位
46	1	1	传输位
[45:40]	6	-	命令索引 SDIO_CMD
[39:8]	32	-	参数 SDIO_ARG
[7:1]	7	-	CRC7
0	1	1	结束位

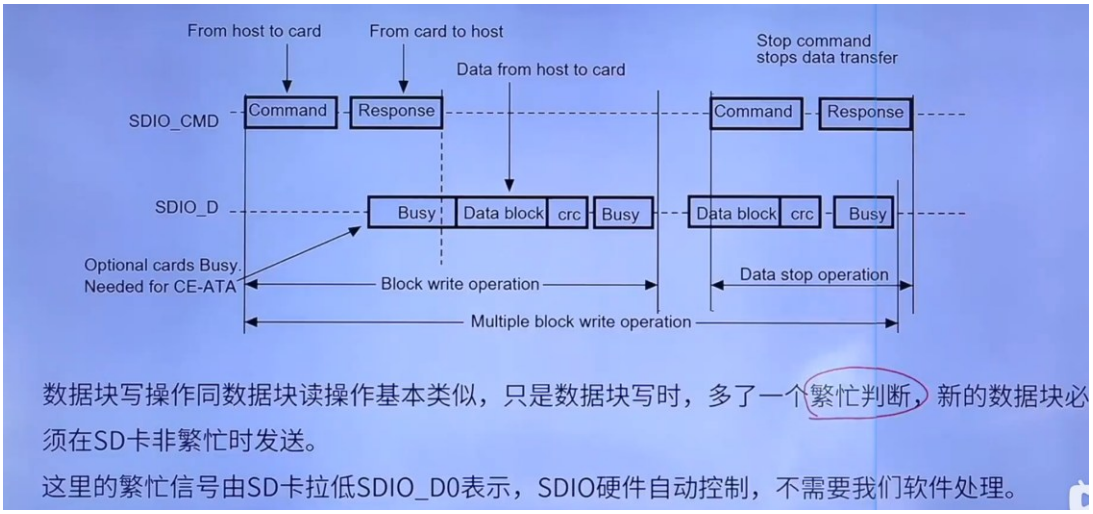
硬件自动过滤　寄存器设置

位	宽度	数值	说明
135	1	0	开始位
134	1	0	传输位
[133:128]	6	111111	保留 SDIO_RESP1~4
[127:1]	127	-	CID或CSD(包含内部CRC7)
0	1	1	结束位

SDIO支持2种响应类型，48位短响应和136位长响应，2种类型都有CRC错误检测

2.3.4 SDIO 块数据传输





2.3.5 SDIO 寄存器介绍

SDIO寄存器

寄存器	名称	作用
SDIO_POWER	SDIO电源控制寄存器	定义卡时钟的当前功能状态
SDIO_CLKCR	SDIO时钟控制寄存器	用于设置SDIO_CK输出时钟(设置分频系数)
SDIO_ARG	SDIO参数寄存器	用于存储命令参数(写命令之前先写)
SDIO_CMD	SDIO命令寄存器	用于设置命令索引和命令类型
SDIO_RESPCMD	SDIO命令响应寄存器	用于存储最后收到的命令响应中的命令索引
SDIO_RESPx(1~4)	SDIO命令响应1~4寄存器	用于存放接收到的卡响应部分信息
SDIO_DTIMER	SDIO数据定时器寄存器	用于存储以卡时钟为周期的数据超时时间
SDIO_DLEN	SDIO数据长度寄存器	用于设置需要传输的数据字节长度
SDIO_DCTRL	SDIO数据控制寄存器	用于控制数据通道状态机
SDIO_STA	SDIO状态寄存器	用于查询SDIO控制器的当前状态
SDIO_FIFO	SDIO数据FIFO寄存器	用于接收或者发送FIFO数据

SDIO电源控制寄存器（SDIO_POWER）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留																														PWRCTRL	
res																														rw	

位1:0	**PWRCTRL**：电源控制位 (Power supply control bits) 这些位用于定义卡时钟的当前功能状态： 00：电源关闭，卡的时钟停止。 01：保留。 10：保留的上电状态。 11：上电状态，卡的时钟开启。

STM32复位后，PWRCTRL=00，处于掉电状态。首先要给SDIO上电，PWRCTRL设置为11。

SDIO时钟控制寄存器（SDIO_CLKCR）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留																	HWFC_EN	NEGEDGE	WIDBUS		BYPASS	PWRSAV	CLKEN	CLKDIV							
res																	rw	rw	rw		rw	rw	rw	r/w							

位12:11	**WIDBUS**：宽总线模式使能位 (Wide bus mode enable bit) 00：默认总线模式，使用SDIO_D0。 01：4位总线模式，使用SDIO_D[3:0]。 10：8位总线模式，使用SDIO_D[7:0]。
位10	**BYPASS**：旁路时钟分频器 (Clock divider bypass enable bit) 0：关闭旁路：驱动SDIO_CK输出信号之前，依据CLKDIV数值对SDIOCLK分频。 1：使能旁路：SDIOCLK直接驱动SDIO_CK输出信号。
位8	**CLKEN**：时钟使能位 (Clock enable bit) 0：SDIO_CK关闭。 1：SDIO_CK使能。
位7:0	**CLKDIV**：时钟分频系数 (Clock divide factor) 这个域定义了输入时钟(SDIOCLK)与输出时钟(SDIO_CK)间的分频系数： SDIO_CK频率 = SDIOCLK/[CLKDIV + 2]。

SDIO初始化频率最大400kHz
48MHz / (CLK_DIV + 2) < 400KHz

SDIO数据传输频率最大25MHz
设置CLKDIV为0

注意：当SDIO_CK频率过快时，可能会导致SD卡通信失败，此时，建议降低SDIO_CK试试。

SDIO参数寄存器（SDIO_ARG）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CMDARG																															
rw																															

位31:0	**CMDARG**：命令参数 (Command argument) 命令参数是发送到卡中命令的一部分，如果一个命令包含一个参数，必须在写命令到命令寄存器之前加载这个寄存器。

位	宽度	数值	说明
47	1	0	开始位
46	1	1	传输位
[45:40]	6	-	命令索引 SDIO_CMD
[39:8]	32	-	参数 SDIO_ARG
[7:1]	7	-	CRC7
0	1	1	结束位

SDIO命令寄存器（SDIO_CMD）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留																	CE_ATACMD	nIEN	ENCMDcompl	SDIOSuspend	CPSMEN	WAITPEND	WAITINT	WAITRESP	CMDINDEX						
res																	rw	rw	rw	rw	rw	rw	rw	rw	rw						

位10	**CPSMEN**：命令通道状态机(CPSM)使能位 (Command path state machine (CPSM) Enable bit) 如果设置该位，则使能CPSM。 一般情况下，开启CPSM，即设置1
位7:6	**WAITRESP**：等待响应位 (Wait for response bits) 这2位指示CPSM是否需要等待响应，如果需要等待响应，则指示响应类型。 00：无响应，期待CMDSENT标志 01：短响应，期待CMDREND或CCRCFAIL标志 10：无响应，期待CMDSENT标志 11：长响应，期待CMDREND或CCRCFAIL标志 根据命令需求
位5:0	**CMDINDEX**：命令索引 (Command index) 命令索引是作为命令的一部分发送到卡中。 发送CMD1，其值为1,索引就设置为1

SDIO命令响应寄存器（SDIO_RESPCMD）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
保留																												RESPCMD			
res																												r			

位5:0	**RESPCMD**：响应的命令索引 (Response command index) 只读位，包含最后收到的命令响应中的命令索引。

SDIO响应1~4寄存器（SDIO_RESP1~4）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CARDSTATUSx																															
r																															

位31:0	**CARDSTATUSx**：见下表。

寄存器	短响应	长响应
SDIO_RESP1	卡状态[31:0]	卡状态[127:96]
SDIO_RESP2	不用	卡状态[95:64]
SDIO_RESP3	不用	卡状态[63:32]
SDIO_RESP4	不用	卡状态[31:1]

SDIO数据定时器寄存器（SDIO_DTIMER）

31:0
DATATIME
rw

位31:0	**DATATIME**：数据超时时间 (Data timeout period) 以卡总线时钟周期为单位的数据超时时间。

在写入数据控制寄存器进行数据传输之前，必须先写入数据定时器寄存器和数据长度寄存器

SDIO数据长度寄存器（SDIO_DLEN）

31:25	24:0
保留	DATALENGTH
res	rw

位24:0	**DATALENGTH**：数据长度 (Data length value) 要传输的数据字节数目。

对于块数据传输，该寄存器值必须是数据块长度的倍数。

SDIO数据控制寄存器（SDIO_DCTRL）

31:12	11	10	9	8	7:4	3	2	1	0
保留	SDIOEN	RWMOD	RWSTOP	RWSTART	DBLOCKSIZE	DMAEN	DTMODE	DTDIR	DTEN

位7:4	**DBLOCKSIZE**：数据块长度 (Data block size) 当选择了块数据传输模式，该域定义数据块长度： 0000：块长度 = 2^0 = 1字节； 0001：块长度 = 2^1 = 2字节； 0010：块长度 = 2^2 = 4字节； 0011：块长度 = 2^3 = 8字节； 0100：(十进制4)块长度 = 2^4 = 16字节； 0101：(十进制5)块长度 = 2^5 = 32字节； 0110：(十进制6)块长度 = 2^6 = 64字节； 0111：块长度 = 2^7 = 128字节； 1000：块长度 = 2^8 = 256字节； 1001：块长度 = 2^9 = 512字节； 1010：块长度 = 2^{10} = 1024字节； 1011：块长度 = 2^{11} = 2048字节； 1100：块长度 = 2^{12} = 4096字节； 1101：块长度 = 2^{13} = 8192字节； 1110：块长度 = 2^{14} = 16384字节； 1111：保留。
位2	**DTMODE**：数据传输模式 (Data transfer mode selection) 0：块数据传输； 1：流数据传输。
位1	**DTDIR**：数据传输方向 (Data transfer direction selection) 0：控制器至卡； 1：卡至控制器。
位0	**DTEN**：数据传输使能位 (Data transfer enabled bit) 如果设置该位为1，则开始数据传输。根据DTSIR方向位，DPSM进入Wait_S或Wait_R状态，如果在传输的一开始就设置了RWSTART位，则DPSM进入该等待状态。不需要在数据传束后清除使能位，但必须更改SDIO_DCTRL以允许新的数据传输。

SDIO状态寄存器（SDIO_STA）

31:24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved	CEATAEND	SDIOIT	RXDAVL	TXDAVL	RXFIFOE	TXFIFOE	RXFIFOF	TXFIFOF	RXFIFOHF	TXFIFOHE	RXACT	TXACT	CMDACT	DBCKEND	STBITERR	DATAEND	CMDSENT	CMDREND	RXOVERR	TXUNDERR	DTIMEOUT	CTIMEOUT	DCRCFAIL	CCRCFAIL
Res.	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

位 23 **CEATAEND**：针对 CMD61 收到了 CE-ATA 命令完成信号

位 22 **SDIOIT**：收到了 SDIO 中断

位 21 **RXDAVL**：接收 FIFO 中有数据可用

位 20 **TXDAVL**：传输 FIFO 中有数据可用

位 18 **TXFIFOE**：发送 FIFO 为空
如果使能了硬件流控制，则 TXFIFOE 信号在 FIFO 包含 2 个字时激活。

位 17 **RXFIFOF**：接收 FIFO 已满 (Receive FIFO full)
如果使能了硬件流控制，则 RXFIFOF 信号在 FIFO 差 2 个字便变满之前激活。

位 16 **TXFIFOF**：传输 FIFO 已满 (Transmit FIFO full)

位 15 **RXFIFOHF**：接收 FIFO 半满：FIFO 中至少有 8 个字

位 14 **TXFIFOHE**：传输 FIFO 半空：至少可以写入 8 个字到 FIFO

位 13 **RXACT**：数据接收正在进行中

位 12 **TXACT**：数据传输正在进行中

位 11 **CMDACT**：命令传输正在进行中

位 10 **DBCKEND**：已发送/接收数据块（CRC 校验通过）

位 9 **STBITERR**：在宽总线模式下，并非在所有数据信号上都检测到了起始位

位 8 **DATAEND**：数据结束（数据计数器 SDIDCOUNT 为零）

位 7 **CMDSENT**：命令已发送（不需要响应）

位 6 **CMDREND**：已接收命令响应（CRC 校验通过）

位 5 **RXOVERR**：收到了 FIFO 上溢错误

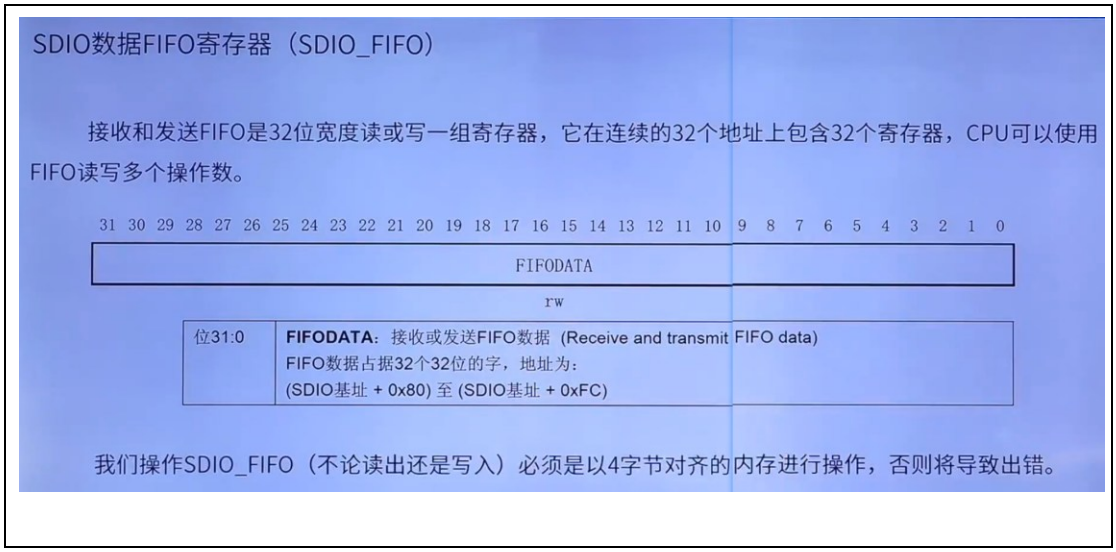
位 4 **TXUNDERR**：传输 FIFO 下溢错误

位 3 **DTIMEOUT**：数据超时

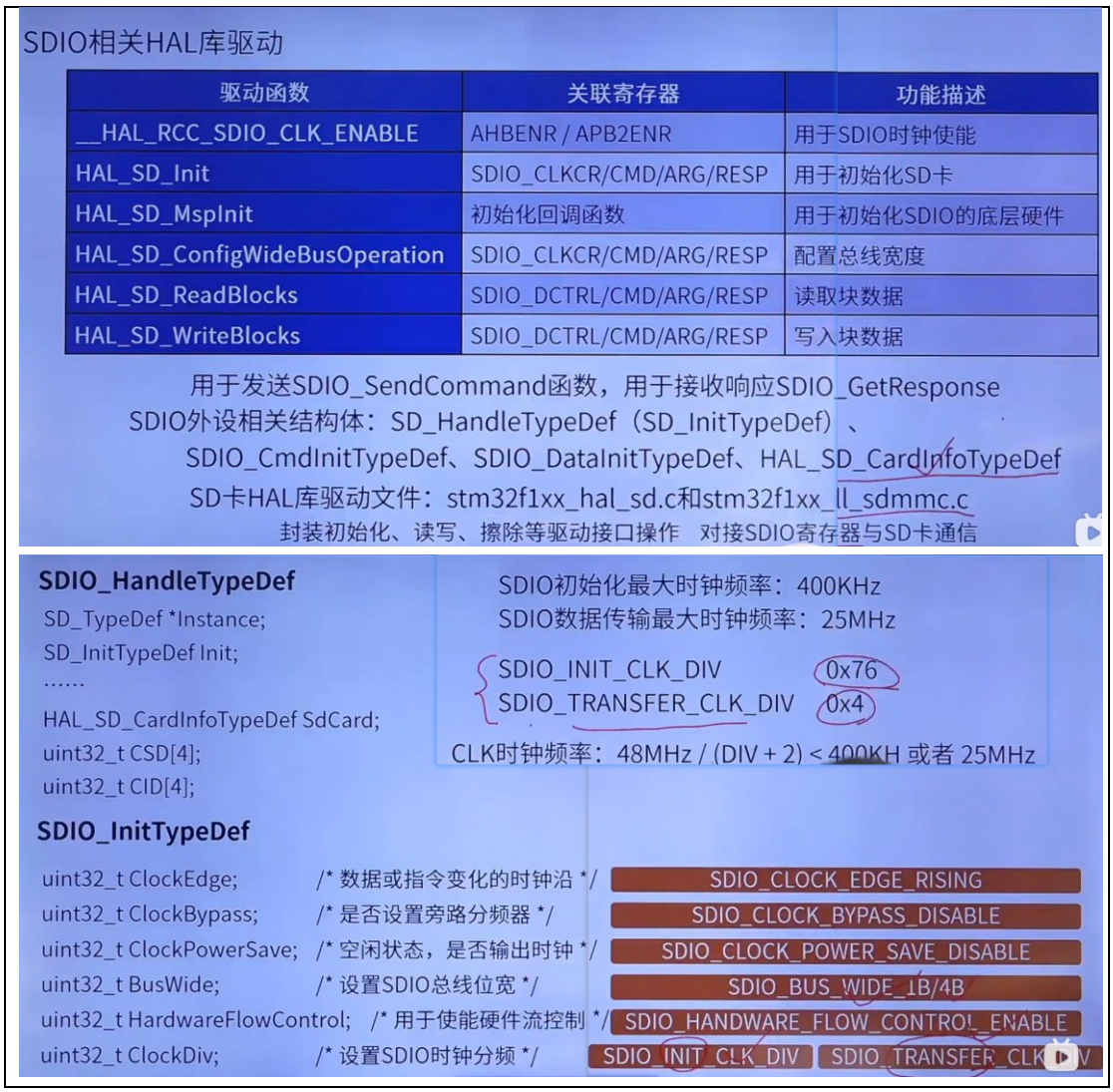
位 2 **CTIMEOUT**：命令响应超时
命令超时周期为固定值 64 个 SDIO_CK 时钟周期。

位 1 **DCRCFAIL**：已发送/接收数据块（CRC 校验失败）

位 0 **CCRCFAIL**：已接收命令响应（CRC 校验失败）



2.3.6 SDIO 相关 HAL 库驱动介绍



SDIO_CmdInitTypeDef

CMD8(0X08)	VHS+Check pattern	R7	主机发送接口状态命令

```
uint32_t Argument;          /* 命令参数 */
uint32_t CmdIndex;          /* 命令索引 */
uint32_t Response;          /* 响应类型 */
uint32_t WaitForInterrupt;  /* 等待中断请求 */
uint32_t CPSM;  /* 是否使用命令通道状态机 */
```

Response：响应类型(无响应/长响应/短响应)

WaitForInterrupt：等待类型选择，三种状态

SDIO_WAIT_NO　　无等待，使用超时检测

SDIO_WAIT_IT　　等待中断

SDIO_WAIT_PEND　　等待传输完成

CPSM：SDIO_CPSM_ENABLE　　使能CPSM

```
uint32_t SDMMC_CmdOperCond(SDIO_TypeDef *SDIOx)
{
  SDIO_CmdInitTypeDef  sdmmc_cmdinit;
  uint32_t errorstate;
  /* Send CMD8 to verify SD card interface operating condition */
  /* Argument: - [31:12]: Reserved (shall be set to '0')
  - [11:8]: Supply Voltage (VHS) 0x1 (Range: 2.7-3.6 V)
  - [7:0]: Check Pattern (recommended 0xAA) */
  /* CMD Response: R7 */
  sdmmc_cmdinit.Argument         = SDMMC_CHECK_PATTERN;
  sdmmc_cmdinit.CmdIndex         = SDMMC_CMD_HS_SEND_EXT_CSD;
  sdmmc_cmdinit.Response         = SDIO_RESPONSE_SHORT;
  sdmmc_cmdinit.WaitForInterrupt = SDIO_WAIT_NO;
  sdmmc_cmdinit.CPSM             = SDIO_CPSM_ENABLE;
  (void)SDIO_SendCommand(SDIOx, &sdmmc_cmdinit);

  /* Check for error conditions */
  errorstate = SDMMC_GetCmdResp7(SDIOx);
  return errorstate;
}
```

注意：主机发送完命令后，需要接收响应，响应类型的情况：Response指明。

SDIO_DataInitTypeDef　与SDIO_DCTRL相关

```
uint32_t DataTimeOut;       /* 数据超时时间 */
uint32_t DataLength;        /* 数据长度 */
uint32_t DataBlockSize;     /* 数据块大小 */
uint32_t TransferDir;       /* 传输方向(读/写) */
uint32_t TransferMode;      /* 传输模式(块/流)*/
Uint32_t DPSM;        /* 是否使用数据通道状态机 */
```

SDMMC_DATATIMEOUT

NumberOfBlocks * BLOCKSIZE

SDIO_DATABLOCK_SIZE_512B

SDIO_TRANSFER_DIR_TO_(CARD/SDIO)

SDIO_TRANSFER_MODE_BLOCK

SDIO_DPSM_ENABLE

SDIO_DataInitTypeDef关联的函数SDIO_ConfigData，发送/接收数据块会使用到。

注意：SD卡操作使用数据块类型，MMC卡才是用数据流类型

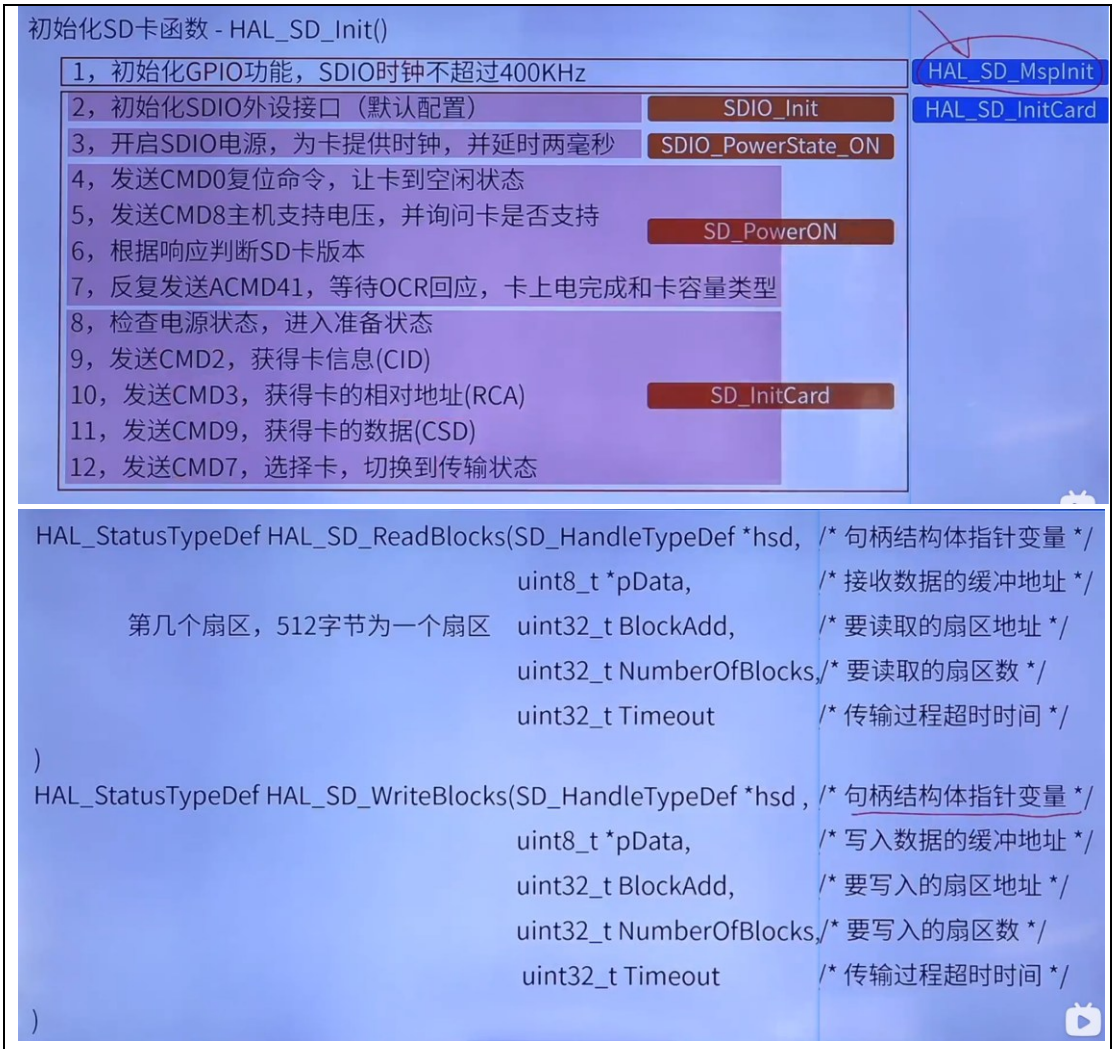
HAL_SD_CardInfoTypeDef　与SD卡的CSD寄存器相关

```
uint32_t CardType;          /* 卡类型 */
uint32_t CardVersion;       /* 卡版本 */
uint32_t Class;             /* 卡类型 */
uint32_t RelCardAdd;        /* 卡相对地址 */
uint32_t BlockNbr;          /* 整个卡的块数 */
uint32_t BlockSize;         /* 每个块的字节数 */
uint32_t LogBlockNbr;       /* 整个卡的逻辑块数 */
uint32_t LogBlockSize;      /* 逻辑块大小 */
```

CARD_SDSC　CARD_SDHC_SDXC　CARD_SECURED

CARD_V1_X　CARD_V2_X

通过HAL_SD_GetCardInfo函数获取卡信息



2.4 SDIO 驱动 SD 卡步骤

2.5 SDMMC 外设介绍（F7/H7）

SDMMC，全称 Secure Digital Input/Output MultiMediaCard interface，安全数字输入输出多媒体卡接口。

STM32外设	支持协议版本	数据传输速度
H7的SDMMC	多媒体卡版本4.51 SD存储卡版本4.1	8位模式，高达208MB/s 速度模式(SDRx/DDRx)
SDIO	多媒体卡版本4.2 SD存储卡版本2.0	8位模式，高达48MB/s

更好驱动emmc
支持超高速I卡
传输速度更快
(需硬件支持)

F7、H7使用SDMMC驱动SD卡与F1、F4使用SDIO驱动SD卡类似

SDMMC在SD/SDIO操作模式下有多种速度模式

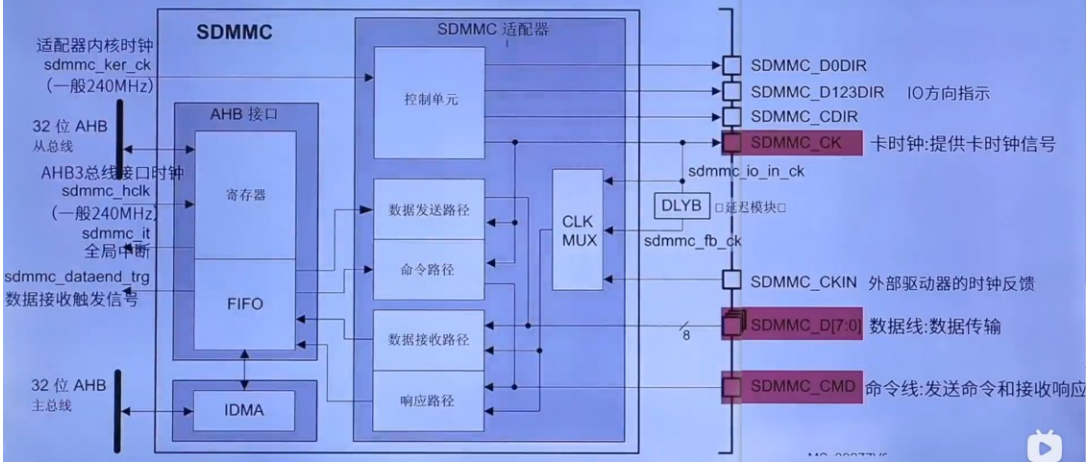
超高速I卡 UHS-I

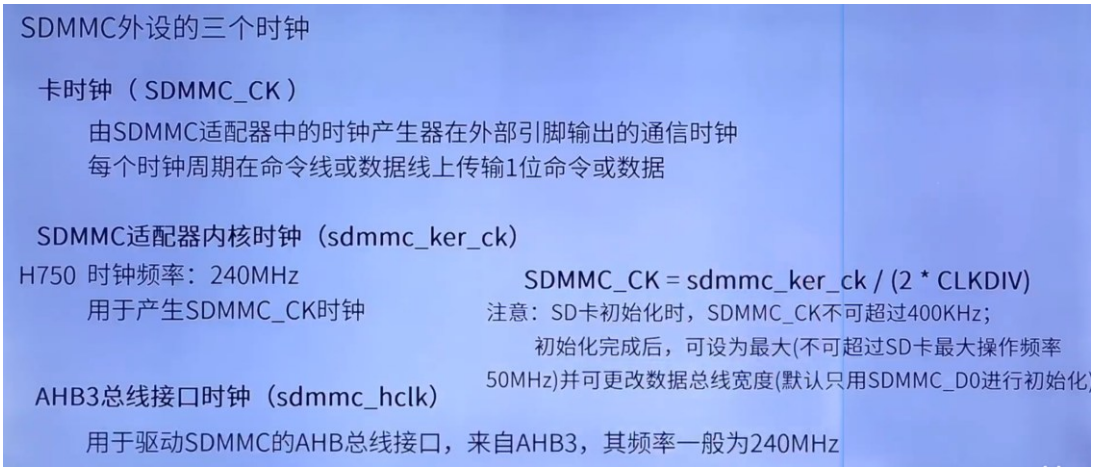
SDIO 总线速度模式	最大速度（MB/s）	最大时钟频率（Mhz）	信号电平（V）
DS（默认速度）	12.5	25	3.3
HS（高速）	25	50	3.3
SDR12	12.5	25	1.8
SDR25	25	50	1.8
DDR50	50	50	1.8
SDR50	50	100	1.8
SDR104	104	208	1.8

开发板电路受限，SD卡接口是3.3V供电

最高25MB/s的传输速度

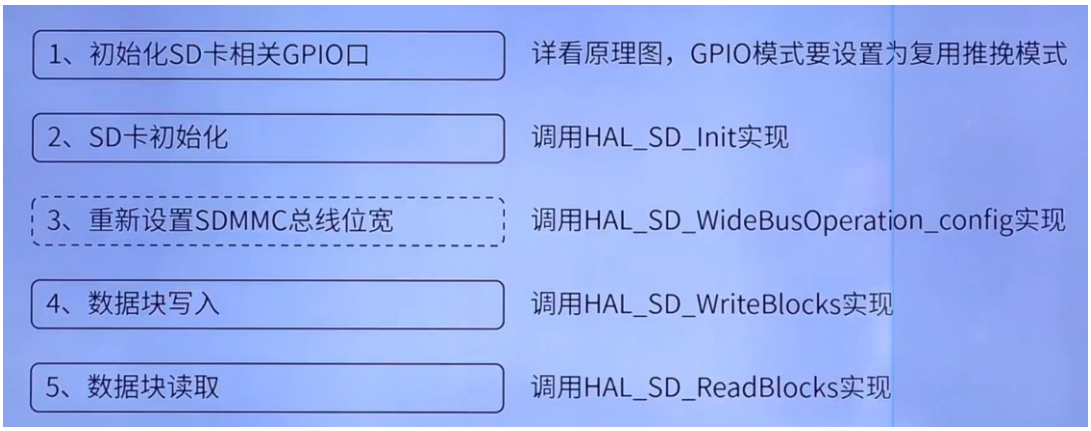
H750 SDMMC框图





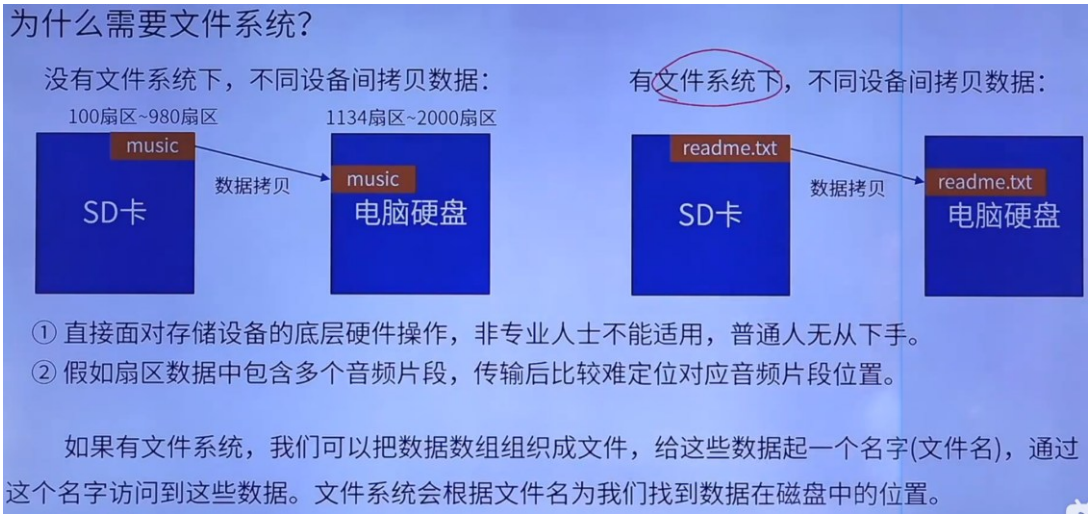
2.6 SDMMC 相关 HAL 库驱动

2.7 SDMMC 驱动 SD 卡步骤



3 FATFS

3.1 文件系统基本介绍



3.2 FATFS 文件系统简介

文件系统是为了存储和管理数据，而在存储设备上建立的一种组织结构。

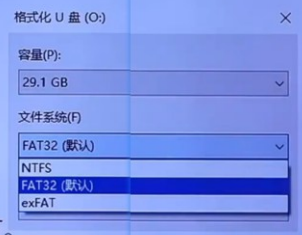
Windows常用的文件系统：

FAT12、FAT16、FAT32

exFAT

NTFS

FAT：File Alloction Table
文件分配表



在小型的嵌入式存储设备大多使用的是FAT32和exFAT。

使用文件系统前，需先对存储设备进行格式化，擦除原来的数据，在存储设备上建立一个文件分配表和目录。

FAT文件系统简介

FAT32文件系统布局

系统引导扇区：引导程序，以及文件系统信息(扇区字节数/每簇扇区数/保留扇区数等)

文件分配表：记录文件存储中簇与簇之间连接的信息

根目录：存在所有文件和子目录信息(文件名/文件夹名/创建时间/文件大小)

数据区：文件等数据存放地方，占用大部分的磁盘空间

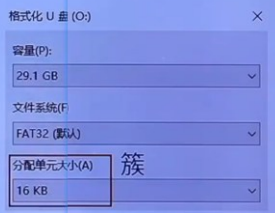
详细内容参考《FAT32文件系统详解》

FAT文件系统简介

2的整数次幂

FAT文件系统用“簇”作为数据单元，一个“簇”由一组连续的扇区组成，而一个扇区的大小为512字节。

所有的簇从2开始进行编号，每个簇都有自己的地址编号，用户文件和数据都存储在簇中。



簇

FATFS是专门用于小型嵌入式系统的通用FAT/exFAT文件系统模块。标准C语言编写，具有良好的硬件平台独立性，简单修改就可移植到单片机上。

Windows/DOS系统兼容的FAT/exFAT文件系统

独立于硬件平台，方便跨硬件平台移植

代码量少、效率高

多种配置选项

支持ANSI/OEM或Unicode中的长文件名

exFAT文件系统、64位LBA和GPT，适用于大型存储

适用于实时操作系统的线程安全

多个卷（物理驱动器和分区）

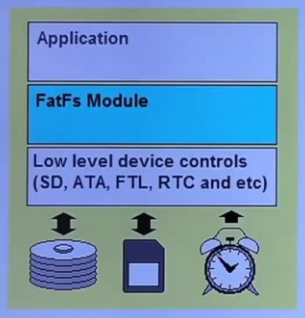
多个代码页，包括DBCS

只读，可选API、I/O缓冲区等

FATFS是可裁剪的文件系统

下载链接 http://elm-chan.org/fsw/ff/archives.html

FATFS模块的层次结构图



① 底层接口(Low level device controls)

存储媒介读/写接口(disk I/O)和供给文件创建修改时间的实时时钟，需要我们根据平台和存储介质编写移植代码

② FATFS模块(FatFs Module)

实现了FAT文件读/写协议.FATFS模块提供的是ff.c和ff.h。

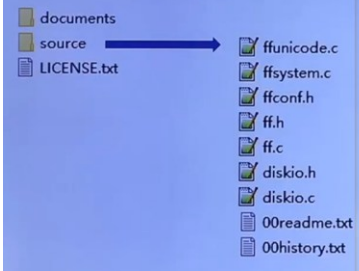
除非有必要，使用者一般不用修改，使用时将头文件直接包含进去。

③ 应用层(Application)

使用者无需理会FATFS的内容结构和复杂的FAT协议，只需要调用FATFS模块提供给用户的一系列应用接口函数，如f_open，f_read，f_write和f_close等，就可以像在PC机上读/写文件。

下载地址：http://elm-chan.org/fsw/ff/00index_e.html（资料包下载/FATFS说明/函数调用）

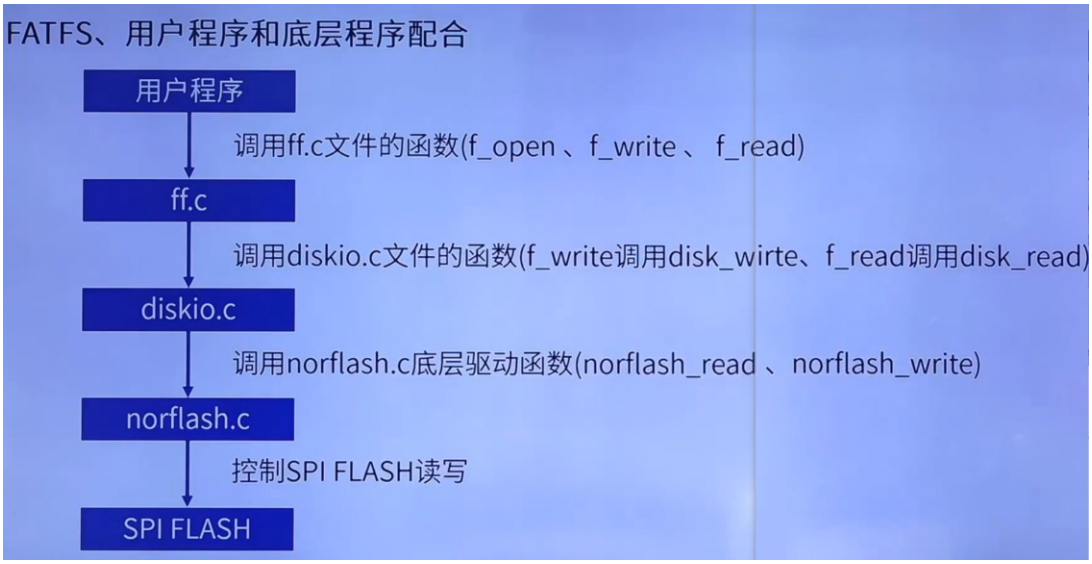
FATFS文件系统包结构



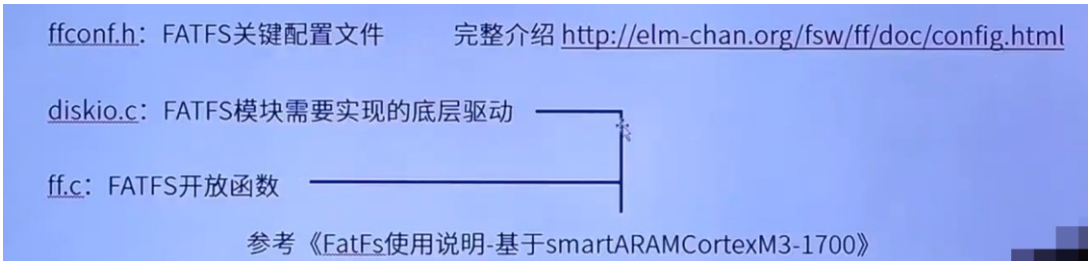
文件名	功能名	说明
ffconf.h	FATFS模块配置文件	根据需求来配置
ff.h	FATFS和应用模块共用的包含文件	不需要修改
ff.c	FATFS模块源码(文件系统API)	不需要修改
diskio.h	FATFS和disk I/O模块共用的包含文件	不需要修改
diskio.c	FATFS和disk I/O模块接口层文件	与硬件相关代码
ffunicode.c	FATFS所支持的字体代码转换表	不需要修改
ffsystem.c	FATFS的OS相关函数示例代码	没用到

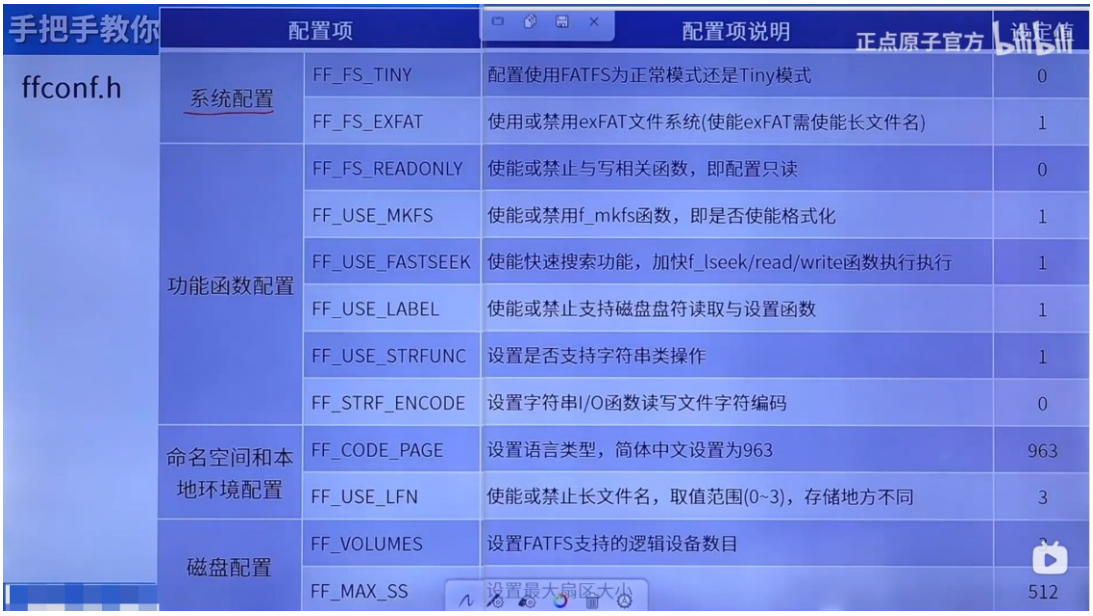
FATFS文件系统的移植需要修改2个文件，ffconf.h和diskio.c。

第三方库的移植，基本上需要用户编写底层驱动源码，然后提供上层配置文件供配置。



3.3 FATFS 配置项和函数介绍





配置项		配置项说明	缺省值
系统配置	FF_FS_TINY	配置使用FATFS为正常模式还是Tiny模式	0
	FF_FS_EXFAT	使用或禁用exFAT文件系统(使能exFAT需使能长文件名)	1
功能函数配置	FF_FS_READONLY	使能或禁止与写相关函数，即配置只读	0
	FF_USE_MKFS	使能或禁用f_mkfs函数，即是否使能格式化	1
	FF_USE_FASTSEEK	使能快速搜索功能，加快f_lseek/read/write函数执行执行	1
	FF_USE_LABEL	使能或禁止支持磁盘盘符读取与设置函数	1
	FF_USE_STRFUNC	设置是否支持字符串类操作	1
	FF_STRF_ENCODE	设置字符串I/O函数读写文件字符编码	0
命名空间和本地环境配置	FF_CODE_PAGE	设置语言类型，简体中文设置为963	963
	FF_USE_LFN	使能或禁止长文件名，取值范围(0~3)，存储地方不同	3
磁盘配置	FF_VOLUMES	设置FATFS支持的逻辑设备数目	
	FF_MAX_SS	设置最大扇区大小	512

diskio.c文件 需要实现的函数

disk_initialize	初始化磁盘驱动器
disk_status	获取磁盘状态
disk_read	从磁盘驱动器读扇区
disk_write	从磁盘驱动器写扇区
disk_ioctl	控制设备实现指定功能，用于辅助FATFS中其他API
get_fattime	获取当前时间

disk_initialize函数

函数名称	disk_initialize
函数原型	DSTATUS disk_initialize(BYTE Drive)
功能描述	初始化磁盘驱动器
函数参数	Drive：指定要初始化的逻辑驱动器号，即盘符，应当取值 0~9
返回值	函数返回一个磁盘状态作为结果，对于磁盘状态的细节信息，请参考 disk_status 函数
所在文件	ff.c
示例	disk_initialize(0); /* 初始化驱动器 0 */
注意事项	disk_initialize 函数初始化一个逻辑驱动器为读/写做准备，函数成功时，返回值的 STA_NOINIT 标志被清零； 应用程序不应调用此函数，否则卷上的 FAT 结构可能会损坏； 如果需要重新初始化文件系统，可使用 f_mount 函数； 在 FatFs 模块上卷注册处理时调用该函数可控制设备的改变； 此函数在 FatFs 挂在卷时调用，应用程序不应该在 FatFs 活动时使用此函数

disk_status函数

函数名称	disk_status
函数原型	DSTATUS disk_status(BYTE Drive)
功能描述	返回当前磁盘驱动器的状态
函数参数	Drive：指定要确认的逻辑驱动器号，即盘符，应当取值 0~9
返回值	磁盘状态返回下列标志的组合，FatFs 只使用 STA_NOINIT 和 STA_PROTECTED STA_NOINIT：表明磁盘驱动未初始化，下面列出了产生该标志置位或清零的原因： 置位：系统复位，磁盘被移除和磁盘初始化函数失败； 清零：磁盘初始化函数成功 STA_NODISK：表明驱动器中没有设备，安装磁盘驱动器后总为 0 STA_PROTECTED：表明设备被写保护，不支持写保护的设备总为 0，当 STA_NODISK 置位时非法
所在文件	ff.c
示例	disk_status(0); /* 获取驱动器 0 的状态

disk_read函数

函数名称	disk_read
函数原型	DRESULT disk_read(BYTE Drive, BYTE* Buffer, DWORD SectorNumber, BYTE SectorCount)
功能描述	从磁盘驱动器上读取扇区
函数参数	Drive：　指定逻辑驱动器号，即盘符，应当取值 0~9 Buffer：　指向存储读取数据字节数组的指针，需要为所读取字节数的大小，扇区统计的扇区大小是需要的 注：FatFs 指定的内存地址并不总是字对齐的，如果硬件不支持不对齐的数据传输，函数里需要进行处理 SectorNumber：指定起始扇区的逻辑块（LBA）上的地址 SectorCount：　指定要读取的扇区数，取值 1~128
返回值	RES_OK(0)：　函数成功 RES_ERROR：　读操作期间产生了任何错误且不能恢复它 RES_PARERR：　非法参数 RES_NOTRDY：　磁盘驱动器没有初始化
所在文件	ff.c

disk_write函数

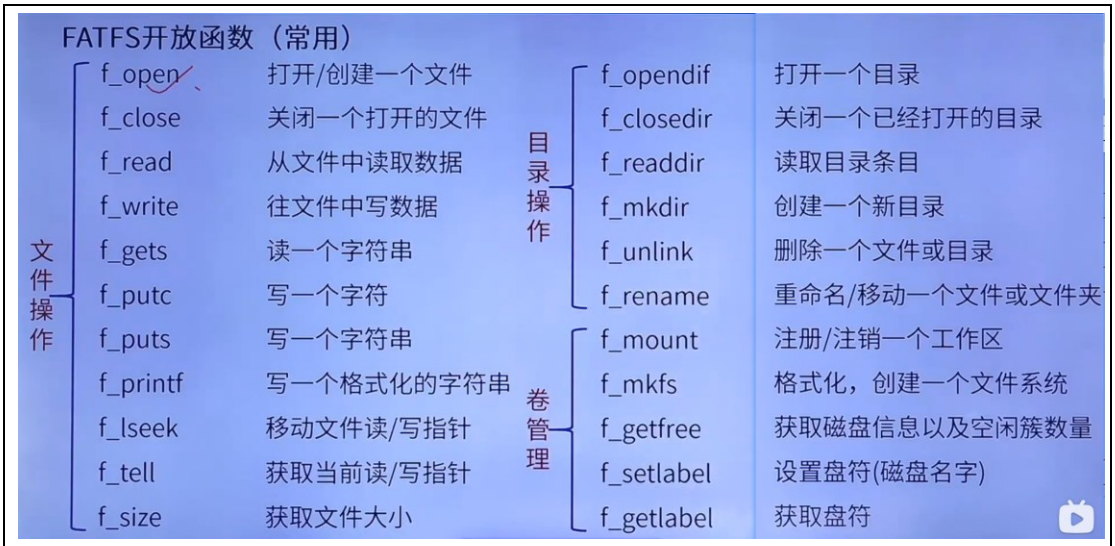
函数名称	disk_write
函数原型	DRESULT disk_write(BYTE Drive, const BYTE* Buffer, DWORD SectorNumber, BYTE SectorCount)
功能描述	向磁盘写入一个或多个扇区
函数参数	Drive：　指定逻辑驱动器号，即盘符，应当取值 0~9 Buffer：指向要写入字节数组的指针， 注：FatFs 指定的内存地址并不总是字对齐的，如果硬件不支持不对齐的数据传输，函数里需要进行处理 SectorNumber：　指定起始扇区的逻辑块（LBA）上的地址 SectorNumber：　指定要写入的扇区数，取值 1~128
返回值	RES_OK(0)：　函数成功 RES_ERROR：　读操作期间产生了任何错误且不能恢复它 RES_WRPRT：　媒体被写保护 RES_PARERR：　非法参数 RES_NOTRDY：磁盘驱动器没有初始化
所在文件	ff.c
注意事项	只读配置中不需要此函数

disk_ioctl函数

函数名称	disk_ioctl
函数原型	DRESULT disk_ioctl(BYTE Drive, BYTE Command, void* Buffer)
功能描述	控制设备指定特性和除了读/写外的杂项功能
函数参数	Drive：　指定逻辑驱动器号，即盘符，应当取值 0~9 Command：指定命令代码 Buffer：　指向参数缓冲区的指针，取决于命令代码，不使用时，指定一个 NULL 指针
返回值	RES_OK(0)：　函数成功 RES_ERROR：读操作期间产生了任何错误且不能恢复它 RES_PARERR：非法参数 RES_NOTRDY：磁盘驱动器没有初始化
所在文件	ff.c
注意事项	CTRL_SYNC：　确保磁盘驱动器已经完成了写处理，当磁盘 I/O 有一个写回缓存，立即刷新原扇区，只读配置下不适用此命令 GET_SECTOR_SIZE：　返回磁盘的扇区大小，只用于 f_mkfs() GET_SECTOR_COUNT：返回可利用的扇区数，_MAX_SS >= 1024 时可用 GET_BLOCK_SIZE：　获取擦除块大小，只用于 f_mkfs() CTRL_ERASE_SECTOR：强制擦除一块的扇区，_USE_ERASE >0 时可用

get_fattime函数　　ffconf.h文件中FF_FS_NOTRTC宏为0时需要实现

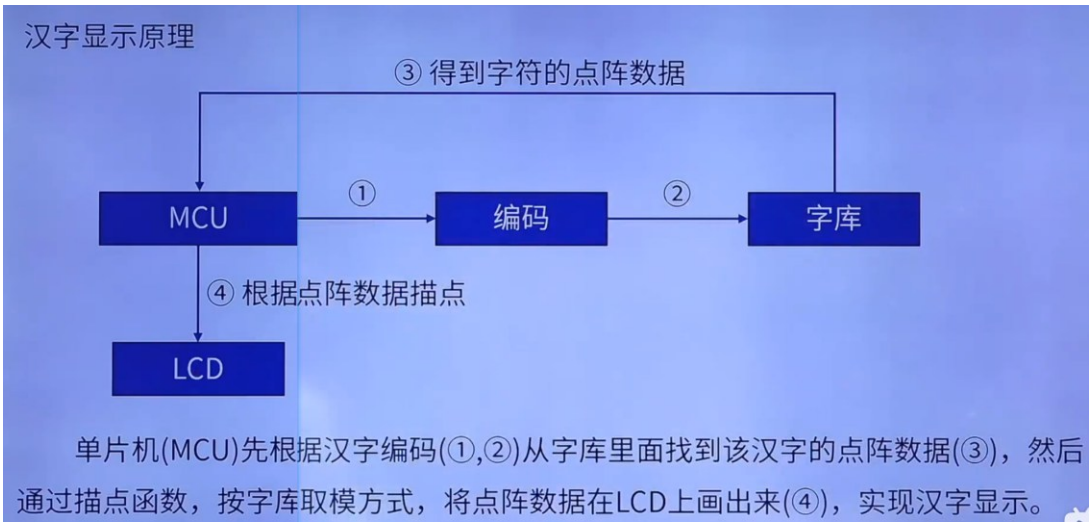
函数名称	get_fattime
函数原型	DWORD get_fattime()
功能描述	获取当前时间
函数参数	无
返回值	当前时间以双字值封装返回，位域如下： bit31:25　年　（0~127）（从 1980 开始） bit24:21　月　（1~12） bit20:16　日　（1~31） bit15:11　小时（0~23） bit10:5　分钟（0~59） bit4:0　秒　（0~29）
所在文件	ff.c
注意事项	get_fattime 函数必须返回一个合法的时间即使系统不支持实时时钟，如果返回 0，文件没有一个合法的时间； 只读配置下无需此函数



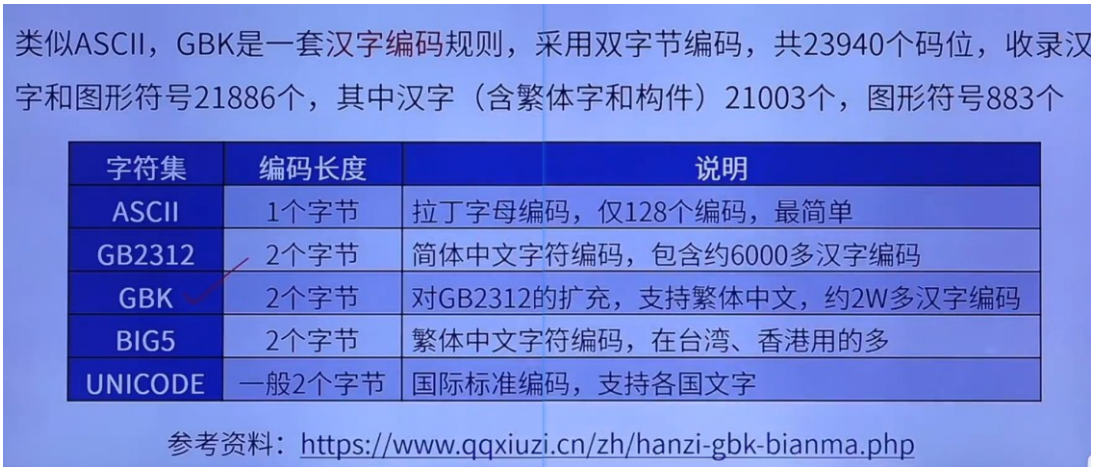
3.4 FATFS 最简单移植步骤

4 汉字显示

4.1 汉字显示原理介绍

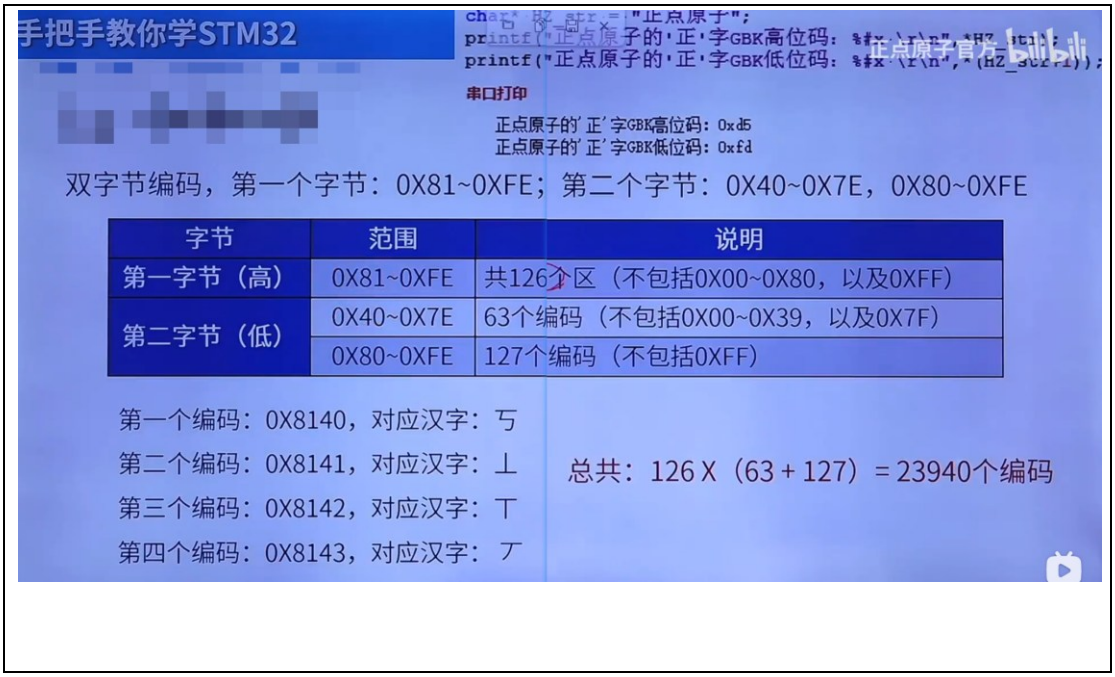


4.1.1 什么是 GBK 编码

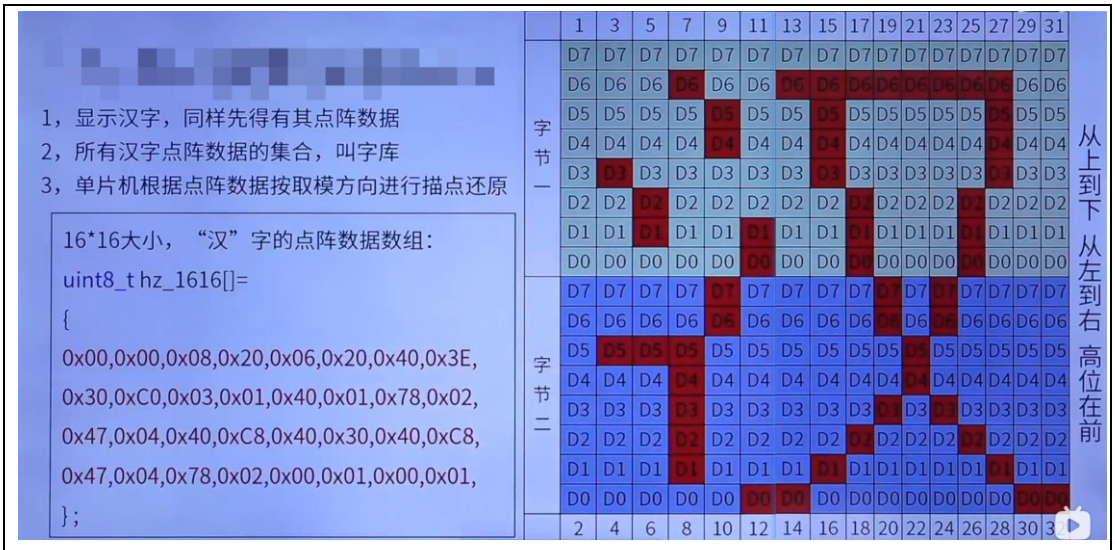


字符集	编码长度	说明
ASCII	1个字节	拉丁字母编码，仅128个编码，最简单
GB2312	2个字节	简体中文字符编码，包含约6000多汉字编码
GBK	2个字节	对GB2312的扩充，支持繁体中文，约2W多汉字编码
BIG5	2个字节	繁体中文字符编码，在台湾、香港用的多
UNICODE	一般2个字节	国际标准编码，支持各国文字

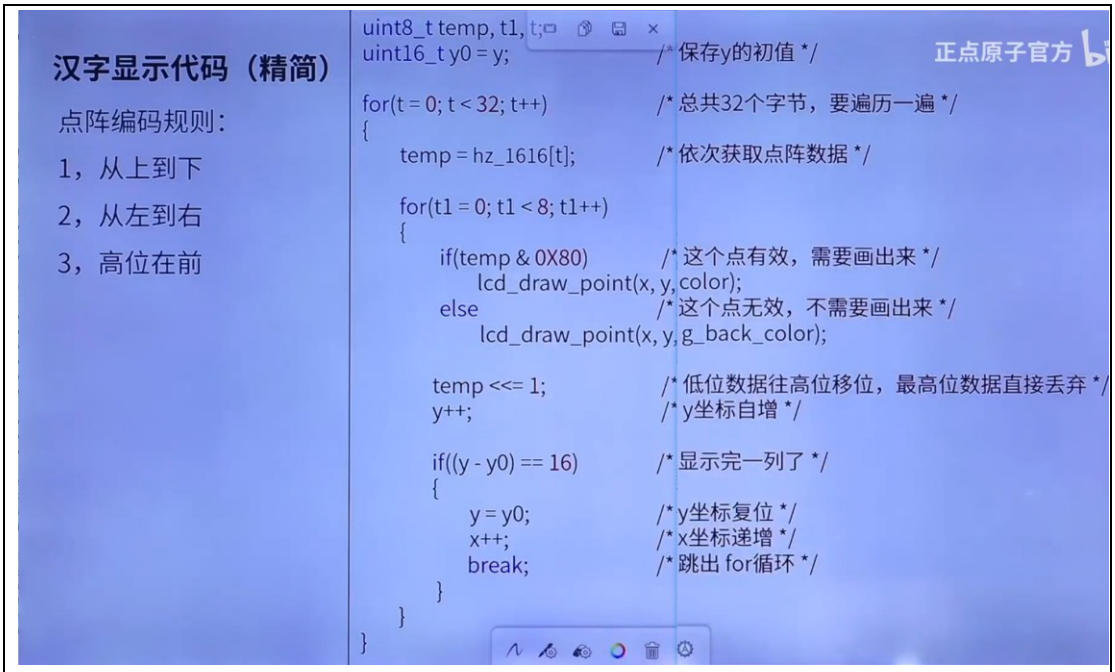
4.1.2 GBK 编码规则



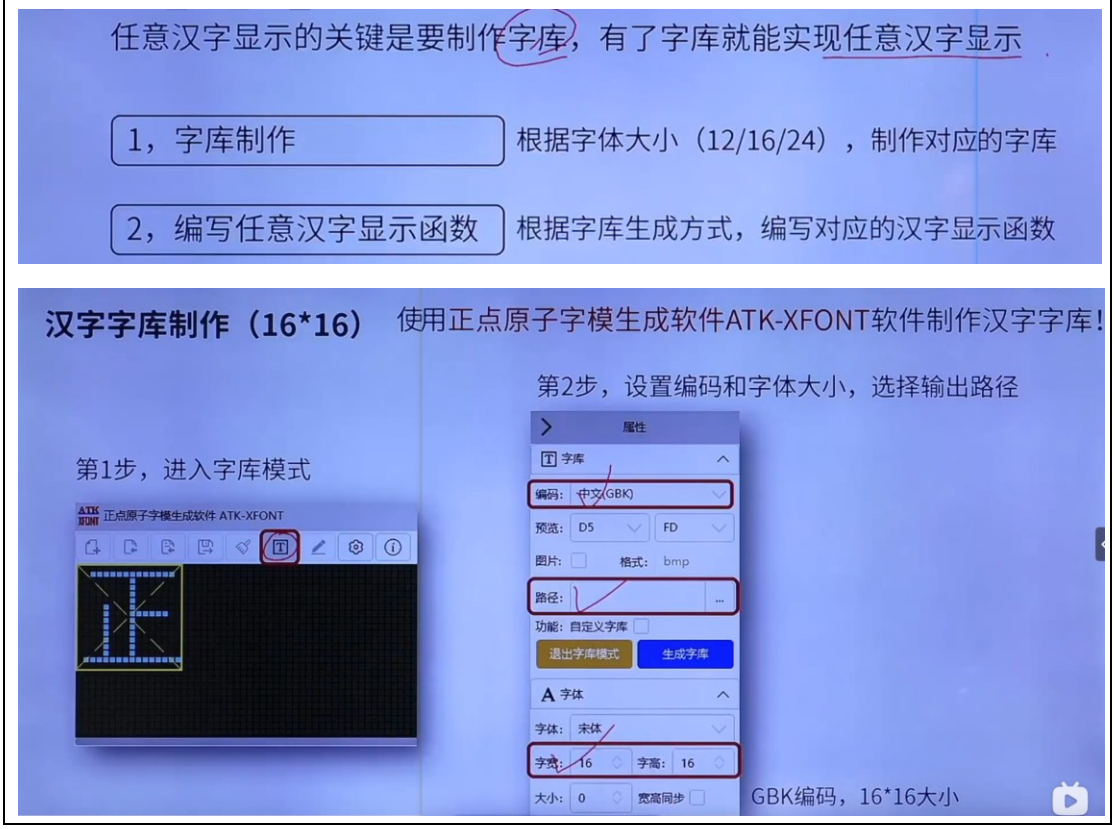
4.1.3 如何将汉字显示在 LCD 上

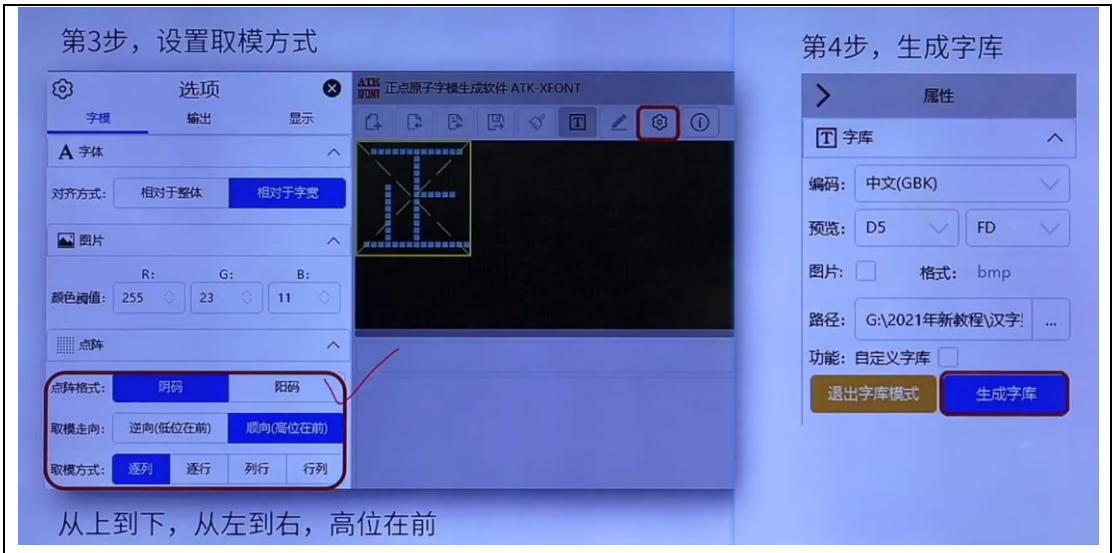


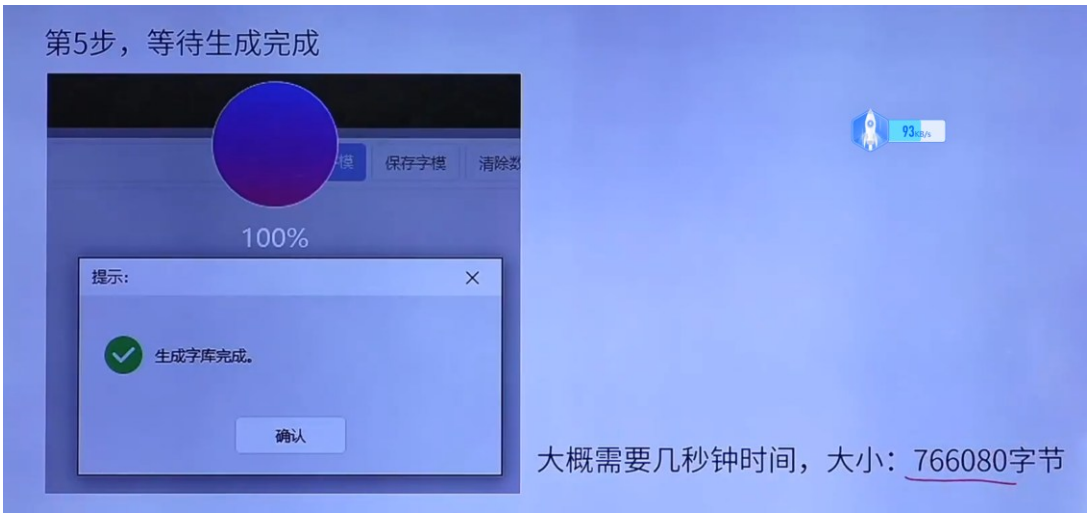
4.1.4 汉字显示代码（精简）



4.1.5 任意汉字显示



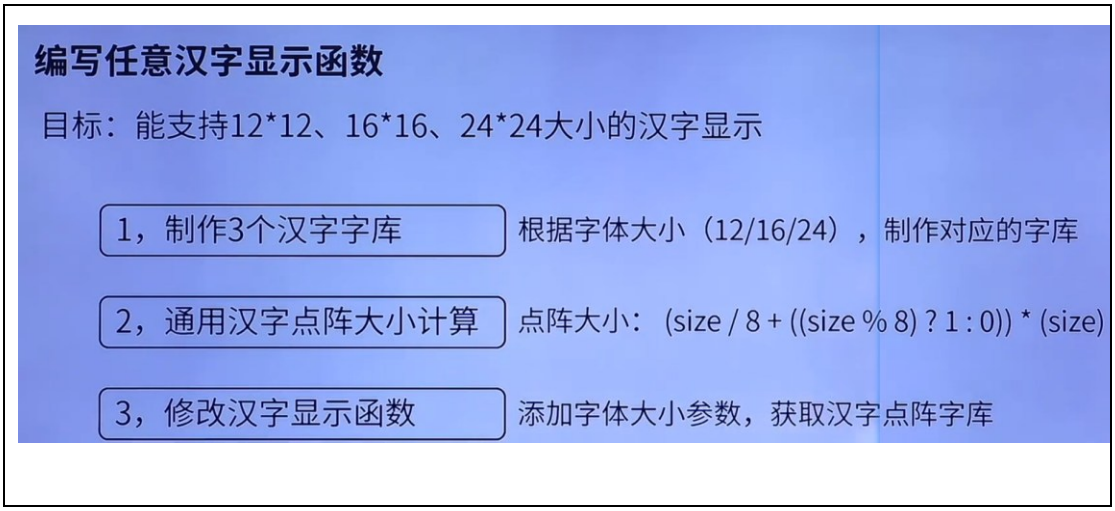




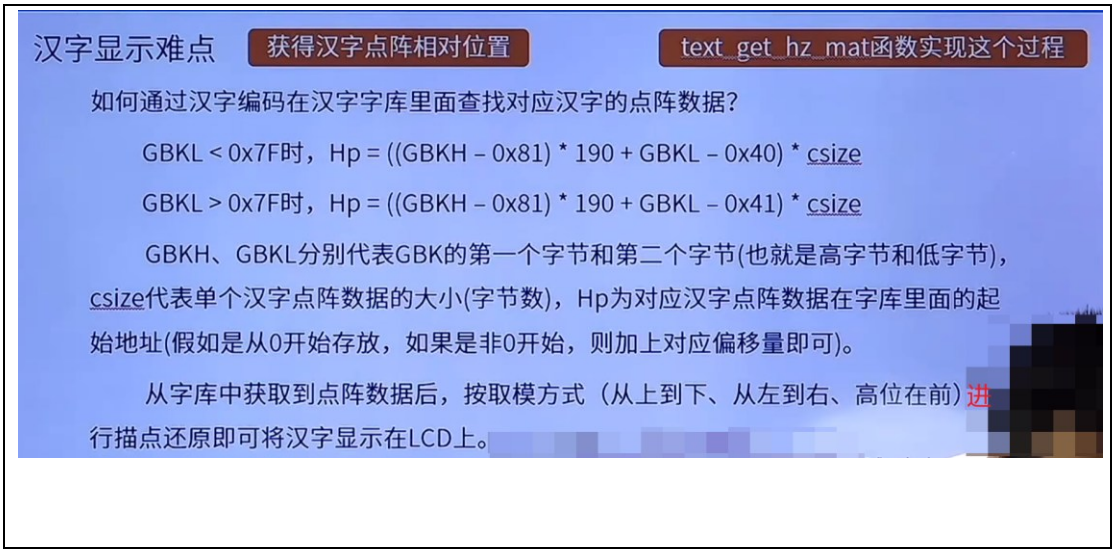
第6步，将字库名字重命名为：GBK16.FON

然后，拷贝到TF卡，通过TF卡将字库更新到SPI FLASH里面备用！(汉字显示实验)

4.1.6 编写任意汉字显示函数



4.1.7 汉字显示难点



5 图片显示

5.1 常见图片格式介绍

5.1.1 常见图片格式

图片格式	文件后缀名	优点	缺点	说明
BMP	.bmp	一般不压缩、画质最好	随分辨率和颜色数增加，占用空间急剧增大	图像色深可选：1、4、8、16、24、32bit
JPEG (JPG)	.jpeg（.jpg)	压缩技术先进、支持多种压缩级别、压缩比高(有损压缩)、文件体积较小	画质有一定损失	在不影响人类可分辨的图片质量的前提下，尽可能的压缩文件大小
GIF	.gif	支持有/无损压缩、压缩比高、支持动画、支持背景透明、文件小	仅支持8位色深(256色)、画质差	文件小，利于网络传输

5.1.2 BMP

BMP全称：Bitmap(位图)，是windows中的标准图像文件格式

采用位映射存储方式，不做任何压缩，图像深度可选：1、4、8、16、24、32bit

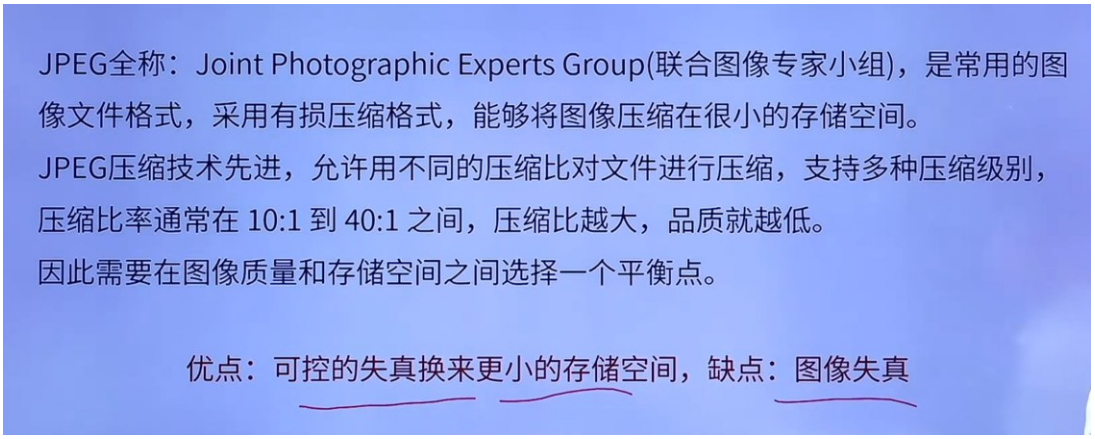
BMP文件存储数据时，图像的扫描方式是按从左到右、从下到上的顺序

优点：不失真，画质好，缺点：占用空间大

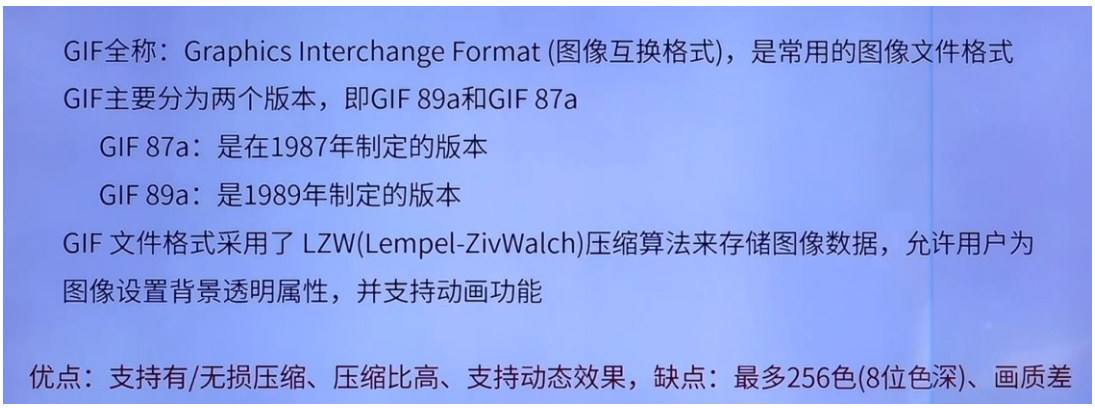
BMP图像文件数据结构由四部分组成：位图文件头、位图信息头、颜色表、位图数据

详细介绍参考：《BMP图片文件详解.pdf》

5.1.3 JPEG(JPG)

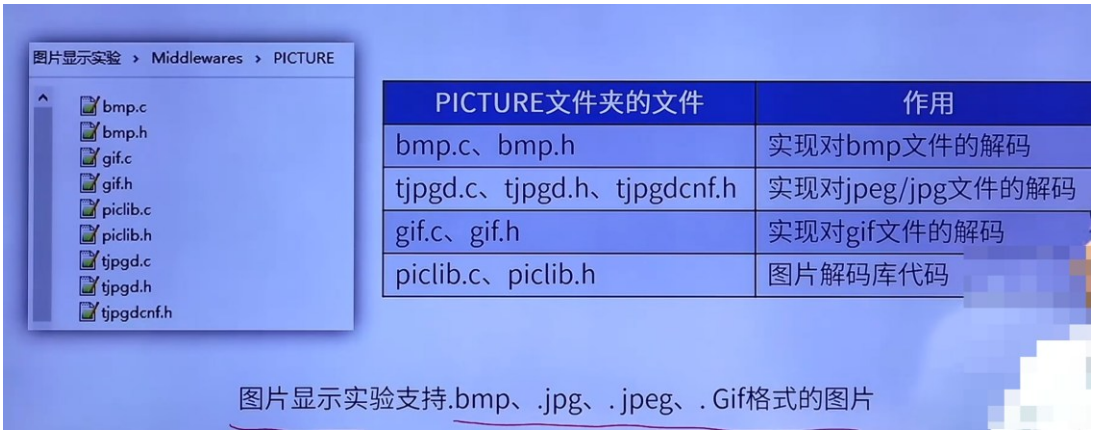


5.1.4 GIF

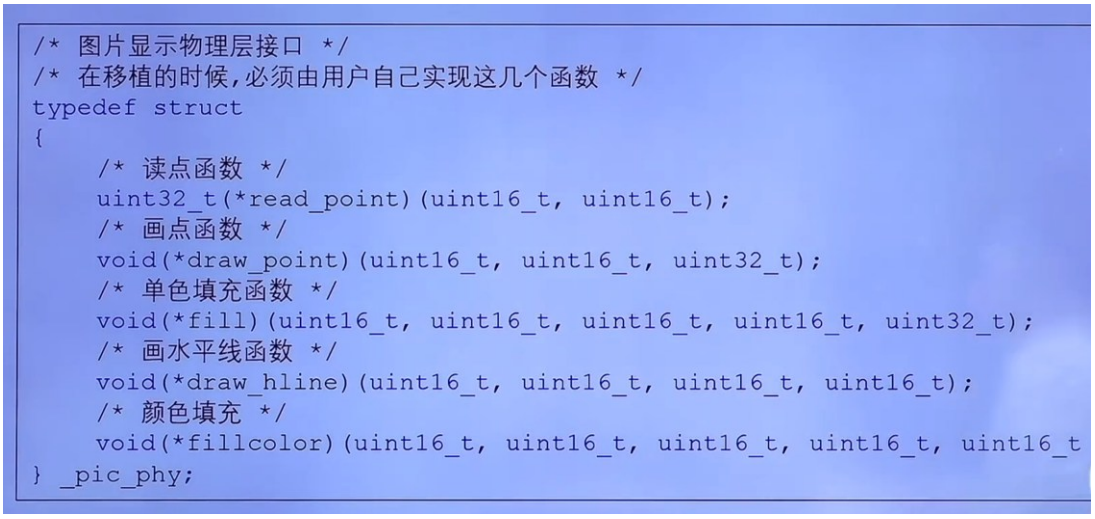


5.2 图片显示

5.2.1 图片显示实验文件简介



5.2.2 图片显示物理层接口结构体



```
/* 图片显示物理层接口 */
/* 在移植的时候,必须由用户自己实现这几个函数 */
typedef struct
{
    /* 读点函数 */
    uint32_t(*read_point)(uint16_t, uint16_t);
    /* 画点函数 */
    void(*draw_point)(uint16_t, uint16_t, uint32_t);
    /* 单色填充函数 */
    void(*fill)(uint16_t, uint16_t, uint16_t, uint16_t, uint32_t);
    /* 画水平线函数 */
    void(*draw_hline)(uint16_t, uint16_t, uint16_t, uint16_t);
    /* 颜色填充 */
    void(*fillcolor)(uint16_t, uint16_t, uint16_t, uint16_t, uint16_t
} _pic_phy;
```

5.2.3 图片信息结构体

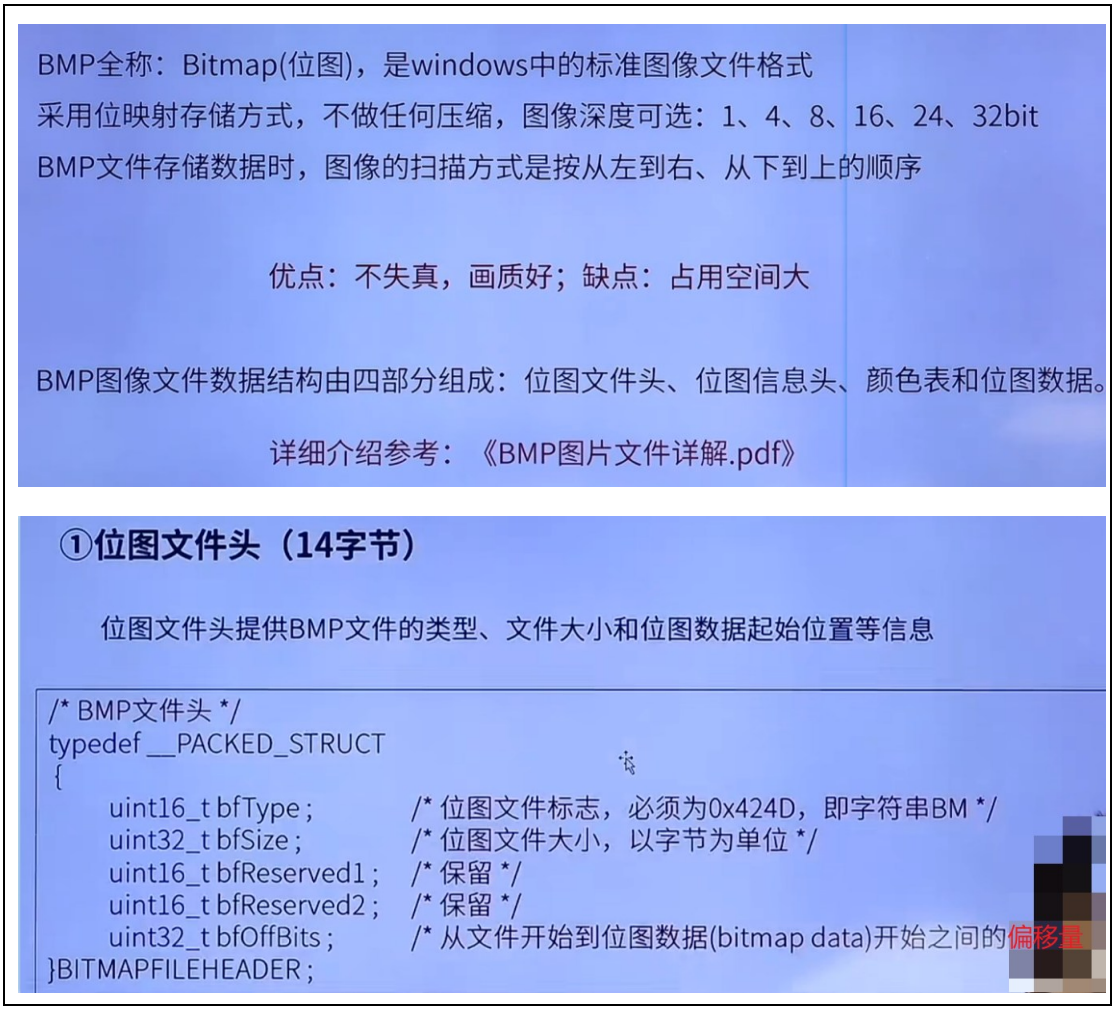
```
/* 图像信息 */
typedef struct
{
    uint16_t lcdwidth;      /* LCD的宽度 */
    uint16_t lcdheight;     /* LCD的高度 */
    uint32_t ImgWidth;      /* 图像的实际宽度和高度 */
    uint32_t ImgHeight;
    uint32_t Div_Fac;       /* 缩放系数 (扩大了8192倍的) */
    uint32_t S_Height;      /* 设定的高度和宽度 */
    uint32_t S_Width;
    uint32_t S_XOFF;        /* x轴和y轴的偏移量 */
    uint32_t S_YOFF;
    uint32_t staticx;       /* 当前显示到的 x y 坐标 */
    uint32_t staticy;
} _pic_info;
```

5.2.4 图片解码库接口函数

函数	作用
piclib_init()	画图初始化函数，初始化_pic_info和_pic_phy结构体变量
piclib_alpha_blend()	快速ALPHA BLENDING算法函数，用于实现半透明效果
piclib_ai_draw_init()	初始化智能画点函数，用于设置图片居中显示
piclib_is_element_ok()	判断一个像素是否应该显示出来，在图片缩放时用到
piclib_ai_load_picfile()	智能画图函数，实现bmp/jpg/jpeg和gif格式图片的显示
*piclib_mem_malloc()	动态分配内存函数
piclib_mem_free()	释放内存函数

6 照相机

6.1 BMP 编码简介



②位图信息头（40字节）

正点原子官方 bilibi

位图信息头提供BMP图片的尺寸等信息

```
/* BMP信息头 */
typedef __PACKED_STRUCT
{
    uint32_t biSize;            /* 说明BITMAPINFOHEADER结构所需要的字数 */
    long biWidth;               /* 说明图象的宽度，以像素为单位 */
    long biHeight;              /* 说明图象的高度，以像素为单位 */
    uint16_t biPlanes;          /* 为目标设备说明位面数，其值将总是被设为1 */
    uint16_t biBitCount;        /* 说明比特数/像素，其值为1、4、8、16、24、32 */
    uint32_t biCompression; /* 说明图像数据压缩的类型。其值可以是下述值之一：
        * BI_RGB(0)： 没有压缩
        * BI_RLE8(1)： 每个像素8比特的RLE压缩编码，压缩格式由2字节组成(重复像素计数和颜色索引)
        * BI_RLE4(2)： 每个像素4比特的RLE压缩编码，压缩格式由2字节组成
        * BI_BITFIELDS(3)：每个像素的比特由指定的掩码决定 */
    uint32_t biSizeImage;   /* 说明图像的大小，以字节为单位。当用BI_RGB格式时，可设置为0 */
    long biXPelsPerMeter;   /* 说明水平分辨率，用像素/米表示 */
    long biYPelsPerMeter;   /* 说明垂直分辨率，用像素/米表示 */
    uint32_t biClrUsed;     /* 说明位图实际使用的彩色表中的颜色索引数 */
    uint32_t biClrImportant; /* 说明对图像显示有重要影响的颜色索引的数目，如果是0，表示都重要 */
}BITMAPINFOHEADER;
```

③ 颜色表（4字节）(可选)

颜色表用于说明位图的颜色，它有若干个表项，每一个表项是一个RGBQUAD类型的结构，定义一种颜色

```
typedef __PACKED_STRUCT
{
    uint8_t rgbBlue;        /* 指定蓝色强度 */
    uint8_t rgbGreen;       /* 指定绿色强度 */
    uint8_t rgbRed;         /* 指定红色强度 */
    uint8_t rgbReserved;    /* 保留，设置为0 */
}RGBQUAD;
```

颜色表中 RGBQUAD 结构数据的个数由 biBitCount 来确定：当 biBitCount=1、4、8时，分别有2、16、256个表项；当 biBitCount 大于 8 时，没有颜色表项

颜色表存在时，位图数据中存储的是各个像素的色彩在调色板中的索引值，查表才知道真实颜色

位图信息

位图文件头、位图信息头和颜色表组成位图信息

```
/* 位图信息 */
typedef __PACKED_STRUCT
{
    BITMAPFILEHEADER bmfHeader;   /* 我们将位图文件头也加进来，方便处理 */
    BITMAPINFOHEADER bmiHeader;   /* 位图信息*/
    uint32_t RGB_MASK[3];         /* 调色板用于存放RGB掩码 */
    //RGBQUAD bmiColors[256];     /* 颜色表*/
}BITMAPINFO;
```

④ 位图数据

位图数据记录了位图的每一个像素值，记录顺序是在扫描行内是从左到右，扫描行之间是从下到上。位图的一个像素值所占的字节数如下：

当 biBitCount=1 时， 8 个像素占 1 个字节；

当 biBitCount=4 时， 2 个像素占 1 个字节；

当 biBitCount=8 时， 1 个像素占 1 个字节；

当 biBitCount=16 时， 1 个像素占 2 个字节；

当 biBitCount=24 时， 1 个像素占 3 个字节；

当 biBitCount=32 时， 1 个像素占 4 个字节；

biBitCount=16，即高彩色(16位色)。当biCompression=BI_RGB(0)，则采用RGB555格式，最高位恒为0；当biCompression= BI_BITFIELDS(3)，则在原来调色板位置用3个DWORD类型的掩码替换，分别代表红、绿、蓝三色的掩码，一般是： 0XF800、0X07E0、0X001F

16位BMP编码

实验中采用16位BMP编码（因为LCD就是16位色的，而且16位BMP编码比24位BMP编码更省空间），故设置biBitCount的值为16，这样得到新的位图信息（BITMAPINFO）结构体

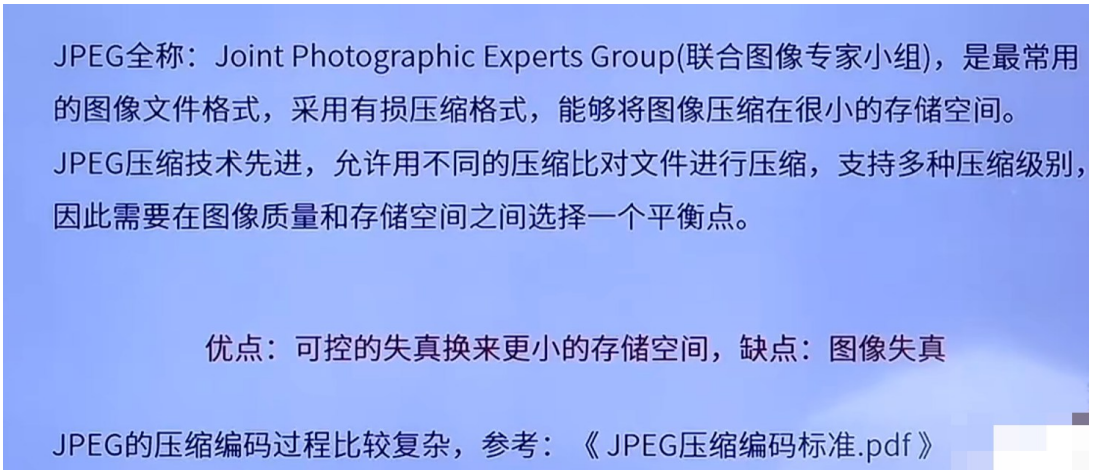
```
/* 位图信息 */
typedef __PACKED_STRUCT
{
    BITMAPFILEHEADER bmfHeader;   /* 我们将位图文件头也加进来，方便处理 */
    BITMAPINFOHEADER bmiHeader;   /* 位图信息*/
    uint32_t RGB_MASK[3];         /* 调色板用于存放RGB掩码 */
}BITMAPINFO;
```

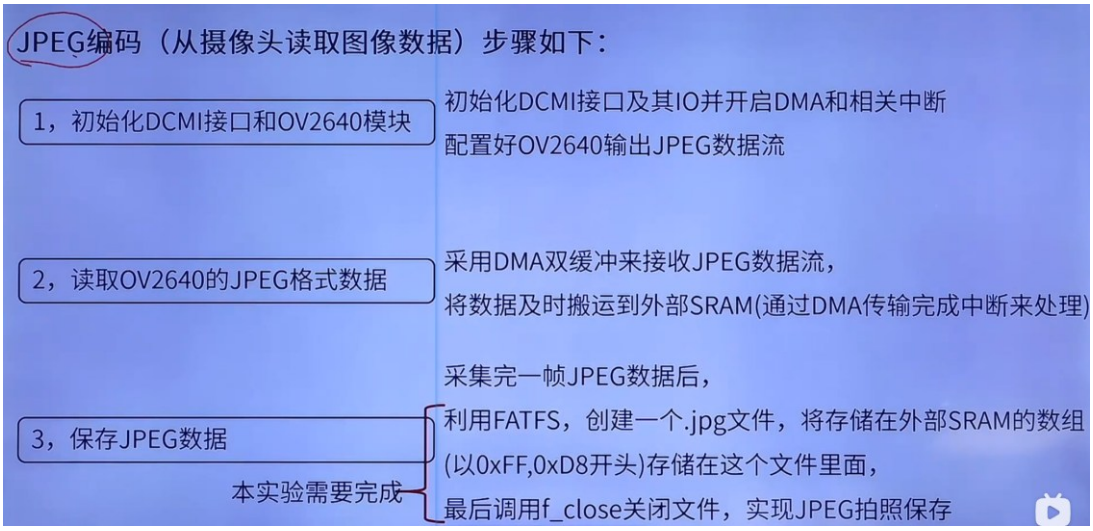
RGB_MASK[3]，即颜色掩码，分别代表红、绿、蓝三色的掩码，分别是：0XF800、0X07E0、0X001F

BMP编码（从LCD读取图像数据）步骤如下：

1，创建BMP位图信息，并初始化各个相关信息	设置BMP图片的大小(整个BMP文件大小)、BMP图片的高度和宽度、BMP图片的像素位数(16位)和掩码等信息
2，创建新BMP文件，写入BMP位图信息	创建文件，并保存BMP位图信息，方便后续BMP数据的写入
3，保存位图数据	从LCD的GRAM里读取各点的颜色值，依次写入第二步创建的BMP文件即可。保存顺序(读GRAM顺序)从左到右，从下到上
4，关闭文件	在文件创建后，必须调用f_close关闭，文件才会真正体现在文件系统里面，否则是不会写入的!

6.2 JPEG 编码简介

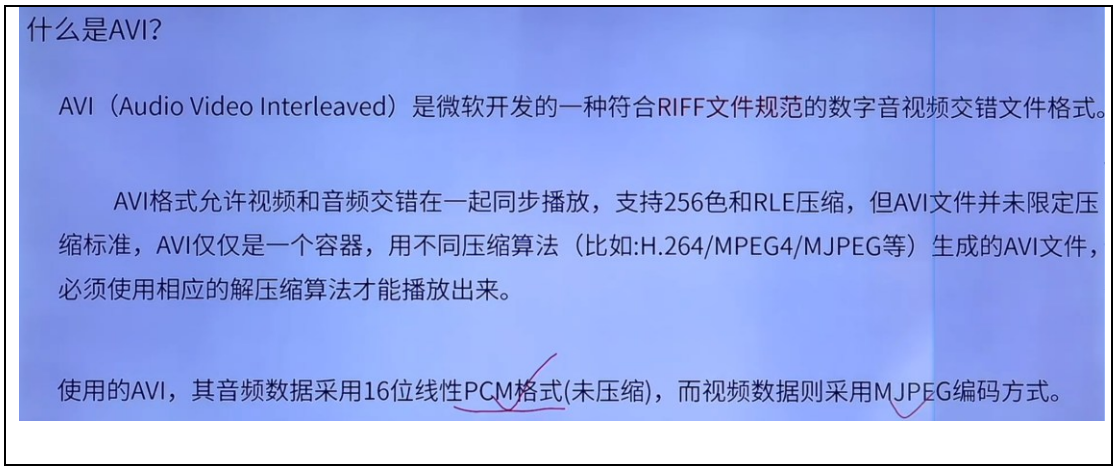

JPEG全称：Joint Photographic Experts Group(联合图像专家小组)，是最常用的图像文件格式，采用有损压缩格式，能够将图像压缩在很小的存储空间。
JPEG压缩技术先进，允许用不同的压缩比对文件进行压缩，支持多种压缩级别，因此需要在图像质量和存储空间之间选择一个平衡点。
优点：可控的失真换来更小的存储空间，缺点：图像失真
JPEG的压缩编码过程比较复杂，参考：《JPEG压缩编码标准.pdf》


JPEG编码（从摄像头读取图像数据）步骤如下：
1，初始化DCMI接口和OV2640模块
初始化DCMI接口及其IO并开启DMA和相关中断
配置好OV2640输出JPEG数据流
2，读取OV2640的JPEG格式数据
采用DMA双缓冲来接收JPEG数据流，
将数据及时搬运到外部SRAM(通过DMA传输完成中断来处理)
采集完一帧JPEG数据后，
3，保存JPEG数据
利用FATFS，创建一个.jpg文件，将存储在外部SRAM的数组(以0xFF,0xD8开头)存储在这个文件里面，
最后调用f_close关闭文件，实现JPEG拍照保存
本实验需要完成

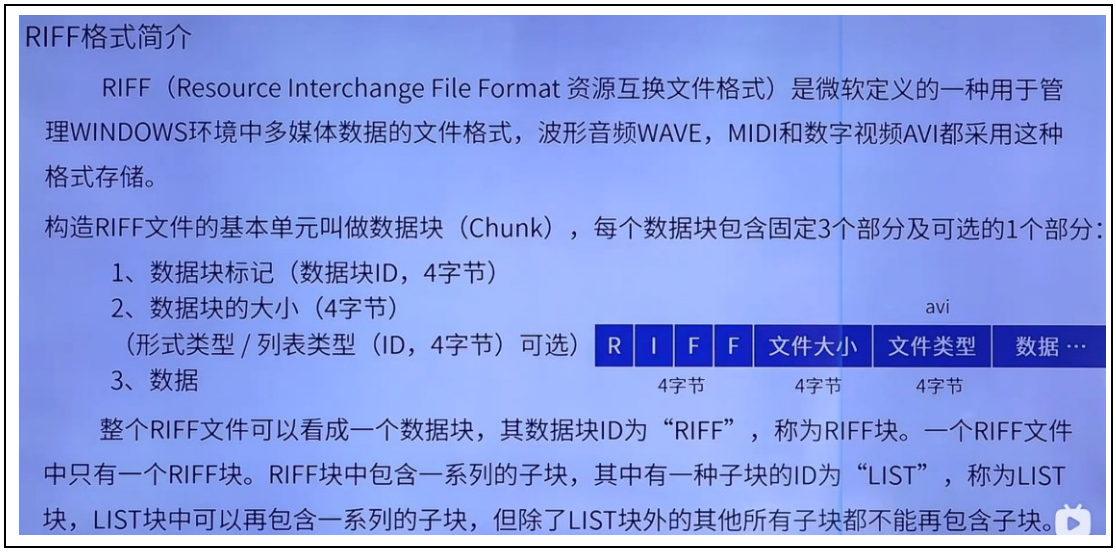
7 视频播放器

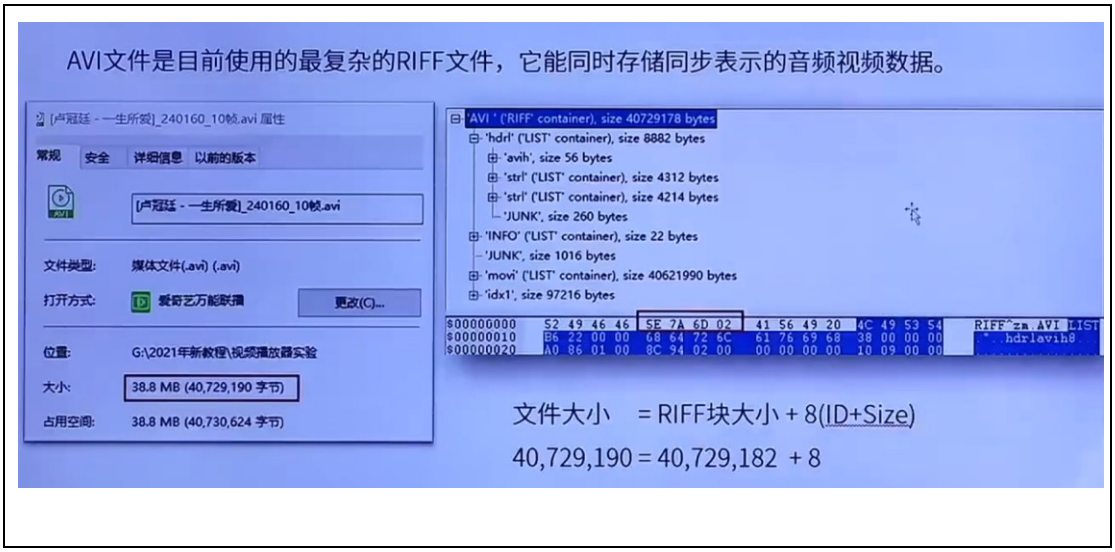
7.1 AVI 简介

7.1.1 什么是 AVI

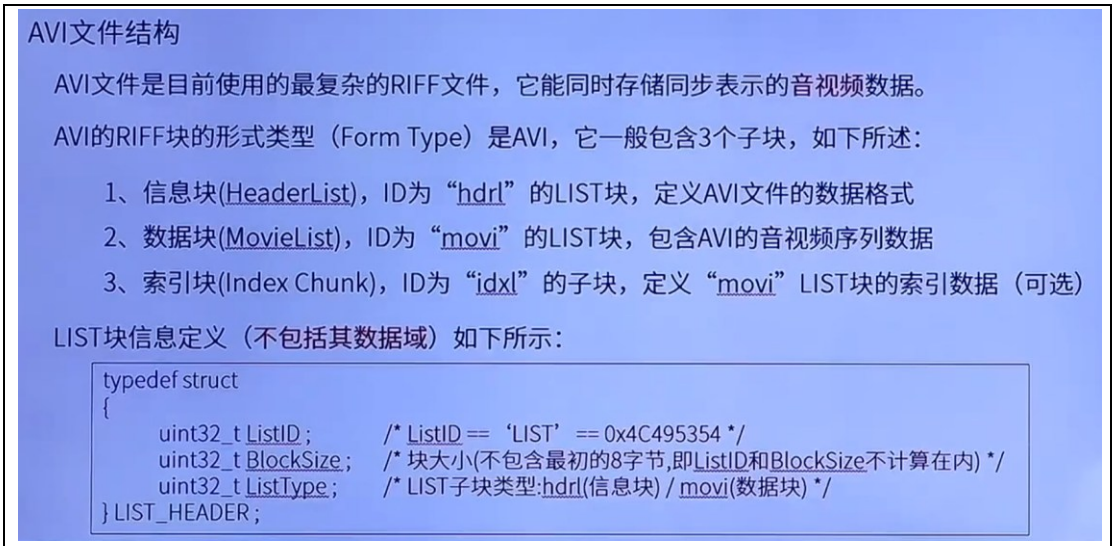


7.1.2 RIFF 格式简介

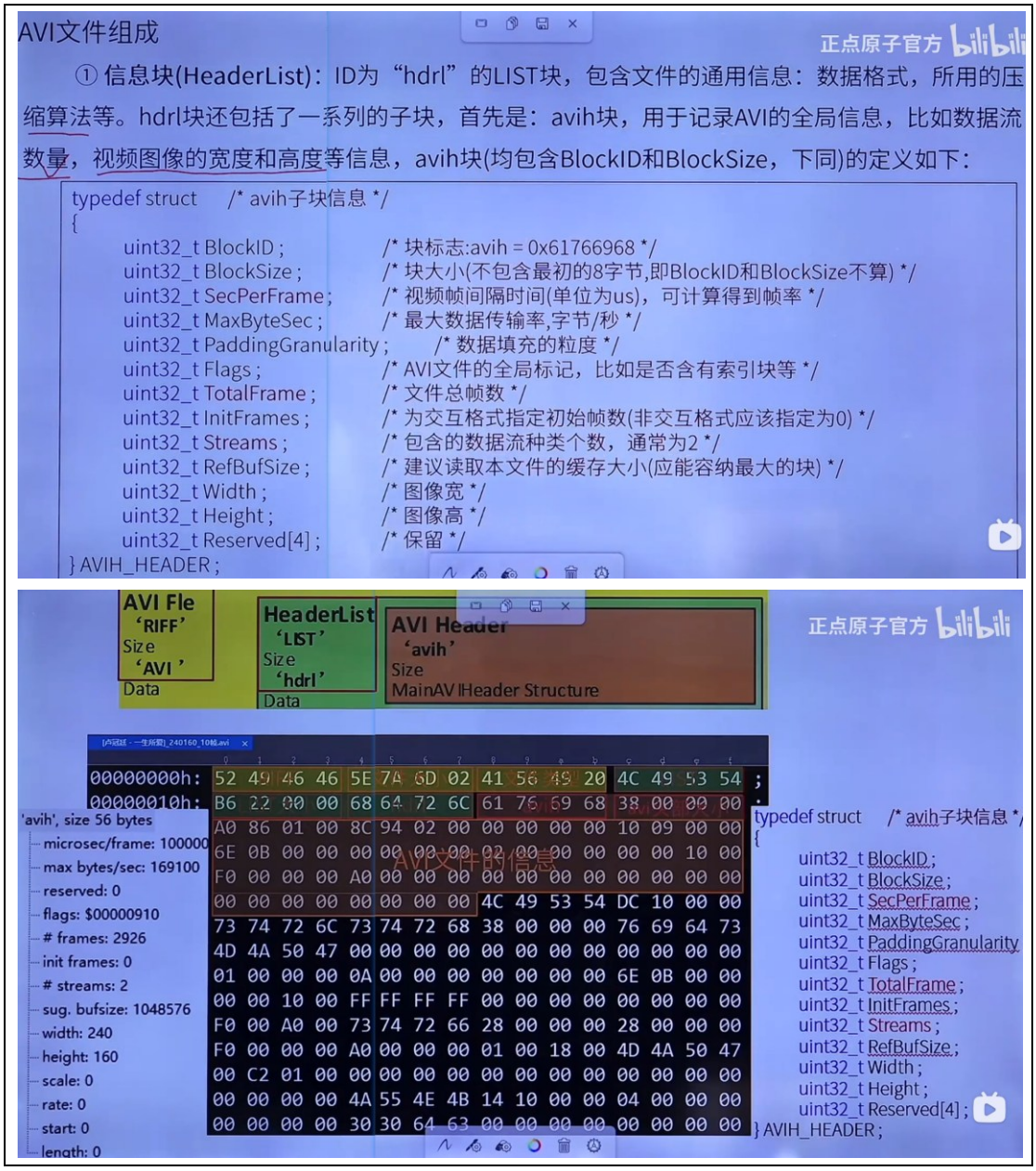



AVI文件是目前使用的最复杂的RIFF文件，它能同时存储同步表示的音频视频数据。
[卢冠廷 - 一生所爱]_240160_10帧.avi 属性
常规
安全
详细信息
以前的版本
[卢冠廷 - 一生所爱]_240160_10帧.avi
文件类型:
媒体文件(.avi) (.avi)
打开方式:
爱奇艺万能联播
更改(C)...
位置:
G:\2021年新教程\视频播放器实验
大小:
38.8 MB (40,729,190 字节)
占用空间:
38.8 MB (40,730,624 字节)
'AVI ' ('RIFF' container), size 40729178 bytes
'hdrl' ('LIST' container), size 8882 bytes
'avih', size 56 bytes
'strl' ('LIST' container), size 4312 bytes
'strl' ('LIST' container), size 4214 bytes
'JUNK', size 260 bytes
'INFO' ('LIST' container), size 22 bytes
'JUNK', size 1016 bytes
'movi' ('LIST' container), size 40621990 bytes
'idx1', size 97216 bytes
$00000000 52 49 46 46 5E 7A 6D 02 41 56 49 20 4C 49 53 54 RIFF^zm.AVI LIST
$00000010 B6 22 00 00 68 64 72 6C 61 76 69 68 38 00 00 00 "..hdrlavih8...
$00000020 A0 86 01 00 8C 94 02 00 00 00 00 00 10 09 00 00
文件大小 = RIFF块大小 + 8(ID+Size)
40,729,190 = 40,729,182 + 8

7.1.3 AVI 文件结构


AVI文件结构
AVI文件是目前使用的最复杂的RIFF文件，它能同时存储同步表示的音视频数据。
AVI的RIFF块的形式类型（Form Type）是AVI，它一般包含3个子块，如下所述：
1、信息块(HeaderList)，ID为“hdrl”的LIST块，定义AVI文件的数据格式
2、数据块(MovieList)，ID为“movi”的LIST块，包含AVI的音视频序列数据
3、索引块(Index Chunk)，ID为“idxl”的子块，定义“movi”LIST块的索引数据（可选）
LIST块信息定义（不包括其数据域）如下所示：
typedef struct
{
uint32_t ListID; /* ListID == ‘LIST’ == 0x4C495354 */
uint32_t BlockSize; /* 块大小(不包含最初的8字节,即ListID和BlockSize不计算在内) */
uint32_t ListType; /* LIST子块类型:hdrl(信息块) / movi(数据块) */
} LIST_HEADER;

7.1.4 AVI 文件组成

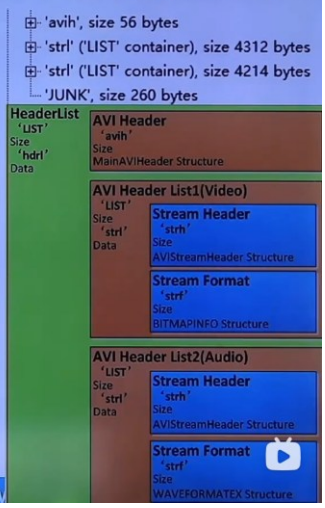

AVI文件组成
正点原子官方 bilibili
① 信息块(HeaderList)：ID为“hdrl”的LIST块，包含文件的通用信息：数据格式，所用的压缩算法等。hdrl块还包括了一系列的子块，首先是：avih块，用于记录AVI的全局信息，比如数据流数量，视频图像的宽度和高度等信息，avih块(均包含BlockID和BlockSize，下同)的定义如下：
typedef struct /* avih子块信息 */
{
uint32_t BlockID; /* 块标志:avih = 0x61766968 */
uint32_t BlockSize; /* 块大小(不包含最初的8字节,即BlockID和BlockSize不算) */
uint32_t SecPerFrame; /* 视频帧间隔时间(单位为us)，可计算得到帧率 */
uint32_t MaxByteSec; /* 最大数据传输率,字节/秒 */
uint32_t PaddingGranularity; /* 数据填充的粒度 */
uint32_t Flags; /* AVI文件的全局标记，比如是否含有索引块等 */
uint32_t TotalFrame; /* 文件总帧数 */
uint32_t InitFrames; /* 为交互格式指定初始帧数(非交互格式应该指定为0) */
uint32_t Streams; /* 包含的数据流种类个数，通常为2 */
uint32_t RefBufSize; /* 建议读取本文件的缓存大小(应能容纳最大的块) */
uint32_t Width; /* 图像宽 */
uint32_t Height; /* 图像高 */
uint32_t Reserved[4]; /* 保留 */
} AVIH_HEADER;
AVI Fle
'RIFF'
Size
'AVI '
Data
HeaderList
'LIST'
Size
'hdrl'
Data
AVI Header
'avih'
Size
MainAVIHeader Structure
正点原子官方 bilibili
00000000h: 52 49 46 46 5E 7A 6D 02 41 56 49 20 4C 49 53 54 ;
00000010h: B6 22 00 00 68 64 72 6C 61 76 69 68 38 00 00 00 ;
A0 86 01 00 8C 94 02 00 00 00 00 00 10 09 00 00
6E 0B 00 00 00 00 00 00 02 00 00 00 00 00 10 00
F0 00 00 00 A0 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 00 00 00 00 4C 49 53 54 DC 10 00 00
73 74 72 6C 73 74 72 68 38 00 00 00 76 69 64 73
4D 4A 50 47 00 00 00 00 00 00 00 00 00 00 00 00
01 00 00 00 0A 00 00 00 00 00 00 00 6E 0B 00 00
00 00 10 00 FF FF FF FF 00 00 00 00 00 00 00 00
F0 00 A0 00 73 74 72 66 28 00 00 00 28 00 00 00
F0 00 00 00 A0 00 00 00 01 00 18 00 4D 4A 50 47
00 C2 01 00 00 00 00 00 00 00 00 00 00 00 00 00
00 00 00 00 4A 55 4E 4B 14 10 00 00 04 00 00 00
00 00 00 00 30 30 64 63 00 00 00 00 00 00 00 00
'avih', size 56 bytes
microsec/frame: 100000
max bytes/sec: 169100
reserved: 0
flags: $00000910
frames: 2926
init frames: 0
streams: 2
sug. bufsize: 1048576
width: 240
height: 160
scale: 0
rate: 0
start: 0
length: 0
AVI文件的信息
typedef struct /* avih子块信息 */
{
uint32_t BlockID;
uint32_t BlockSize;
uint32_t SecPerFrame;
uint32_t MaxByteSec;
uint32_t PaddingGranularity;
uint32_t Flags;
uint32_t TotalFrame;
uint32_t InitFrames;
uint32_t Streams;
uint32_t RefBufSize;
uint32_t Width;
uint32_t Height;
uint32_t Reserved[4];
} AVIH_HEADER;

在avih块之后，有一个或者多个strl子列表，文件中有多少种数据流（即前面的Streams），就有多少个strl子列表。

每个strl子列表，至少包含一个strh(Stream Header)块和一个strf(Stream Format)块，另有strn(Stream Name)块(可选)。

注意：不同类型的strl子列表内容不同，且出现的顺序与媒体流的编号是对应的（比如：00dc，前面的00，即媒体流编号00）。

比如第一个strl子列表说明的是第一个流(Stream 0)，假设是视频流，则表征视频数据块的四字符码为“00dc”，第二个strl子列表说明的是第二个流（Stream 1），假设是音频流，则表征音频数据块的四字符码位“01wb”，以此类推。



“原子哥”在线教学平台：www.yuanzige.com 技术交流论坛：http://

strh子列表情况，音频和视频内容会不同

正点原子官方 BiliBili

```
typedef struct {
    uint32_t BlockID;          /* 块标志:strh = 0x73747268 */
    uint32_t BlockSize;        /* 块大小(不包含最初的8字节,即BlockID和BlockSize不算)*/
    uint32_t StreamType;       /* 数据流种类，vids(0x73646976):视频 auds(0x73647561):音频 */
    uint32_t Handler;          /* 指定流的处理者，即解码器，如MJPEG/H264等*/
    uint32_t Flags;            /* 标记：是否允许这个流输出？调色板是否变化？ */
    uint16_t Priority;         /* 流优先级(有多个同类型的流时优先级最高的为默认流) */
    uint16_t Language;         /* 音频的语言代号 */
    uint32_t InitFrames;       /* 为交互格式指定初始帧数 */
    uint32_t Scale;            /* 数据量，视频每帧的大小或者音频的采样大小 */
    uint32_t Rate;             /* scale/rate = 每秒采样数 */
    uint32_t Start;            /* 数据流开始播放的位置，单位为scale */
    uint32_t Length;           /* 数据流的数据量，单位为scale */
    uint32_t RefBufSize;       /* 建议使用的缓冲区大小 */
    uint32_t Quality;          /* 解压缩质量参数，值越大，质量越好 */
    uint32_t SampleSize;       /* 音频的样本大小 */
    struct {                   /* 视频帧所占的矩形 */
        short Left;
        short Top;
        short Right;
        short Bottom;
    } Frame;
} STRH_HEADER;        /* 说明这个流的头信息 */
```

Handler为1时即为音频流，由strf确定音频格式

```
'strl' ('LIST' container), size 4312 bytes
'strh', size 56 bytes
  fccType: vids
  fccHandler: MJPG
  flags: $00000000
  reserved1: 0
  init frames: 0
  scale: 1
  rate: 10
  start: 0
  length: 2926
  sug bufsize: 1048576
  quality: -1
  sample size: 0
  reserved2: 0
  reserved3: 10486000
```

strl子列表为视频流情况

正点原子官方 BiliBili

```
typedef struct {
    uint32_t BmpSize;          /* bmp结构体大小(包含BmpSize在内) */
    long Width;                /* 图像宽 */
    long Height;               /* 图像高 */
    uint16_t Planes;           /* 平面数，必须为1 */
    uint16_t BitCount;         /* 像素位数，0x0018表示24位*/
    uint32_t Compression;      /* 压缩类型，比如:MJPG/H264等 */
    uint32_t SizeImage;        /* 图像大小*/
    long XpixPerMeter;         /* 水平分辨率 */
    long YpixPerMeter;         /* 垂直分辨率 */
    uint32_t ClrUsed;          /* 实际使用了调色板中的颜色数，压缩格式中不使用 */
    uint32_t ClrImportant;     /* 重要的颜色 */
} BMP_HEADER;
typedef struct {               /* 颜色表*/
    uint8_t rgbBlue;           /* 蓝色的亮度(值范围为0~255) */
    uint8_t rgbGreen;          /* 绿色的亮度(值范围为0~255) */
    uint8_t rgbRed;            /* 红色的亮度(值范围为0~255) */
    uint8_t rgbReserved;       /* 保留，必须为0 */
} AVIRGBQUAD;
typedef struct {               /* 对于strh,如果是视频流strf(流格式)是STRH_BMPHEADER块 */
    uint32_t BlockID;          /* 块标志，strf==0x73747266 */
    uint32_t BlockSize;        /* 块大小(不含前8字节，即BlockID和BlockSize不算) */
    BMP_HEADER bmiHeader;      /* 位图信息头 */
    AVIRGBQUAD bmColors[1];    /* 颜色表 */
} STRF_BMPHEADER;
```

视频分辨率

所用编码器

```
'strf', size 40 bytes
  size: 40
  width: 240
  height: 160
  planes: 1
  bit count: 24
  compression: MJPG
  size image: $0001c200
  XPelsPerMeter: 0
  YPelsPerMeter: 0
  clr used: 0
  biclr important: 0
'JUNK', size 4116 bytes
'vprp', size 68 bytes
```

如果strh子块是音频数据流(Steam Type = “auds”)，strf子块的内容定义情况：

```
typedef struct          /* 对于strh，如果是音频流，strf(流格式)是STRF_WAVHEADER块 */
{
    uint32_t BlockID;       /* 块标志:strf = 0x73747266 */
    uint32_t BlockSize;     /* 块大小(不包含最初的8字节,即BlockID和BlockSize不算)*/
    uint16_t FormatTag;     /* 格式标志：0x0001=PCM, 0x0055=MP3 */
    uint16_t Channels;      /* 声道数，一般为2，表示立体声 */
    uint32_t SampleRate;    /* 音频采样率 */
    uint32_t BaudRate;      /* 波特率 */
    uint16_t BlockAlign;    /* 数据块对齐标志 */
    uint16_t Size;          /* 结构体大小 */
} STRF_WAVHEADER;
```

'strf', size 18 bytes
- format tag: 1
- channels: 2
- samples/sec: 11025
- avg bytes/sec: 44100
- block align: 4
- bits/sample: 16

本结构体对音频数据解码起决定性的作用，告诉我们音频信号的编码方式(FormatTag)、声道数(Channels)和采样率(SampleRate)等重要信息。例程仅支持PCM格式(FormatTag=0x0001)的音频数据解码。

② 数据块（MovieList），即ID为“movi”的LIST块，它包含AVI的音视频序列数据，是这个AVI文件的主体部分。音视频数据块交错的嵌入在“movi”LIST块里面，通过标准类型码进行区分，标准类型码有如下4种：

1、“##db”（非压缩视频帧）
2、“##dc”（压缩视频帧）
3、“##pc”（改用新的调色板）
4、“##wb”（音频帧）

其中##是编号，得根据数据流顺序来确定，也就是前面的strl块。比如，如果第一个strl块是视频数据，那么对于视频帧，标准类型码就是：00dc。第二个strl块是音频数据，那么对于音频帧，标准类型码就是：01wb。

紧跟标准类型码的是4字节的数据长度(不含类型码和长度参数本身)，该长度必须是偶数，如果读到为奇数，则加1。读数据时，一般一次性要读完一个标准类型码所表征的数据，方便解码。

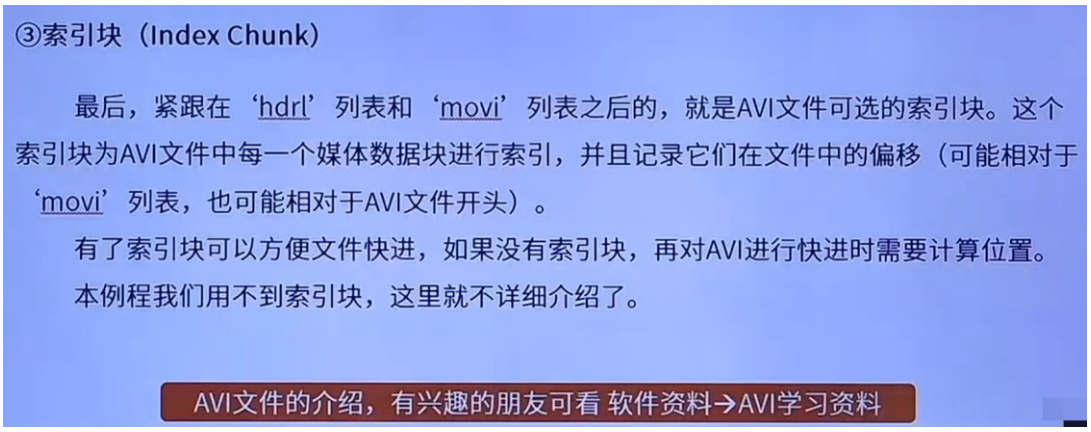
数据块(movi)

'movi' ('LIST' container), size 40621990 bytes
- '01wb', size 4096 bytes
- '01wb', size 4096 bytes
- '01wb', size 4096 bytes
- '00dc', size 0 bytes
- '00dc', size 0 bytes
- '00dc', size 4644 bytes
- '01wb', size 4096 bytes
- '00dc', size 6554 bytes
- '01wb', size 4096 bytes
- '00dc', size 7470 bytes
- '01wb', size 4096 bytes
- '00dc', size 7540 bytes
- '01wb', size 4096 bytes
- '00dc', size 7582 bytes
- '01wb', size 4096 bytes
- '00dc', size 7588 bytes
- '01wb', size 4096 bytes
- '00dc', size 7606 bytes
- '01wb', size 4096 bytes

第一帧是音频数据，大小为4096字节。

数据块ID

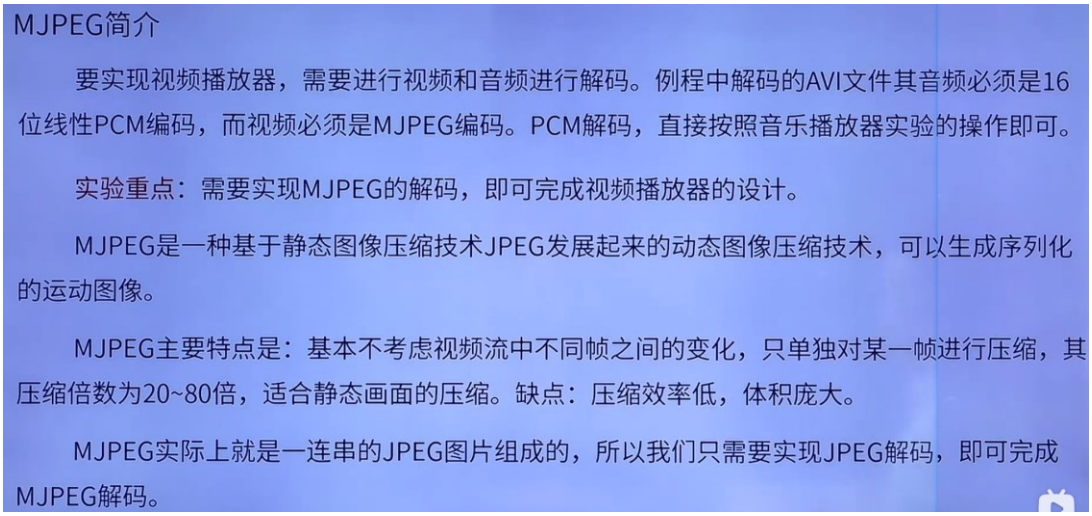
```
000026e0h: 00 00 00 00 00 00 00 00 00 00 00 00 4C 49 53 54 ; ............LIST
000026f0h: AA D7 6B 02 6D 6F 76 69 30 31 77 62 00 10 00 00 ;   k.movi01wb....
00002700h: 01 00 01 00 01 00 02 00 01 00 03 00 01 00 06 00 ; ................
00002710h: 00 00 0E 00 02 00 1B 00 0F 00 2D 00 2A 00 47 00 ; ..........-.*.G.
00002720h: 4F 00 6D 00 76 00 A0 00 99 00 D3 00 AC 00 F4 00 ; O.m.v.?????
00002730h: AF 00 0A 01 AB 00 28 01 9F 00 38 01 5F 00 F4 00 ; ?..?(.?8._.?
00002740h: AF FF 2A 00 86 FE F4 FE E4 FC 80 FD D2 FA D5 FB ; ?*.嘁酤潼€    ?
00002750h: 9D F8 EB F9 5E F6 AD F7 27 F4 4E F5 7E F2 BE F3 ; 濾膛^霏?鼍羔蚪?
00002760h: 9B F1 6F F3 4A F1 AE F3 FC F1 56 F4 6A F4 18 F6 ; 湮o驤癱簏駒餾??
00002770h: A9 F8 65 F9 3E FE 5B FE A1 04 48 04 48 0B E9 09 ;   e?嘣  .H.H.?
00002780h: 4F 11 95 0E CF 15 1D 12 B6 18 00 15 BB 1A D1 17 ; O.??..?..??
00002790h: 9E 1B 26 19 88 1A 93 17 FB 17 45 14 75 14 9A 10 ; ?&.???E.u.?
000027a0h: 9B 0F 93 0C 0C 0A EC 07 A3 04 75 02 C2 FF 9F FC ; ??..??u.?熖
000027b0h: 46 FC F2 F7 4E FA 6A F5 BF F8 25 F4 CD F6 83 F2 ; F  鱔鷍蹩?敉鷍?
000027c0h: E0 F4 6D F0 E1 F3 65 EF 3A F4 16 F0 14 F6 99 F1 ; 圄m疳骸???徽?
```



7.2 Libjpeg 简介

对 avi 文件视频数据进行解码

7.2.1 MJPEG 简介



7.2.2 什么是 libjpeg

什么是libjpeg

libjpeg是一种完全用C语言编写的库，包含了被广泛使用的JPEG解码、JPEG编码和其他的JPEG功能的实现。

这个库由IJG组织（Independent JPEG Group 独立JPEG小组）提供并维护。

libjpeg目前最新版本为v9e，可以在http://www.ijg.org 这个网站下载到。

libjpeg具有稳定、兼容性强和解码速度较快等优点。

7.2.3 Libjpeg VS TJPGD

libjpeg VS TJPGD

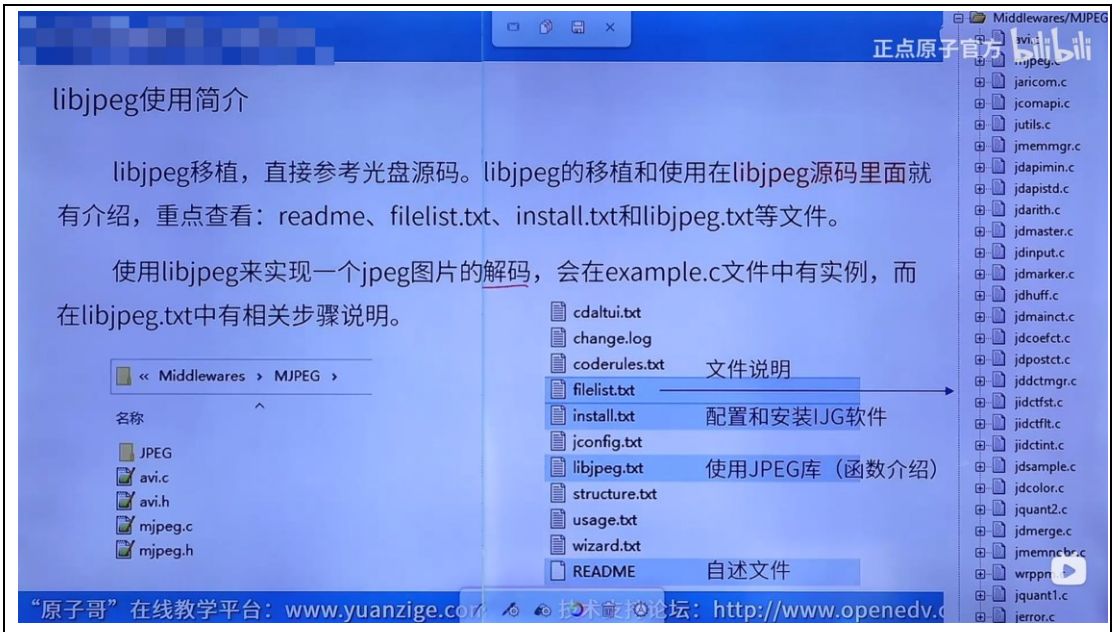
图片显示实验，使用TJPGD来做JPEG解码。TJPGD特点：占用资源少，解码速度慢。

开发板	TJPGD	libjpeg	硬件JPEG
STM32F407	54.3ms	22.1ms	没有
STM32H750	17.7ms	不用	5.3ms

在STM32F4上，同样一张240*160的JPG图片，用TJPGD来解码，得要50多ms，而用libjpeg只要20ms左右即可完成解码，在速度上明显比TJPGD快不少，使得解码视频成为可能。

实际上，经过优化后的libjpeg，使用STM32F4，在不超频的情况下，可以流畅播放480*272@10帧的MJPEG视频(带音频)。

7.2.4 Libjpeg 使用接简介



libjpeg解码JPEG时，会用到一个非常重要的结构体：struct jpeg_decompress_struct，定义成cinfo变量，该变量保存着jpeg数据的详细信息，也保存着解码之后输出数据的详细信息。

```
struct jpeg_decompress_struct {        /* 94个结构体成员 */
    jpeg_common_fields;
    struct jpeg_source_mgr * src;
    JDIMENSION image_width;
    JDIMENSION image_height;
    int num_components;
    J_COLOR_SPACE jpeg_color_space;
    ……
    struct jpeg_color_deconverter * cconvert;
    struct jpeg_color_quantizer * cquantize;
};
```

一般情况下，每次调用libjpeg库API时都需要把这个变量作为第一个参数传入。另外用户也可以通过修改该变量来修改libjpeg行为，比如输出数据格式，libjpeg库可用的最大内存等等。

7.2.5 Libpeg 解码步骤

libjpeg解码步骤（8个步骤）(对应到mjpeg.c的mjpegdec_decode函数)

① 分配，并初始化解码对象结构体(cinfo)

这里做了两件事：1，错误管理，2，初始化解码对象。

首先，错误管理使用setjmp和longjmp机制（不懂请百度）来实现类似C++的异常处理功能，外部代码可以调用longjmp来跳转到setjmp位置，执行错误管理（释放内存，关闭文件等）。这里注册了my_error_exit函数，来执行错误退出处理，另外，使用：my_emit_message函数，来输出警告信息，方便调试代码。

然后，通过调用jpeg_create_decompress函数实现初始化解码对象结构体：cinfo。

```
cinfo->err = jpeg_std_error(&jerr->pub);          /* 建立JPEG错误处理流程 */
jerr->pub.error_exit = my_error_exit;             /* 处理函数指向my_error_exit */
jerr->pub.emit_message = my_emit_message;         /* 输出警告信息 */

jpeg_create_decompress(cinfo);                    /* 初始化解码对象cinfo */
```

② 指定数据源

示例代码（example.c里面的JPEG解码参考代码read_JPEG_file，下同）用的是jpeg_stdio_src函数。本例程，我们用另外一个函数实现：

主要设置了cinfo->src各函数指针，用于获取外部数据。

```
/* 初始化jpeg解码数据源 */
static void jpeg_filerw_src_init(j_decompress_ptr cinfo)
{
  if (cinfo->src == NULL)   /* first time for this JPEG object? */
  {
    cinfo->src = (struct jpeg_source_mgr *) (*cinfo->mem->alloc_small)((j_common_ptr)
                cinfo, JPOOL_PERMANENT,sizeof(struct jpeg_source_mgr));
  }
  cinfo->src->init_source = init_source;
  cinfo->src->fill_input_buffer = fill_input_buffer;          /* 用于填充数据给libjpeg */
  cinfo->src->skip_input_data = skip_input_data;              /* 用于跳过一定字节的数据 */
  cinfo->src->resync_to_restart = jpeg_resync_to_restart;     /* use default method */
  cinfo->src->term_source = term_source;
  cinfo->src->bytes_in_buffer = 0;                            /* forces fill_input_buffer on first read */
  cinfo->src->next_input_byte = NULL;                         /* until buffer loaded */
```

③ 读取文件参数

通过jpeg_read_header函数实现，将读取JPEG的很多参数（图像宽和高、色彩空间、颜色通道数等），必须解码前调用。

④ 设置解码参数

示例代码没有做任何设置（使用默认值）。本章代码则做了设置，如下：

```
cinfo->dct_method = JDCT_IFAST;
cinfo->do_fancy_upsampling = 0;
```

设置使用快速整型DCT(离散余弦变换)，并设置do_fancy_upsampling的值为假(0)，以提高解码速度

⑤ 开始解码

示例代码里面先调用jpeg_start_decompress函数，然后计算样本输出buffer大小，并为其申请内存，为后续读取解码后的数据做准备。 根据cinfo->max_v_samp_factor选择使用哪个

不过为了提高速度，本例程就没做这些处理了，而是直接修改底层函数：

h2v1_merged_upsample和h2v2_merged_upsample（在jdmerge.c里面），将输出的RGB888数据转换成RGB565，送给LCD。为了正确的输出到LCD，我们在jpeg_start_decompress函数之后，加入如下代码：

```
lcd_set_window(imgoffx,imgoffy,cinfo->output_width,cinfo->output_height); /* 设置开窗大小 */
lcd_write_ram_prepare();                                                  /* 开始写入GRAM */
```

解码时，直接在h2v1_merged_upsample和h2v2_merged_upsample里面丢数据给LCD，实现jpeg解码输出到LCD。（步骤6会调用执行）

⑥ 循环读取数据

通过jpeg_read_scanlines函数，循环解码并读取jpeg图片数据，实现jpeg解码。示例代码通过put_scanline_someplace函数，输出到某个地方（如lcd，文件等），本例程则直接输出到LCD了，所以仅剩jpeg_read_scanlines函数，循环调用即可实现jpeg→LCD的操作。最后要重新恢复窗口。

⑦ 解码结束

解码完成后，通过jpeg_finish_decompress函数，结束jpeg解码。

⑧ 释放解码对象资源

在所有操作完成后，通过jpeg_destroy_decompress，释放解码过程中用到的资源（比如释放内存）。

经过这8步处理，即可完成一帧jpeg图片的解码。详细的代码请参考本例程源码：mjpeg.c。libjpeg的使用，就介绍到这里。

```
jpeg_read_scanlines      /* 解码一行数据 */

(*cinfo->main->process_data) (cinfo, scanlines, &row_ctr, max_lines);      /* 处理数据 */
```

process_data_simple_main

```
(*cinfo->post->post_process_data) (cinfo, mainp->buffer,   /* 做上采样和颜色转换 */
                                   &mainp->rowgroup_ctr, rowgroups_avail,
                                   output_buf, out_row_ctr, out_rows_avail);
```

Merged_2v_upsample

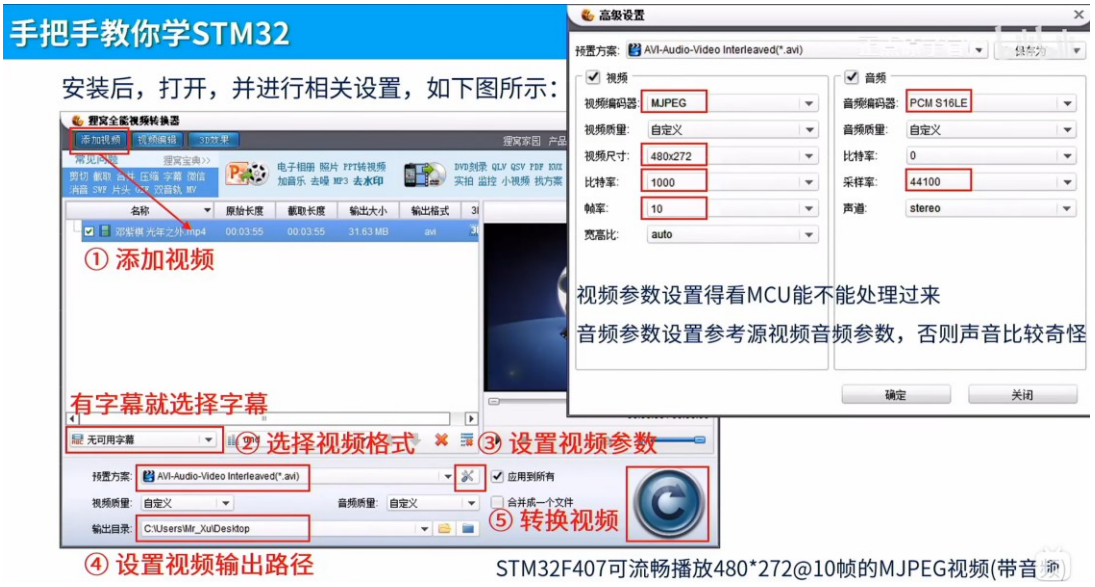
```
(*upsample->upmethod) (cinfo, input_buf, *in_row_group_ctr, work_ptrs);
```

h2v2_merged_upsample

7.3 视频转换软件使用

本例程所设计的视频播放器，仅支持MJPEG编码的AVI格式视频，且音频必须为PCM格式，另外视频分辨率必须≤屏幕分辨率。

很难找到满足这些要求的avi视频，所以我们需要用软件，将通用视频（任何视频都可以）转换为我们需要的格式，这里我们通过：狸窝全能视频转换器，这款软件来实现（光盘→软件资料→软件→其它软件→视频转换软件→狸窝全能视频转换器.exe）。



STM32F407可流畅播放480*272@10帧的MJPEG视频(带音

音频编码器：PCM S16LE

采样率：设置44100，即44.1KHz。（采样率越高，声音质量越好，转换文件变大，视频可能变卡）

视频编码器：选择MJPEG。本例程仅支持MJPEG视频解码，所以选择这个编码器。

视频尺寸：480x272。根据LCD分辨率来选择。

比特率：1000。设置越大，视频质量越好，解码越慢(可能会卡)。设置1000，综合得到较好的效果。

帧率：10。每秒钟10帧。若想提高帧率：1、降低分辨率 2、降低比特率 3、降低音频采样率

视频转换中：

转换结束后，将转好的视频(.avi文件)拷贝到SD卡：VIDEO文件夹内，然后将SD卡插入开发板的SD卡接口，即可完成视频播放实验的准备工作。

7.4 AVI 视频播放步骤

1，初始化各外设

要解码视频，相关外设肯定要先初始化好
(SDIO、I2S、DMA、ES8388、LCD和按键等)

2，读取AVI文件并解析

要解码，先读取avi文件，首先读出音视频关键信息。
(音频参数、视频参数、数据流起始地址等)

3，根据解析结果，设置相关参数

根据第2步解析结果，设置I2S的音频采样率和位数，设置视频显示在LCD中间区域，根据图片尺寸，设置LCD开窗时x,y方向的偏移量。

4，读取数据流，开始解码

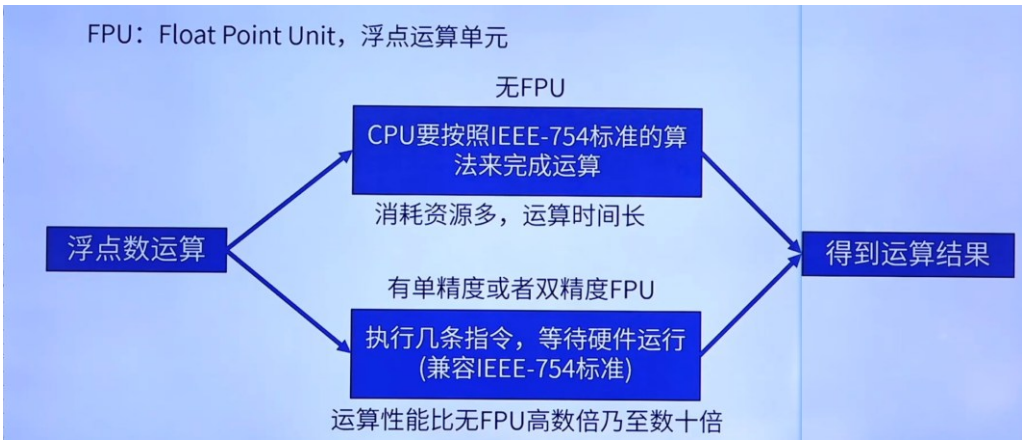
前3步完成，就可以正式开始播放视频。
根据音视频流数据(movi块)，根据类型码执行音频/视频解码。
音频数据直接填充到DMA缓冲区，视频数据使用libjpeg解码

5，解码完成，释放资源

文件读取完后(或者出错了)，需要释放申请的内存，恢复LCD窗口、关闭定时器、停止I2S播放音乐和关闭文件等一系列操作。

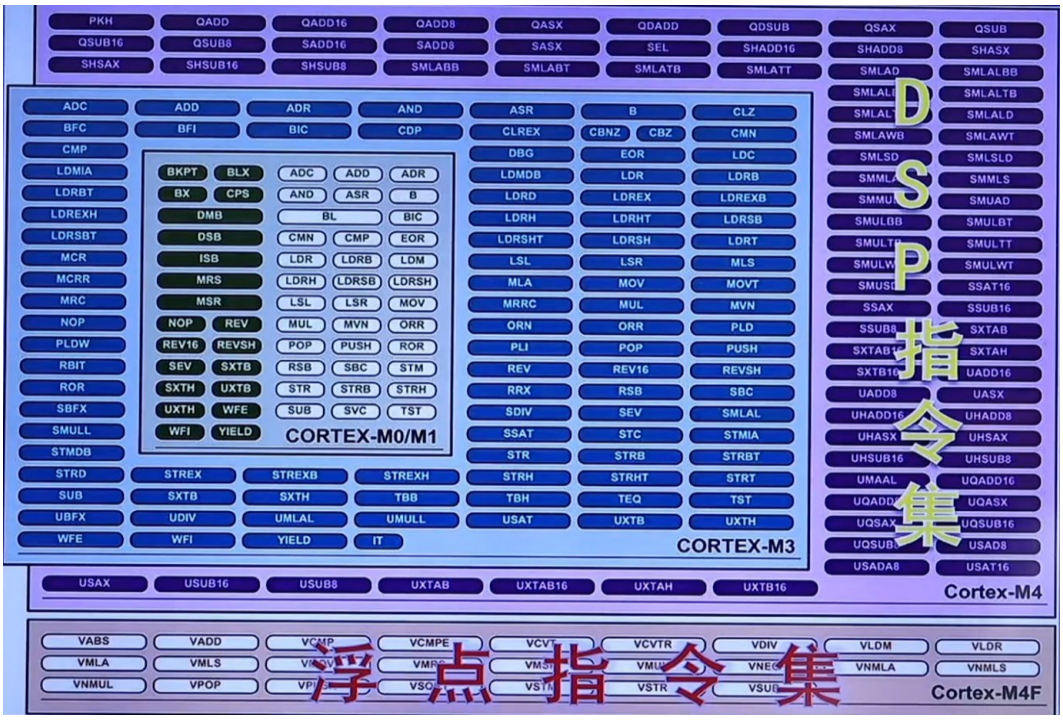
8 FPU

8.1 FPU 简介



F1/F4/F7/H7的FPU情况对比

对比项	F1	F4	F7	H7
有无FPU	无	有	有	有
是否支持单精度FPU	不支持	支持	支持	支持
是否支持双精度FPU	不支持	不支持	支持	支持

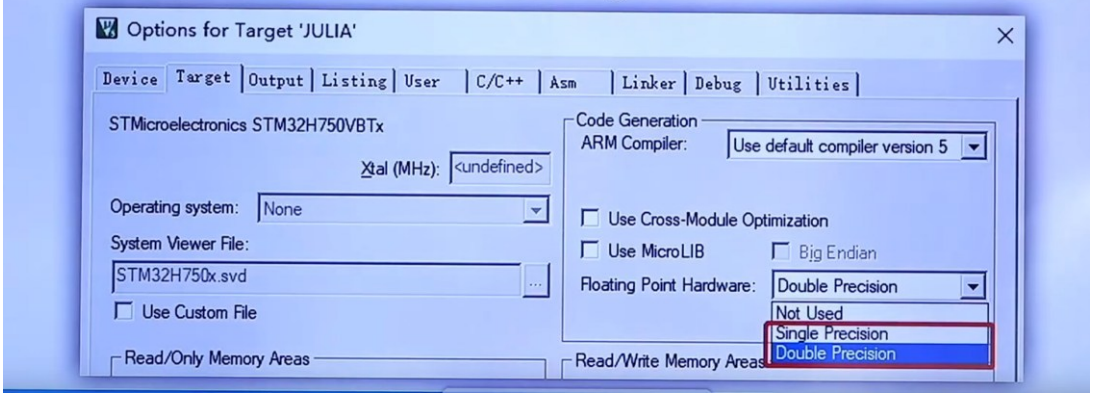


8.2 FPU 开启方法

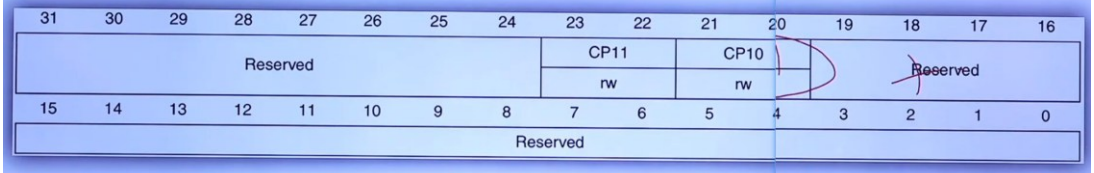
开启硬件FPU的两个要点：

1，设置CPACR寄存器bit20~23为1，使能硬件FPU

2，Floating Point Hardware设置为：Single Precision或Double Precision



协处理器控制寄存器(SCB->CPACR)



复位后，CP10和CP11 四个位默认为0，此时禁止访问协处理器（禁止硬件FPU)。

将这四个位设置为1，即可完全访问协处理器，从而可以开启硬件FPU功能

8.3 HAL 库的 DPU 相关配置

```
void SystemInit(void)
{
    /* FPU settings ------------------------------------------------------------*/
    #if (__FPU_PRESENT == 1) && (__FPU_USED == 1)
    SCB->CPACR |= ((3UL << 10*2)|(3UL << 11*2));  /* 设置FPU完全访问权限 */
    #endif
    ......
}
```

```
#define  __FPU_PRESENT    1        /* 在STM32F/Hxxxxx.h中被定义 */
```

9 DSP

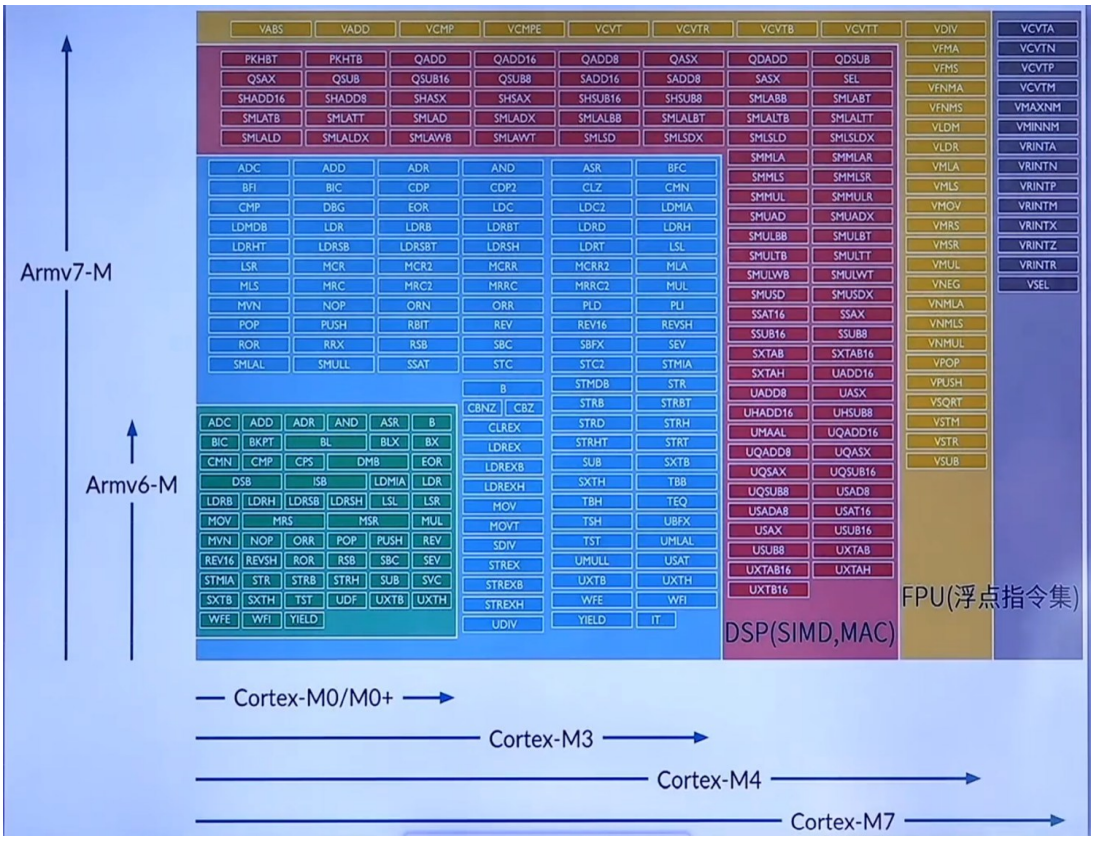
9.1 DSP 功能简介

DSP：Digital Signal Processing，数字信号处理

M3内核没有硬件FPU，也没有DSP指令集

M4和M7等内核自带FPU和DSP单元，并集成了专用的DSP指令集（如：单周期乘加指令(MAC)、优化的单指令多数据流指令(SIMD)、饱和算术运算等多种数字信号处理指令）

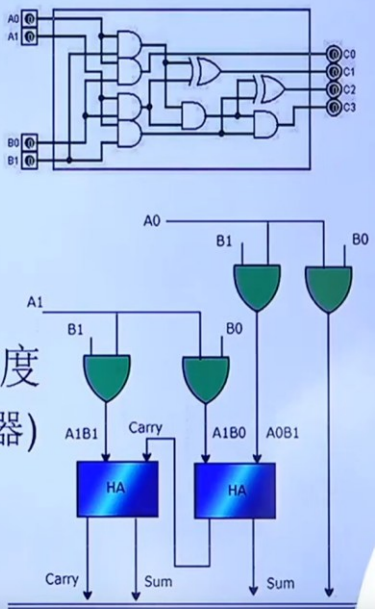
M4/M7执行的DSP指令可以在单周期内完成，M3则需要多个指令和多个周期才能完成



9.2 显著增强的 DSP 指令

- 单周期的乘加指令(MAC)
  - 乘法器可以在单周期内完成相乘或乘加指令
    - 有符号和无符号的乘法
    - 有符号和无符号的乘加
    - 有符号和无符号的长数据乘加(64位)
- 单周期的SIMD指令
  - 同时有多个数据参与运算
    - 例如：16x16 + 16x16 + 32
- 相对Cortex-M3指令显著提高了速度
  - 16位乘加指令提高4倍(双16位乘加器)
  - 32位乘加指令提高2倍
  - 32位乘加指令提高至7倍

①

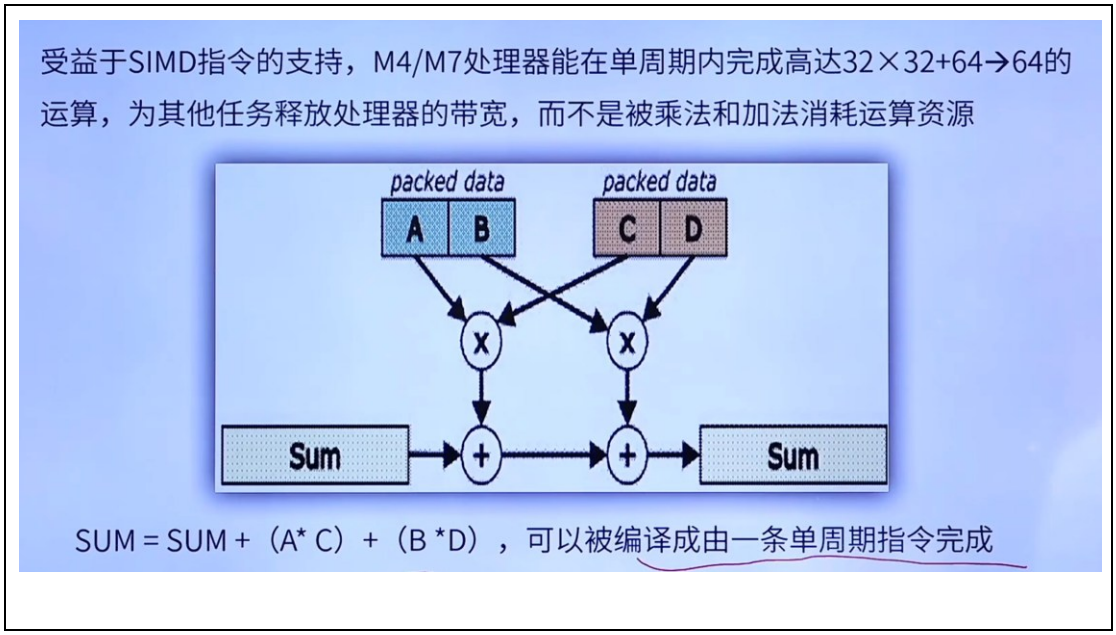


单周期的乘加运算

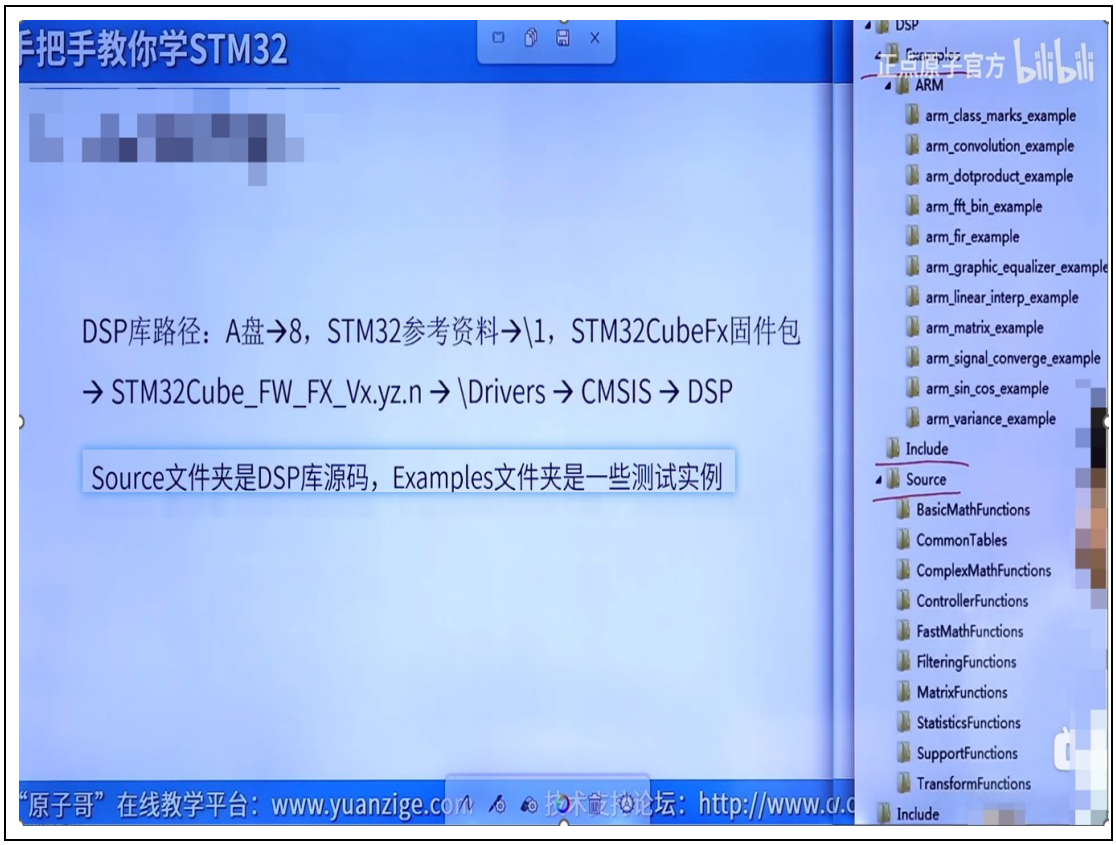
运算	指令	CM3	CM4
16 x 16 = 32	SMULBB, SMULBT, SMULTB, SMULTT	n/a	1
16 x 16 + 32 = 32	SMLABB, SMLABT, SMLATB, SMLATT	n/a	1
16 x 16 + 64 = 64	SMLALBB, SMLALBT, SMLALTB, SMLALTT	n/a	1
16 x 32 = 32	SMULWB, SMULWT	n/a	1
(16 x 32) + 32 = 32	SMLAWB, SMLAWT	n/a	1
(16 x 16) ± (16 x 16) = 32	SMUAD, SMUADX, SMUSD, SMUSDX	n/a	1
(16 x 16) ± (16 x 16) + 32 = 32	SMLAD, SMLADX, SMLSD, SMLSDX	n/a	1
(16 x 16) ± (16 x 16) + 64 = 64	SMLALD, SMLALDX, SMLSLD, SMLSLDX	n/a	1
32 x 32 = 32	MUL	1	1
32 ± (32 x 32) = 32	MLA, MLS	2	1
32 x 32 = 64	SMULL, UMULL	5-7	1
(32 x 32) + 64 = 64	SMLAL, UMLAL	5-7	1
(32 x 32) + 32 + 32 = 64	UMAAL	n/a	1
32 ± (32 x 32) = 32 (高位)	SMMLA, SMMLAR, SMMLS, SMMLSR	n/a	1
(32 x 32) = 32 (高位)	SMMUL, SMMULR	n/a	1

指令周期数

M


受益于SIMD指令的支持，M4/M7处理器能在单周期内完成高达32×32+64→64的运算，为其他任务释放处理器的带宽，而不是被乘法和加法消耗运算资源
packed data
packed data
A
B
C
D
X
X
Sum
+
+
Sum
SUM = SUM + （A* C）+（B *D），可以被编译成由一条单周期指令完成

9.3 DSP 源码库简介


手把手教你学STM32
DSP库路径：A盘→8，STM32参考资料→\1，STM32CubeFx固件包
→ STM32Cube_FW_FX_Vx.yz.n → \Drivers → CMSIS → DSP
Source文件夹是DSP库源码，Examples文件夹是一些测试实例
“原子哥”在线教学平台：www.yuanzige.com
DSP
Examples
ARM
arm_class_marks_example
arm_convolution_example
arm_dotproduct_example
arm_fft_bin_example
arm_fir_example
arm_graphic_equalizer_example
arm_linear_interp_example
arm_matrix_example
arm_signal_converge_example
arm_sin_cos_example
arm_variance_example
Include
Source
BasicMathFunctions
CommonTables
ComplexMathFunctions
ControllerFunctions
FastMathFunctions
FilteringFunctions
MatrixFunctions
StatisticsFunctions
SupportFunctions
TransformFunctions
Include

9.4 DSP 库介绍

文件夹	描述
BasicMathFunctions	基本数学函数，提供浮点数的各种基本运算函数，如向量加减乘除等运算
CommonTables	arm_common_tables.c文件提供位翻转或相关参数表
ComplexMathFunctions	复杂数学功能，如向量处理，求模运算
ControllerFunctions	控制功能函数，包括正弦余弦，PID电机控制，矢量Clarke变换，矢量Clarke逆变换等
FastMathFunctions	快速数学功能函数，提供了一种快速的近似正弦，余弦和平方根等相比CMSIS计算库要快的数学函数
FilteringFunctions	滤波函数功能，主要为FIR和LMS（最小均方根）等滤波函数
MatrixFunctions	矩阵处理函数，包括矩阵加法、矩阵初始化、矩阵反、矩阵乘法、矩阵规模、矩阵减法、矩阵转置等函数
StatisticsFunctions	统计功能函数，如求平均值、最大值、最小值、计算均方根RMS、计算方差/标准差等
SupportFunctions	支持功能函数，如数据拷贝，Q格式和浮点格式相互转换，Q任意格式相互转换
TransformFunctions	变换功能，包括复数FFT（CFFT）/复数FFT逆运算（CIFFT）、实数FFT（RFFT）/实数FFT逆运算（RIFFT）、和DCT（离散余弦变换）和配套的初始化函数

9.5 DSP_Lib 库

DSP_Lib库 Lib库路径：A盘→8，STM32参考资料→\1，STM32CubeFx固件包→

STM32Cube_FW_FX_Vx.yz.n → \Drivers → CMSIS → Lib → ARM

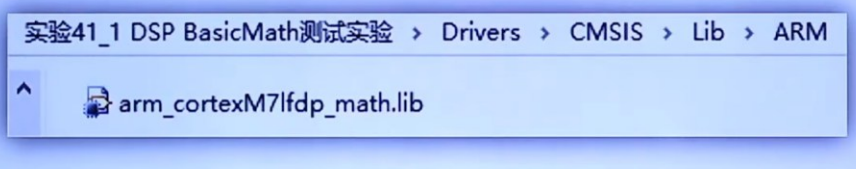
① arm_cortexM3b_math.lib (Cortex-M3大端模式)
② arm_cortexM3l_math.lib (Cortex-M3小端模式)

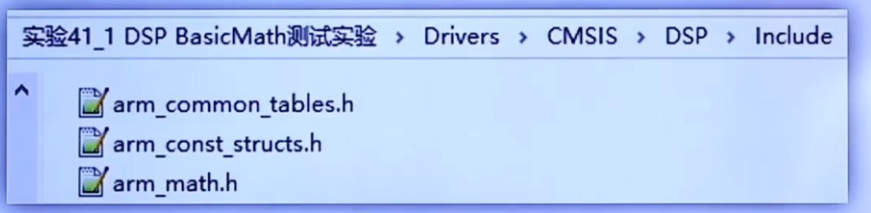
① arm_cortexM4b_math.lib (Cortex-M4大端模式)
② arm_cortexM4l_math.lib (Cortex-M4小端模式)
③ arm_cortexM4bf_math.lib (浮点Cortex-M4大端模式)
④ arm_cortexM4lf_math.lib (浮点Cortex-M4小端模式)

① arm_cortexM7b_math.lib (Cortex-M7大端模式)
② arm_cortexM7l_math.lib (Cortex-M7小端模式)
③ arm_cortexM7bfdp_math.lib (双精度浮点Cortex-M7大端模式)
④ arm_cortexM7lfdp_math.lib (双精度浮点Cortex-M7小端模式)
⑤ arm_cortexM7bfsp_math.lib (单精度浮点Cortex-M7大端模式)
⑥ arm_cortexM7lfsp_math.lib (单精度浮点Cortex-M7小端模式)

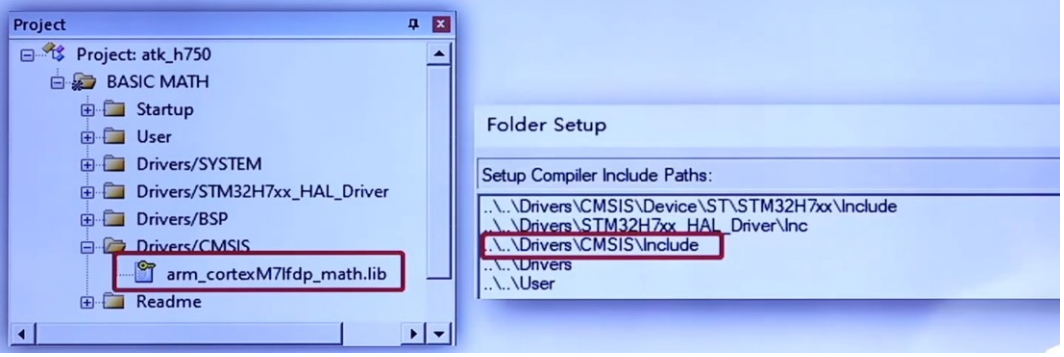
9.6 搭建 DSP 库运行环境

① 添加文件





② 添加DSP_Lib库到工程，添加头文件包含路径



③ 添加全局宏定义

为了使用DSP库的所有功能，我们需要添加几个全局宏定义：

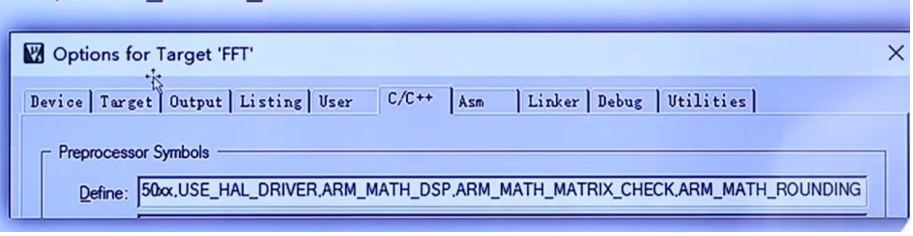
1，__FPU_USED

2，__FPU_PRESENT

3，ARM_MATH_DSP

4，ARM_MATH_MATRIX_CHECK

5，ARM_MATH_ROUNDING



10 手写识别

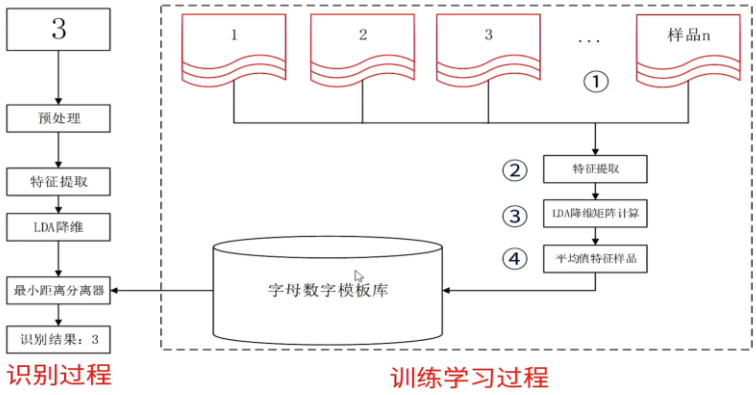
10.1 手写识别简介

手写识别，是指对在手写设备上书写时产生的有序轨迹信息进行识别的过程

手写识别是用户最自然、最方便的文字输入方式，易学易用，可取代键盘或者鼠标

手写输入的设备有：触摸屏、电磁感应手写板、压感式手写板、触控板、超声波笔等

10.2 正点原子手写识别库简介



①采集大量数据样本，样本类别有：0~9，a~z，A~Z总共62类，每个类别5~10个样本不等（样本越多识别率就越高）
②对这些样本进行传统的把方向特征提取，提取后特征维数为512维
③按照LDA线性判决分析的方法进行降维，将维度降到64维
④然后针对各个样本类别进行平均计算得到该类别的样本模板

LDA知识链接

10.3 正点原子手写识别库文件

正点原子手写识别库支持数字、小写字母、大写字母识别

文件	描述
ATKNCR_M_V2.0.lib (二选一)	使用内存管理，需要实现alientek_ncr_malloc和alientek_ncr_free两个函数
ATKNCR_N_V2.0.lib(二选一)	不需要使用内存管理，通过全局变量来定义缓存区，缓存区需要提供至少3K左右的RAM
atk_ncr.c	实现内存设置函数、分配内存函数、释放内存函数
atk_ncr.h	声明初始化识别器函数、识别器识别函数、停止识别器函数等

正点原子手写识别库资源需求：FLASH：52KB左右，RAM：6KB左右

10.4 手写识别步骤

①调用alientek_ncr_init函数，初始化识别程序

该函数用来初始化识别器，在手写识别进行之前，必须调用该函数

② 获取输入的点阵数据

通过触摸屏获取输入轨迹点阵坐标，并存放到缓存区里，注意至少输入2个不同坐标的点阵数据，才能正常识别。输入点数越多，需要内存越大，推荐输入点数范围：100~200点

③ 调用alientek_ncr函数,得到识别结果

通过调用alientek_ncr函数，得到输入点阵的识别结果，结果将保存在result参数里面，采用ASCII码格式存储

④调用alientek_ncr_stop函数，终止识别

调用alientek_ncr_stop函数，可停止识别。如果还需要继续识别，重复步骤②和步骤③即可

11 T9 拼音输入法

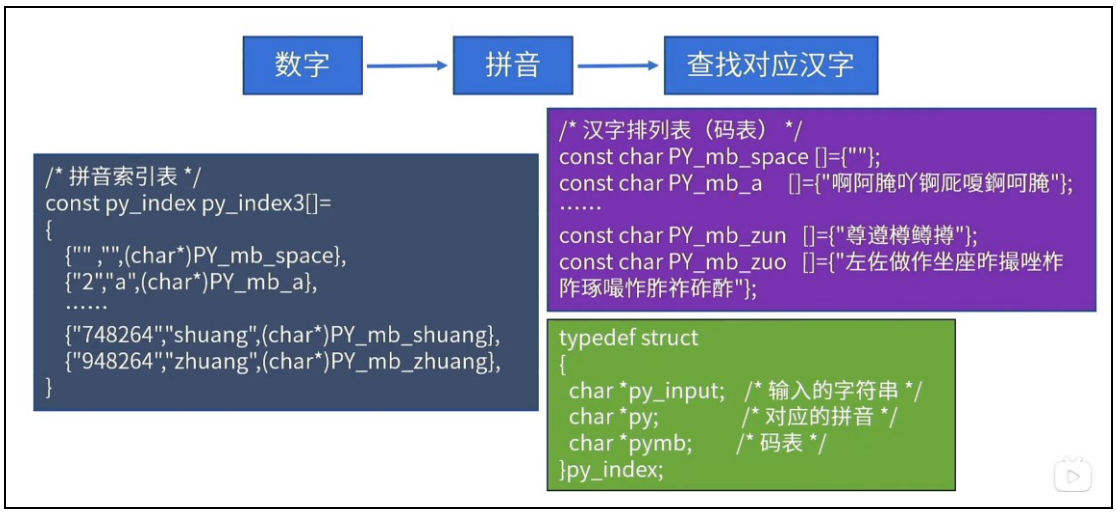
11.1 T9 拼音输入法简介

T9输入法是由美国特捷通讯（Tegic Communications）软件公司开发的，该输入法解决了小型掌上设备的文字输入问题，已经成为全球手机文字输入的标准之一

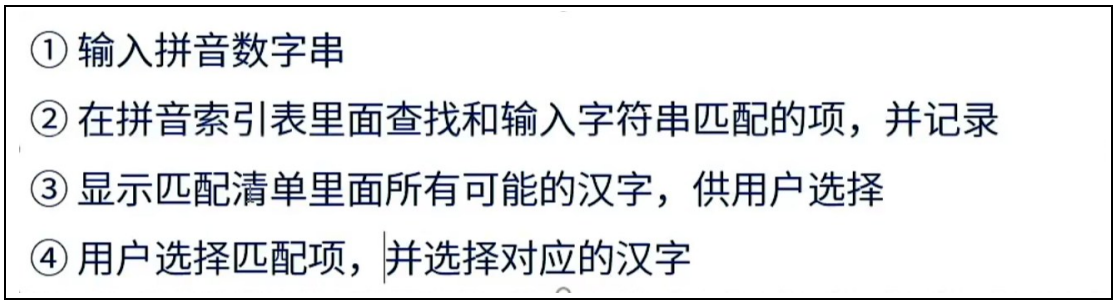
1 ;	2 abc	3 def
4 ghi	5 jkl	6 mno
7 pqrs	8 tuv	9 wxyz

输入“中”字，只需要输入：9、4、6、6、4，即可实现输入“中”字，在选择中字之后，T9会联想出一系列同中字组合的次，如文、国、断、山等。这样输入“国”字，我们直接选择即可，所以输入“国”字按键0次，这样T9总共只需要5次按键

11.2 T9 拼音输入法实现原理简介

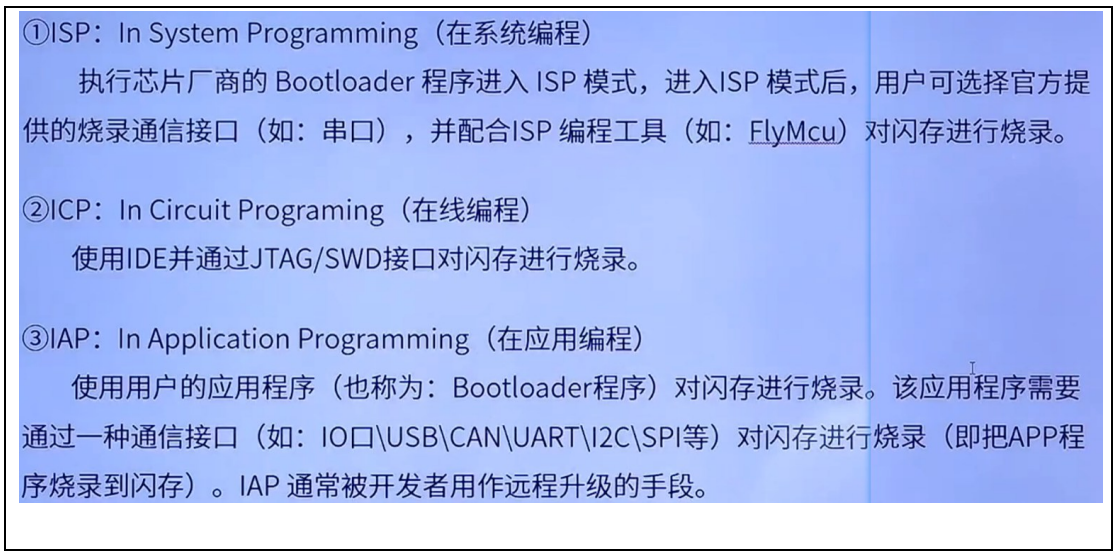


11.3 T9 拼音输入步骤

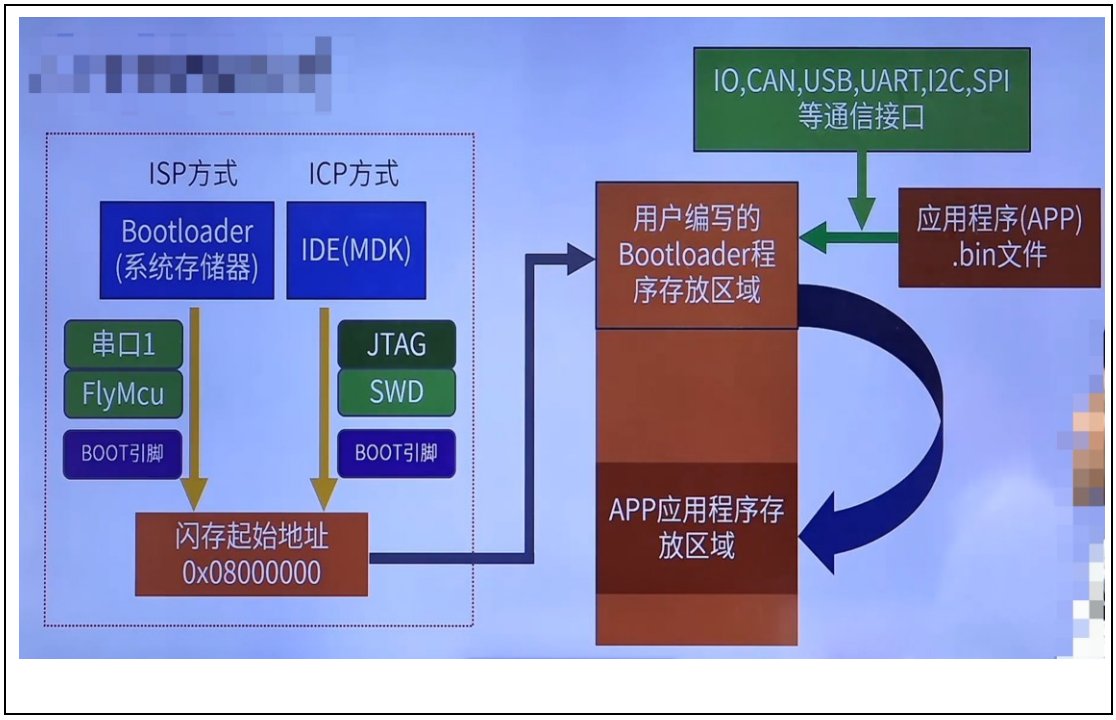


12 IAP

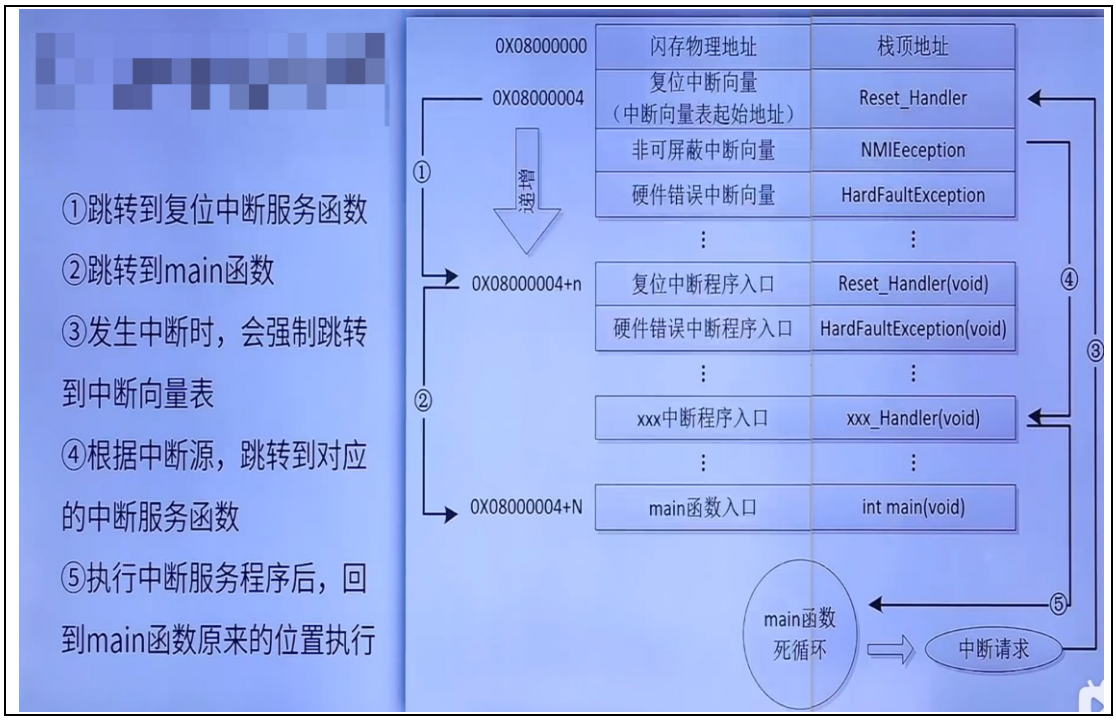
12.1 STM32 三种烧录方式



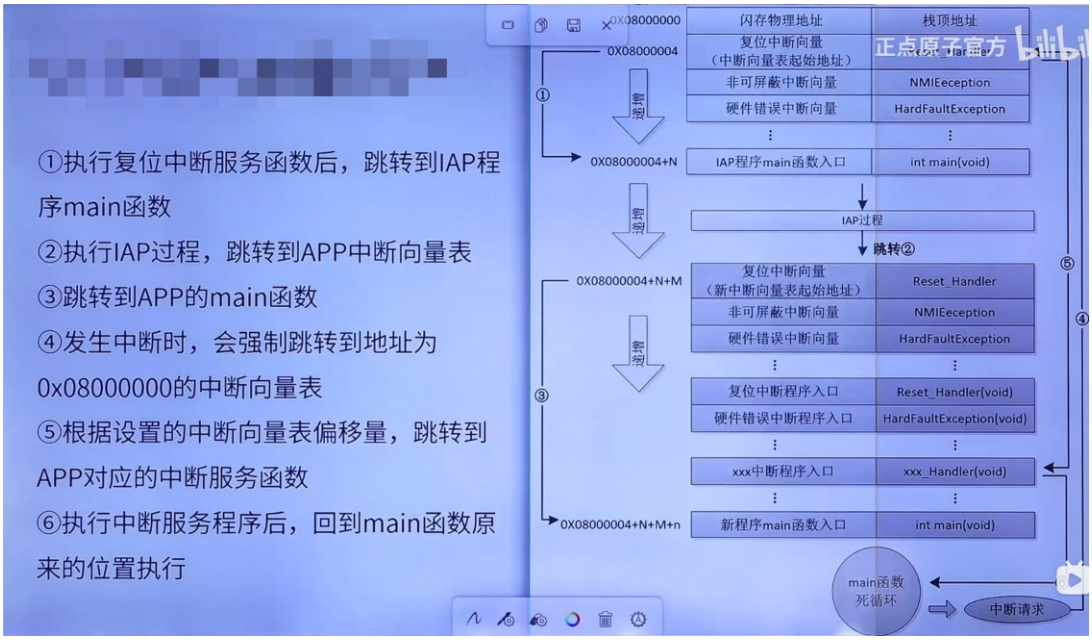
12.2 IAP 应用原理介绍



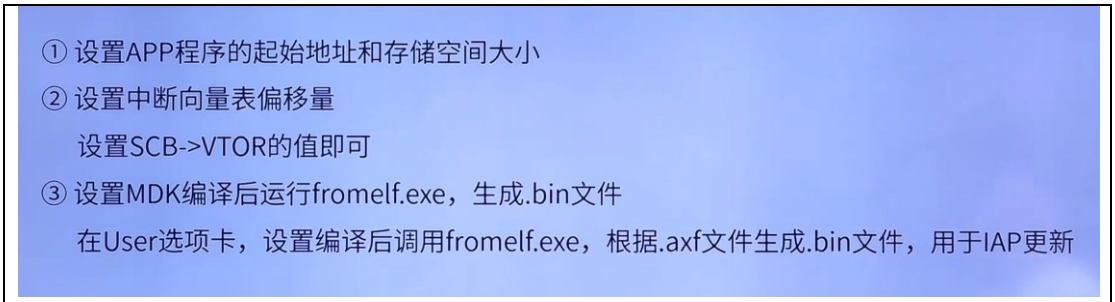
12.3 程序正常运行过程

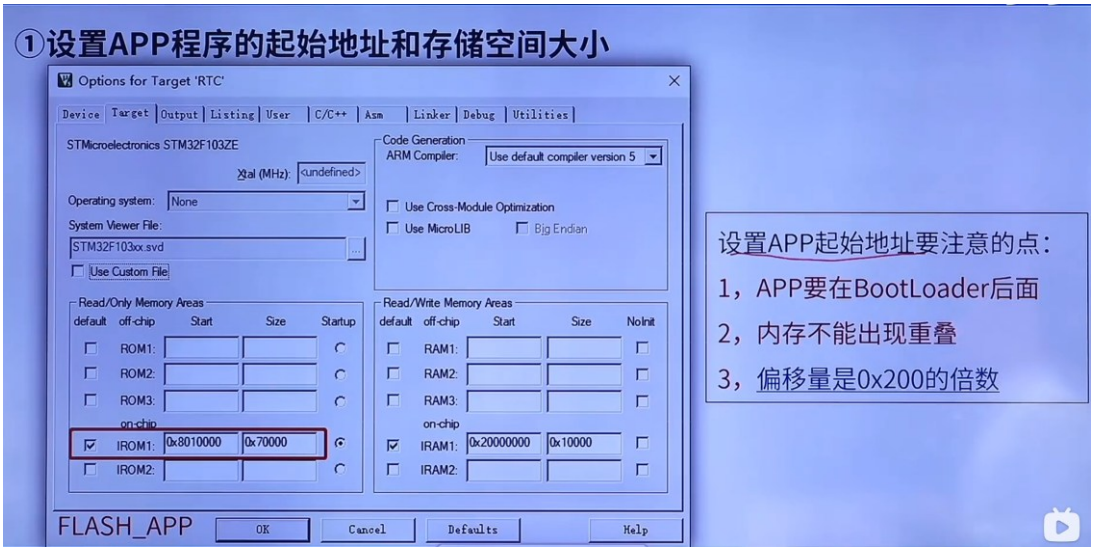


12.4 加入 IAP 后程序运行过程

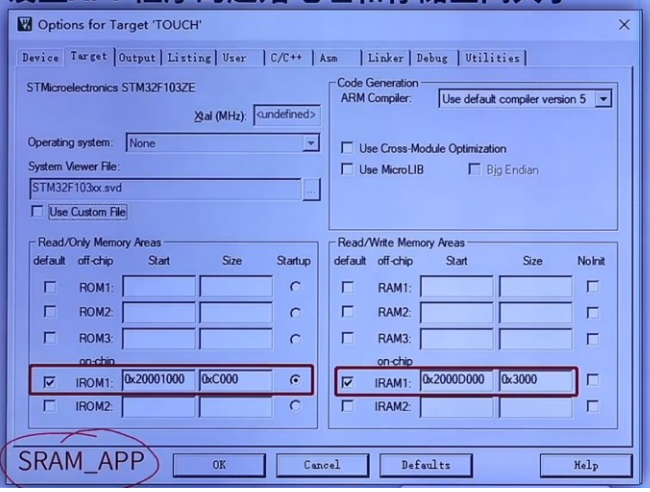


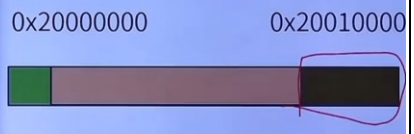
12.5 APP 程序生成步骤





①设置APP程序的起始地址和存储空间大小





Bootloader程序运行预留4KB SRAM

存放APP程序预留48KB SRAM

APP程序运行预留12KB SRAM

② 设置中断向量表偏移量

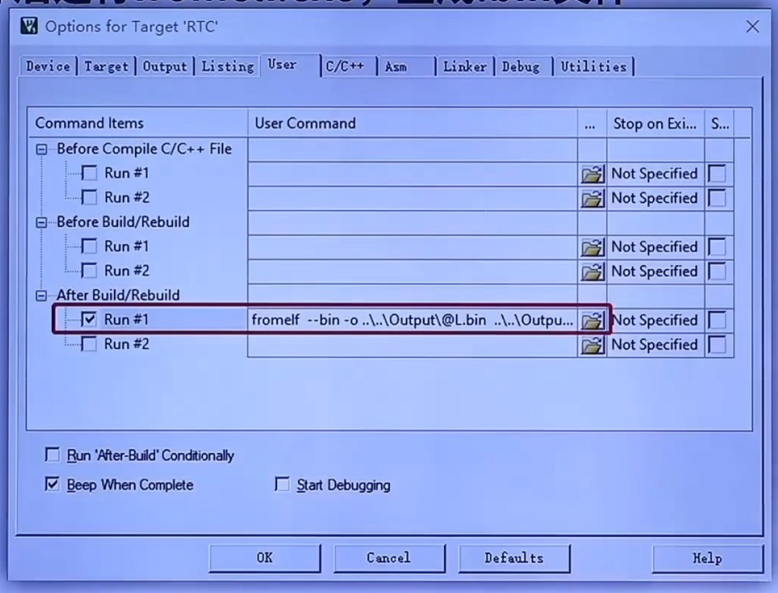
APP存放在FLASH中的设置方法

```
SCB->VTOR = FLASH_BASE | 0x10000;
```

APP存放在SRAM中的设置方法

```
SCB->VTOR = SRAM_BASE | 0x1000;
```

③ 设置MDK编译后运行fromelf.exe，生成.bin文件



D:\MDK5.36\ARM\ARMCC\bin\fromelf --bin -o ..\..\Output\@L.bin ..\..\Output\%L

13 USB

13.1 USB 简介

13.1.1 USB 是什么

USB， 英文全称：Universal Serial Bus，即通用串行总线

USB提供适合各种应用的传输协议，而且协议标准向下兼容

标准	推出时间	最大传输速度	电缆长度	速度模式
USB 1.0	1996	1.5Mbps	3m	低速模式
USB 1.1	1998	12Mbps	3m	全速模式
USB 2.0	2000	480Mbps	5m	高速模式
USB 3.2 Gen 1	2008	5Gbps	3m	超高速模式
USB 3.2 Gen 2	2013	10Gbps	3m	
USB 3.2 Gen 2x2	2017	20Gbps	3m	
USB 4	2019	40Gbps+	0.8m	

USB的优缺点

USB的优点	USB的缺点
1，支持热插拔 2，差分信号传输，抗干扰能力较强 3，USB2.0总线可提供5V/500mA的电源 4，传输速率快 5，易于扩展：可通过Hub连接多个设备 6，应用广泛：鼠标键盘、游戏手柄、打印机、移动硬盘、数码相机、U盘、USB声卡、USB扫码仪、USB摄像头、USB网卡、USB口的仪表仪器...	1，供电能力较差 2，传输距离较短 3，协议比较复杂

13.1.2 USB2.0 拓扑结构

USB是一种主从结构的系统，数据交换只能发生在主从设备之间，且只能由主机主动发起

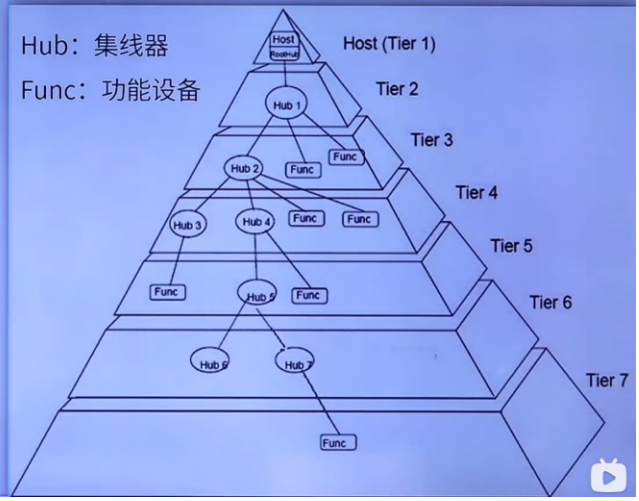
USB主机具有一个或者多个USB主控制器(Host controller)和根集线器(Root Hub)
USB主控制器：主要负责数据处理
根集线器：提供一个连接主控制器与设备之间的接口和通路
USB从机可以是各种USB设备也可以是USB集线器

USB集线器(USB Hub)：用于对原有的USB接口数量进行扩展，但不能扩展更多的带宽

USB OTG作为USB协议的补充版本，允许同一设备，在不同场合下，在主机和从机之间切换

USB2.0拓扑结构图

USB2.0协议中规定最多扩展6层
总线上最多可连接127个设备
每个USB设备具有一个7bit的地址
每层Hub最大允许串联5个设备
线缆长度最长5米



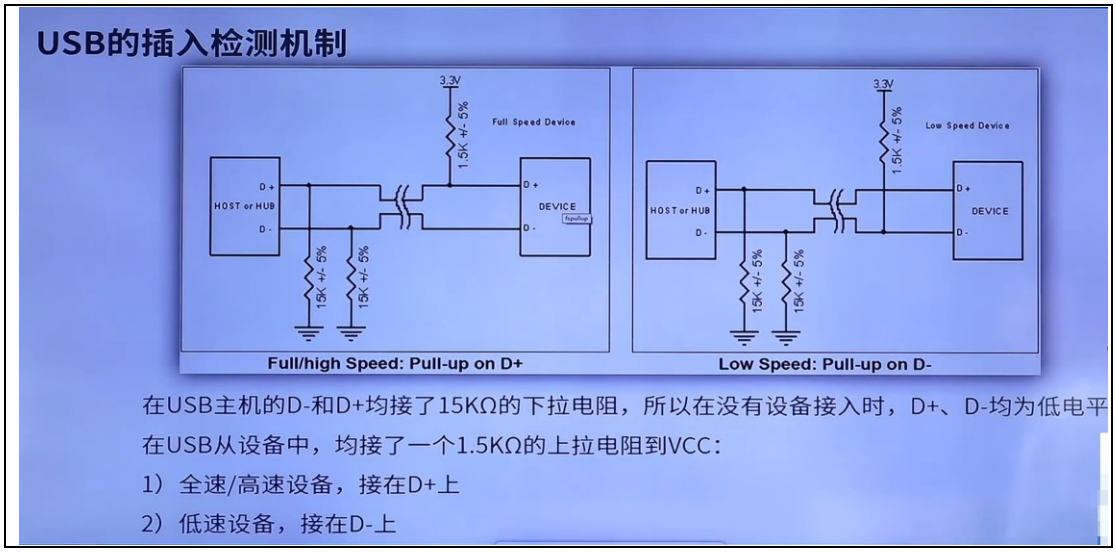
13.1.3 USB2.0 电气特性

标准的USB由四根线组成：
5V的电源线（VBUS）
差分数据线正（D+）
差分数据线负（D-）
地线（GND）

在USB OTG中，比标准的USB多了一根身份识别（ID）线，用来表明是主机还是从机

USB2.0支持3种传输速度：低速模式(1.5Mb/s)、全速模式(12Mb/s)、高速模式(480Mb/s)

USB实际的数据速率比上述的低一些，因为有很多协议开销，例如：同步、令牌、校验、位填充



13.1.4 USB 学习资料

正点原子开发板光盘资料：A 盘→8， STM32 参考资料→2， STM32 USB学习资料

想专门学习USB通信的朋友，这里推荐一本书给大家，**《圈圈教你玩USB》**

要驱动STM32的USB，就得编写USB驱动，USB驱动的编写非常复杂，所以我们直接用ST给我提供的USB驱动库来开发，后面的课程将带大家来移植ST官方开发板的USB例程

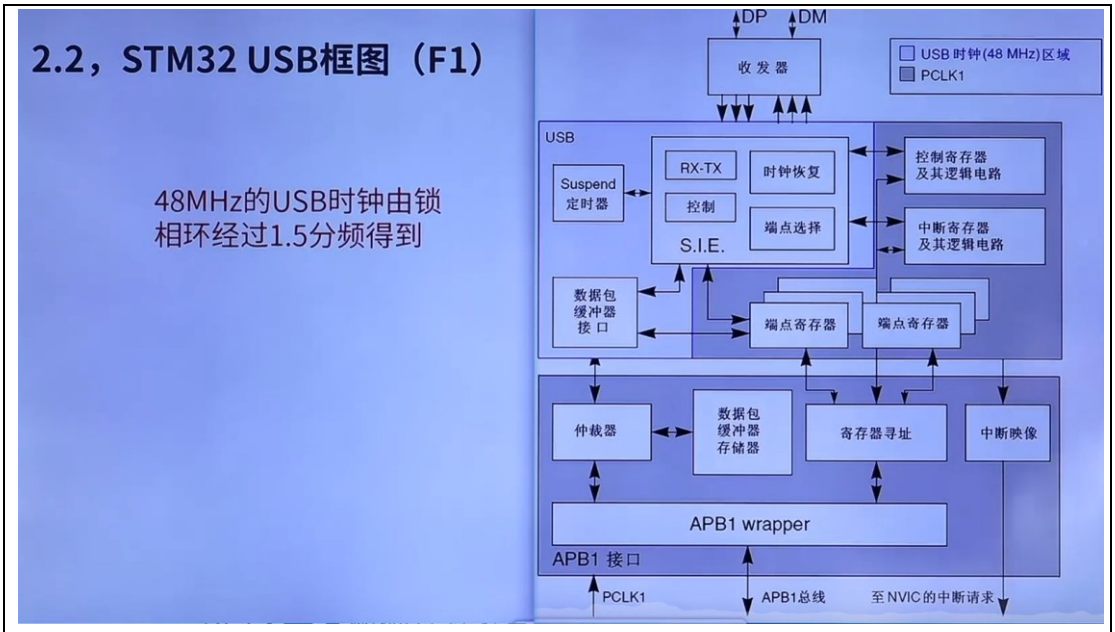
13.2 STM32 的 USB 介绍

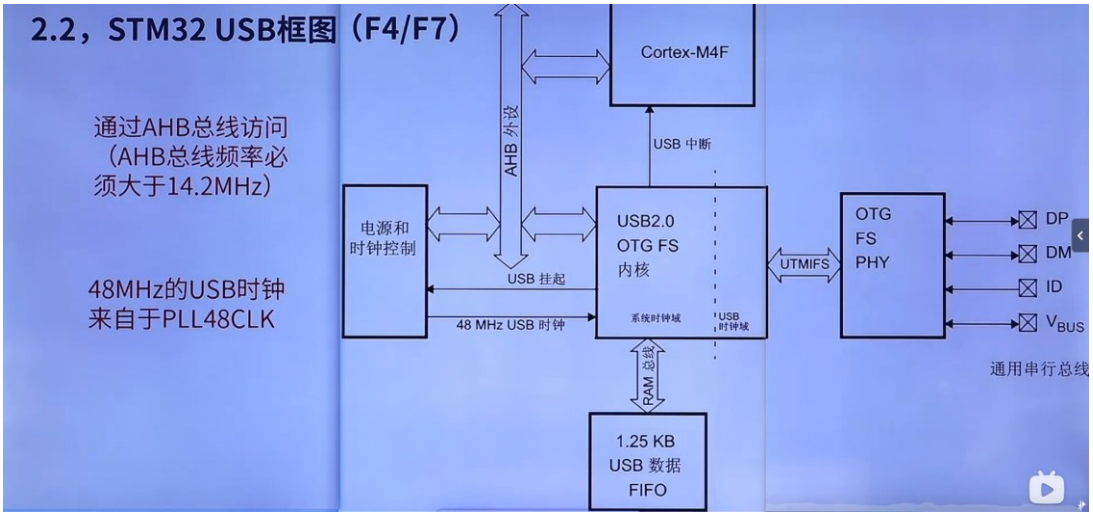
13.2.1 STM32 的 USB 特性

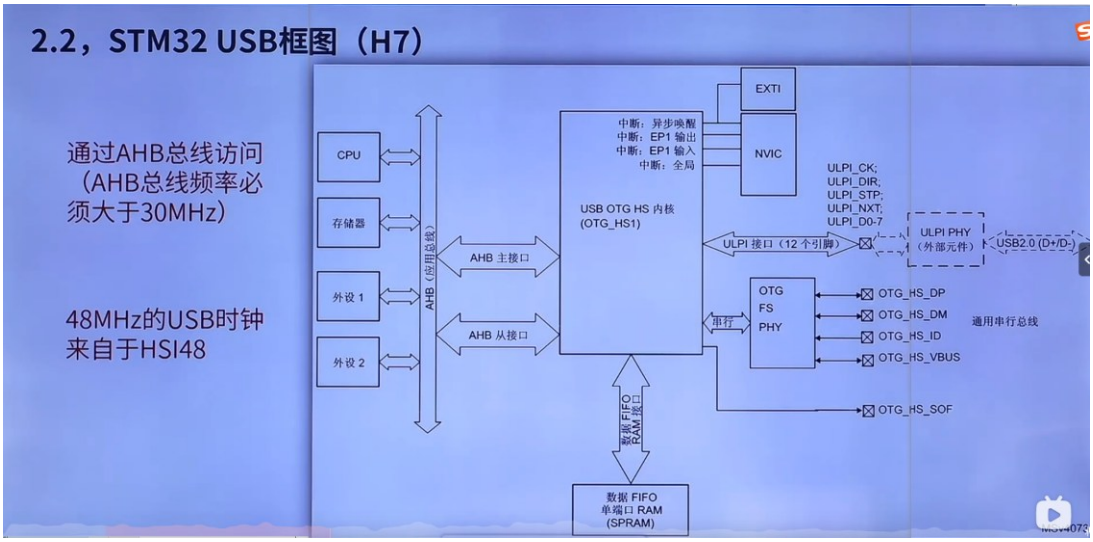
对比项	F103	F4	F7	H7
USB2.0规范	支持			
全速(FS,12Mbps)	支持			
高速(HS,480Mbps)	不支持	支持（需外扩高速PHY芯片实现）		
OTG	不支持	支持		
主/从机模式	不支持	支持		
USB通信时钟	48MHz			
其它	USB和CAN共享一个512字节的SRAM，所以不能同时使用	D+和D-内置1.5K上拉电阻，可以配置，即支持软断开功能		

STM32的USB的详细特性请参考对应系列的参考手册

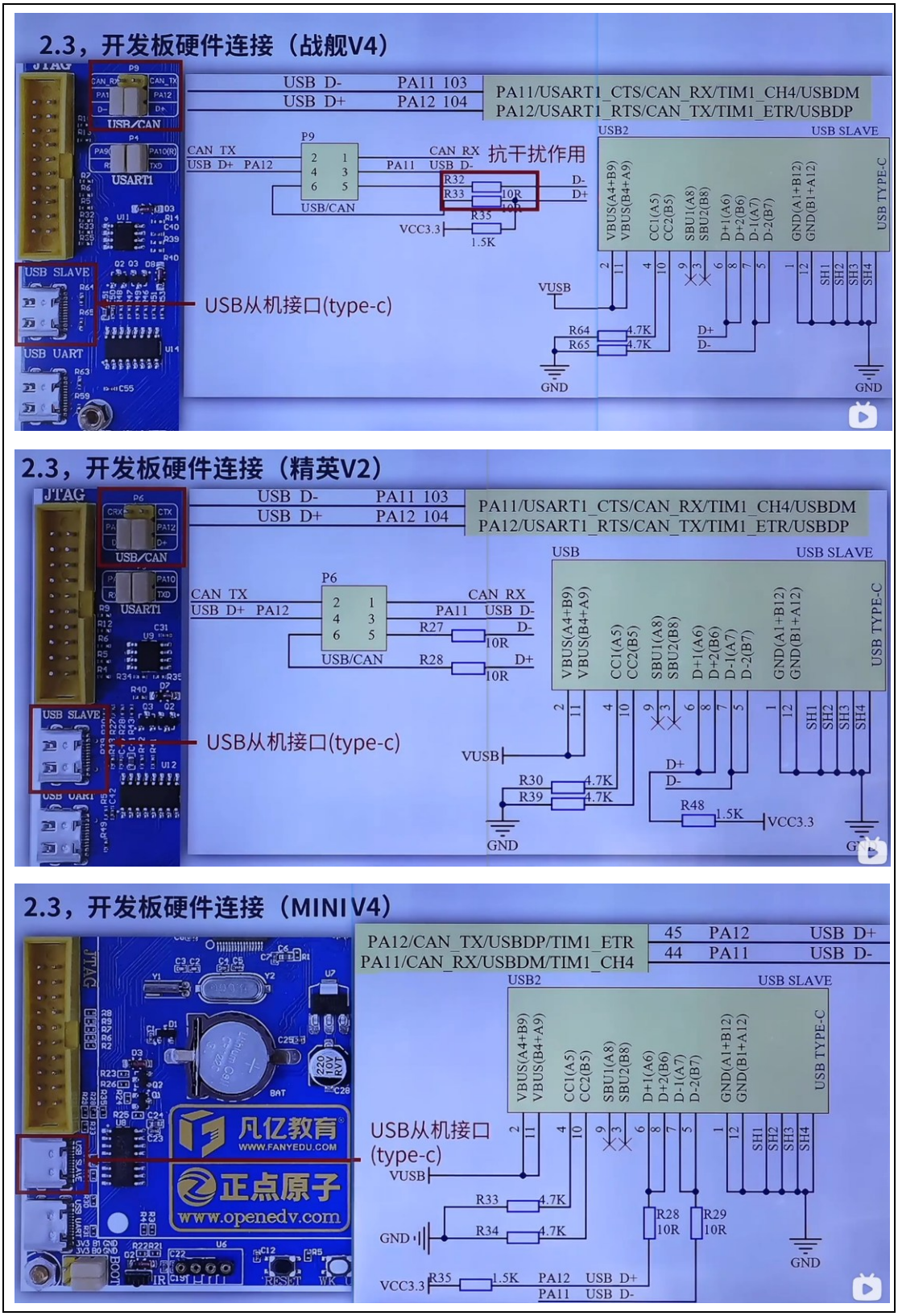
13.2.2 STM32 USB 框图

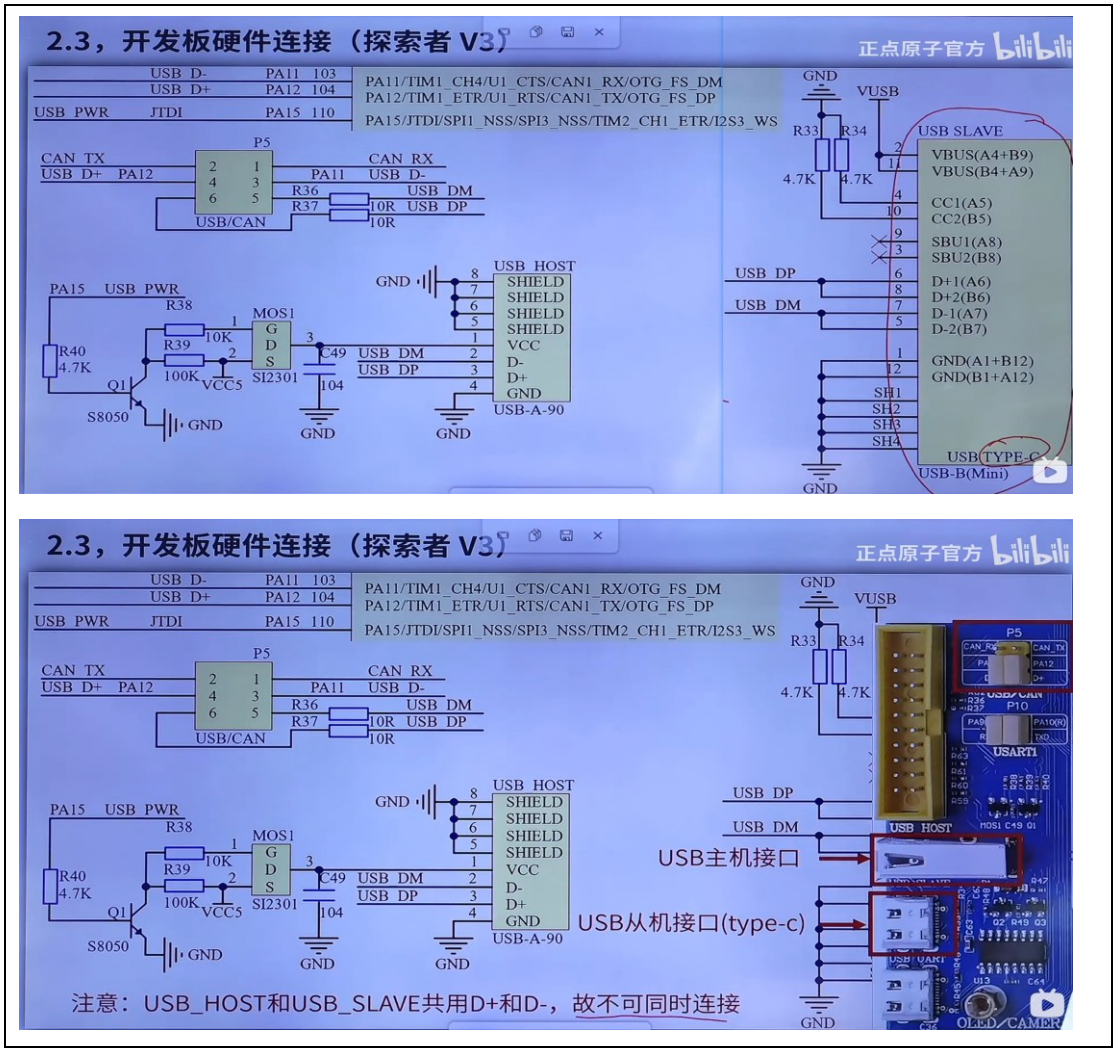






13.2.3 开发板硬件连接





13.2.4 STM32 USB 驱动库简介